

User Manual

for S32K1_S32M24X FLS Driver

Document Number: UM2FLSASR4.4 Rev0000R3.0.0 QLP02 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.2.1 Linear Address	6
3.2.2 Fls Programming Size	8
3.2.3 Fls QuadSPI driver architecture	8
3.2.3.1 High-level overview	8
3.2.3.2 Use cases	13
3.2.3.3 Supported memories	23
3.3 Hardware Resources	23
3.3.1 External flash	23
3.3.2 S32K1_S32M24X Flash Banks/Arrays, Sectors details	24
3.4 Deviations from Requirements	29
3.5 Driver Limitations	30
3.5.1 For internal flash	30
3.5.2 For external flash	30
3.6 Driver usage and configuration tips	30
3.6.1 Introduction	30
3.6.2 Fls Enable Check Configuration CRC	31
3.6.3 Avoiding RWW problem	31
3.6.4 Flash memory physical sectors unlock example	32
3.6.5 Development Error Description	33
3.7 Runtime errors	33
3.8 Symbolic Names Disclaimer	34
3.9 Usage in non AUTOSAR context	34
4 Configuration Parameters	37
4.1 Fls	38
4.1.1 Module Fls	44
4.1.2 Container FlsConfigSet	44
4.1.3 Parameter FlsAcErase	44
4.1.4 Parameter FlsAcWrite	45
4.1.5 Parameter FlsAcErasePointer	45
4.1.6 Parameter FlsAcWritePointer	46
4.1.7 Parameter FlsCallCycle	46
4.1.8 Parameter FlsDefaultMode	47
4.1.9 Parameter FlsACCallback	47
4.1.10 Parameter FlsJobEndNotification	47
4.1.11 Parameter FlsJobErrorNotification	48
4.1.12 Parameter FlsStartFlashAccessNotif	48
4.1.13 Parameter FlsFinishedFlashAccessNotif	49
4.1.14 Parameter FlsReadFunctionCallout	49
4.1.15 Parameter FlsQspiInitCallout	50
4.1.16 Parameter FlsQspiResetCallout	50
4.1.17 Parameter FlsQspiErrorCheckCallout	51

4.1.18	Parameter	FlsQspiEccCheckCallout	52
4.1.19	Parameter	FlsMaxReadFastMode	52
4.1.20	Parameter	FlsMaxReadNormalMode	53
4.1.21	Parameter	FlsMaxWriteFastMode	53
4.1.22	Parameter	FlsMaxWriteNormalMode	54
4.1.23	Parameter	FlsProtection	54
4.1.24	Container	FlsExternalDriver	55
4.1.25	Reference	FlsSpiReference	55
4.1.26	Container	ControllerCfg	56
4.1.27	Parameter	FlsHwUnitReadMode	56
4.1.28	Parameter	FlsSerialFlashA1Size	56
4.1.29	Parameter	FlsSerialFlashA2Size	57
4.1.30	Parameter	FlsSerialFlashB1Size	57
4.1.31	Parameter	FlsSerialFlashB2Size	58
4.1.32	Parameter	FlsHwUnitSamplingModeA	58
4.1.33	Parameter	FlsHwUnitSamplingModeB	59
4.1.34	Parameter	IdleSignalDriveIOFB3HighLvl	59
4.1.35	Parameter	IdleSignalDriveIOFB2HighLvl	60
4.1.36	Parameter	IdleSignalDriveIOFA3HighLvl	60
4.1.37	Parameter	IdleSignalDriveIOFA2HighLvl	60
4.1.38	Parameter	FlsHwUnitSamplingEdge	61
4.1.39	Parameter	FlsHwUnitSamplingDly	61
4.1.40	Parameter	FlsHwUnitDqsLatencyEnable	62
4.1.41	Parameter	FlsHwUnitTdh	62
4.1.42	Parameter	FlsHwUnitTcsh	63
4.1.43	Parameter	FlsHwUnitTcss	63
4.1.44	Parameter	FlsHwUnitColumnAddressWidth	64
4.1.45	Parameter	FlsHwUnitWordAddressable	64
4.1.46	Container	FlsAhbBuffer	65
4.1.47	Parameter	FlsAhbBufferInstance	65
4.1.48	Parameter	FlsAhbBufferMasterId	66
4.1.49	Parameter	FlsAhbBufferSize	66
4.1.50	Parameter	FlsAhbBufferAllMasters	66
4.1.51	Container	FlsHwUnitSpecificSettings	67
4.1.52	Parameter	FlsHwUnitInputClockSelect	67
4.1.53	Parameter	FlsHwUnitInternalRefClockSelect	68
4.1.54	Parameter	FlsHwUnitProgrammableDivider	68
4.1.55	Parameter	FlsHwUnitInvertClkDqsA	69
4.1.56	Parameter	FlsHwUnitInvertClkDqsB	69
4.1.57	Parameter	FlsHwUnitFineDelayA	70
4.1.58	Parameter	FlsHwUnitFineDelayB	70
4.1.59	Container	MemCfg	70
4.1.60	Parameter	MemCfgSize	71
4.1.61	Parameter	MemCfgPageSize	71
4.1.62	Reference	MemCfgReadLUT	72
4.1.63	Reference	MemCfgWriteLUT	72
4.1.64	Reference	MemCfgRead0xxLUT	73
4.1.65	Reference	MemCfgRead0xxLUTAHB	73
4.1.66	Reference	ctrlAutoCfgPtr	73
4.1.67	Container	MemCfgReadIdSettings	74
4.1.68	Parameter	MemCfgReadIdSize	74
4.1.69	Parameter	FlsQspiDeviceId	75
4.1.70	Reference	MemCfgReadIdLUT	75
4.1.71	Container	MemCfgEraseSettings	76

4.1.72	Parameter	MemCfgErase1Size	76
4.1.73	Parameter	MemCfgErase2Size	76
4.1.74	Parameter	MemCfgErase3Size	77
4.1.75	Parameter	MemCfgErase4Size	77
4.1.76	Reference	MemCfgErase1LUT	78
4.1.77	Reference	MemCfgErase2LUT	78
4.1.78	Reference	MemCfgErase3LUT	78
4.1.79	Reference	MemCfgErase4LUT	79
4.1.80	Reference	ChipEraseLUT	79
4.1.81	Container	statusConfig	80
4.1.82	Parameter	regSize	80
4.1.83	Parameter	busyOffset	80
4.1.84	Parameter	busyValue	81
4.1.85	Parameter	writeEnableOffset	81
4.1.86	Parameter	blockProtectionOffset	82
4.1.87	Parameter	blockProtectionWidth	82
4.1.88	Parameter	blockProtectionValue	82
4.1.89	Reference	statusRegInitReadLut	83
4.1.90	Reference	statusRegReadLut	83
4.1.91	Reference	statusRegWriteLut	84
4.1.92	Reference	writeEnableSRLut	84
4.1.93	Reference	writeEnableLut	84
4.1.94	Container	suspendSettings	85
4.1.95	Reference	eraseSuspendLut	85
4.1.96	Reference	eraseResumeLut	86
4.1.97	Reference	programSuspendLut	86
4.1.98	Reference	programResumeLut	86
4.1.99	Container	resetSettings	87
4.1.100	Parameter	resetCmdCount	87
4.1.101	Reference	resetCmdLut	87
4.1.102	Container	initResetSettings	88
4.1.103	Parameter	resetCmdCount	88
4.1.104	Reference	resetCmdLut	89
4.1.105	Container	initConfiguration	89
4.1.106	Parameter	opType	89
4.1.107	Parameter	addr	90
4.1.108	Parameter	size	90
4.1.109	Parameter	shift	91
4.1.110	Parameter	width	91
4.1.111	Parameter	value	92
4.1.112	Reference	command1Lut	92
4.1.113	Reference	command2Lut	93
4.1.114	Reference	weLut	93
4.1.115	Reference	ctrlCfgPtr	93
4.1.116	Container	FlsLUT	94
4.1.117	Parameter	FlsLUTIndex	94
4.1.118	Container	FlsInstructionOperandPair	95
4.1.119	Parameter	FlsInstrOperPairIndex	95
4.1.120	Parameter	FlsLUTInstruction	96
4.1.121	Parameter	FlsLUTPad	96
4.1.122	Parameter	FlsLUTOperand	97
4.1.123	Container	FlsController	97
4.1.124	Parameter	ControllerName	98
4.1.125	Reference	FlsControllerCfgRef	98

4.1.126	Container FlsMem	98
4.1.127	Parameter FlsMemName	99
4.1.128	Parameter MemAlignment	99
4.1.129	Parameter AHBReadEnable	100
4.1.130	Parameter FlsMemUseSfdp	100
4.1.131	Parameter connectionType	101
4.1.132	Reference FlsMemCfgRef	101
4.1.133	Reference qspiInstance	101
4.1.134	Container FlsSectorList	102
4.1.135	Container FlsSector	102
4.1.136	Parameter FlsSectorIndex	103
4.1.137	Parameter FlsPhysicalSector	103
4.1.138	Parameter FlsNumberOfSectors	105
4.1.139	Parameter FlsPageSize	105
4.1.140	Parameter FlsSectorSize	106
4.1.141	Parameter FlsSectorStartaddress	106
4.1.142	Parameter FlsSectorEraseAsynch	106
4.1.143	Parameter FlsPageWriteAsynch	107
4.1.144	Parameter FlsHwCh	107
4.1.145	Parameter FlsSectorHwAddress	108
4.1.146	Reference flashInstance	108
4.1.147	Container AutosarExt	109
4.1.148	Parameter FlsEnableUserModeSupport	109
4.1.149	Parameter FlsQspiLockLUT	110
4.1.150	Parameter FlsQspiHangRecovery	110
4.1.151	Parameter FlsSynchronizeCache	111
4.1.152	Parameter FlsInvalidPrefetchBufFromRam	111
4.1.153	Parameter FlsInternalSectorsConfigured	112
4.1.154	Parameter FlsExternalSectorsConfigured	112
4.1.155	Container FlsGeneral	113
4.1.156	Parameter FlsEnableDevAssert	113
4.1.157	Parameter FlsEnableCheckCfgCrc	113
4.1.158	Parameter FlsAcLoadOnJobStart	114
4.1.159	Parameter FlsCleanCacheAfterLoadAc	114
4.1.160	Parameter FlsBaseAddress	115
4.1.161	Parameter FlsBlankCheckApi	115
4.1.162	Parameter FlsCancelApi	116
4.1.163	Parameter FlsCompareApi	116
4.1.164	Parameter FlsDevErrorDetect	117
4.1.165	Parameter FlsDriverIndex	117
4.1.166	Parameter FlsGetJobResultApi	117
4.1.167	Parameter FlsGetStatusApi	118
4.1.168	Parameter FlsSetModeApi	118
4.1.169	Parameter FlsTotalSize	119
4.1.170	Parameter FlsUseInterrupts	119
4.1.171	Parameter FlsVersionInfoApi	120
4.1.172	Parameter FlsECCCheck	120
4.1.173	Parameter FlsECCHandlingProtectionHook	121
4.1.174	Parameter FlsEraseVerificationEnabled	121
4.1.175	Parameter FlsWriteVerificationEnabled	121
4.1.176	Parameter FlsMaxEraseBlankCheck	122
4.1.177	Parameter FlsTimeoutSupervisionEnabled	122
4.1.178	Parameter FlsTimeoutMethod	123
4.1.179	Parameter FlsAsyncWriteTimeout	123

4.1.180	Parameter	FlsAsyncEraseTimeout	124
4.1.181	Parameter	FlsSyncWriteTimeout	124
4.1.182	Parameter	FlsSyncEraseTimeout	125
4.1.183	Parameter	FlsAbortTimeout	125
4.1.184	Parameter	FlsQspiIpTimeoutOsifCounterType	126
4.1.185	Parameter	FlsQspiSyncReadTimeout	126
4.1.186	Parameter	FlsQspiAsyncWriteTimeout	127
4.1.187	Parameter	FlsQspiAsyncEraseTimeout	127
4.1.188	Parameter	FlsQspiSyncWriteTimeout	127
4.1.189	Parameter	FlsQspiSyncEraseTimeout	128
4.1.190	Parameter	FlsQspiCommandCompleteTimeout	128
4.1.191	Parameter	FlsQspiResetTimeout	129
4.1.192	Parameter	FlsQspiFlashInitTimeout	129
4.1.193	Parameter	FlsQspiSoftwareResetDelay	130
4.1.194	Parameter	FlsQspiTxBufferResetDelay	130
4.1.195	Parameter	FlsQspiWriteEnableRetries	131
4.1.196	Reference	FlsEcucPartitionRef	131
4.1.197	Container	FlsPublishedInformation	131
4.1.198	Parameter	FlsAcLocationErase	132
4.1.199	Parameter	FlsAcLocationWrite	132
4.1.200	Parameter	FlsAcSizeErase	133
4.1.201	Parameter	FlsAcSizeWrite	133
4.1.202	Parameter	FlsEraseTime	133
4.1.203	Parameter	FlsErasedValue	134
4.1.204	Parameter	FlsECCValue	134
4.1.205	Parameter	FlsExpectedHwId	135
4.1.206	Parameter	FlsSpecifiedEraseCycles	135
4.1.207	Parameter	FlsWriteTime	136
4.1.208	Container	CommonPublishedInformation	136
4.1.209	Parameter	ArReleaseMajorVersion	136
4.1.210	Parameter	ArReleaseMinorVersion	137
4.1.211	Parameter	ArReleaseRevisionVersion	137
4.1.212	Parameter	ModuleId	138
4.1.213	Parameter	SwMajorVersion	138
4.1.214	Parameter	SwMinorVersion	139
4.1.215	Parameter	SwPatchVersion	139
4.1.216	Parameter	VendorApiInfix	139
4.1.217	Parameter	VendorId	140
4.2	FTFC_IP		140
4.2.1	Module	Ftfc_Ip	141
4.2.2	Parameter	IpDevErrorDetect	141
4.3	QSPI_IP		142
4.3.1	Module	Qspi_Ip	143
4.3.2	Parameter	IpDevErrorDetect	143
5	Module Index		144
5.1	Software Specification		144
6	Module Documentation		145
6.1	FLS Driver		145
6.1.1	Detailed Description		145
6.1.2	Data Structure Documentation		148
6.1.2.1	struct	Fls_Flash_InternalSectorInfoType	148
6.1.2.2	struct	Fls_QspiCfgConfigType	149

6.1.2.3 struct Fls_ConfigType	151
6.1.3 Macro Definition Documentation	156
6.1.3.1 FLS_DEVICE_INSTANCE_INVALID	156
6.1.3.2 FLS_API_VENDOR_ID	156
6.1.3.3 FLS_MODULE_ID	156
6.1.3.4 FLS_INSTANCE_ID	156
6.1.3.5 FLS_E_PARAM_CONFIG	157
6.1.3.6 FLS_E_PARAM_ADDRESS	157
6.1.3.7 FLS_E_PARAM_LENGTH	157
6.1.3.8 FLS_E_PARAM_DATA	157
6.1.3.9 FLS_E_UNINIT	157
6.1.3.10 FLS_E_BUSY	157
6.1.3.11 FLS_E_PARAM_POINTER	158
6.1.3.12 FLS_E_VERIFY_ERASE_FAILED	158
6.1.3.13 FLS_E_VERIFY_WRITE_FAILED	158
6.1.3.14 FLS_E_TIMEOUT	158
6.1.3.15 FLS_E_ERASE_FAILED	158
6.1.3.16 FLS_E_WRITE_FAILED	158
6.1.3.17 FLS_E_READ_FAILED	159
6.1.3.18 FLS_E_COMPARE_FAILED	159
6.1.3.19 FLS_INIT_ID	159
6.1.3.20 FLS_ERASE_ID	159
6.1.3.21 FLS_WRITE_ID	159
6.1.3.22 FLS_CANCEL_ID	159
6.1.3.23 FLS_GETJOBRESULT_ID	160
6.1.3.24 FLS_MAINFUNCTION_ID	160
6.1.3.25 FLS_READ_ID	160
6.1.3.26 FLS_COMPARE_ID	160
6.1.3.27 FLS_SETMODE_ID	160
6.1.3.28 FLS_GETVERSIONINFO_ID	160
6.1.3.29 FLS_BLANK_CHECK_ID	161
6.1.3.30 FLS_SECTOR_ERASE_ASYNC	161
6.1.3.31 FLS_PAGE_WRITE_ASYNC	161
6.1.3.32 FLS_START_SEC_CODE	161
6.1.3.33 FLS_STOP_SEC_CODE	161
6.1.3.34 FLS_IPW_CFG_INVALID	161
6.1.4 Types Reference	161
6.1.4.1 Fls_SectorIndexType	162
6.1.4.2 Fls_AddressType	162
6.1.4.3 Fls_LengthType	162
6.1.4.4 Fls_SectorCountType	162
6.1.4.5 Fls_BlockNumberOfSectorType	162
6.1.4.6 Fls_InternalConfigType	162
6.1.4.7 Fls_JobEndNotificationPtrType	163
6.1.4.8 Fls_JobErrorNotificationPtrType	163
6.1.4.9 Fls_ACCallbackPtrType	163
6.1.4.10 Fls_AcErasePtrType	163
6.1.4.11 Fls_AcWritePtrType	163
6.1.5 Enum Reference	163
6.1.5.1 Fls_HwChType	164
6.1.5.2 Fls_JobType	164
6.1.5.3 Fls_LLDReturnTypes	164
6.1.5.4 Fls_LLDJobType	164
6.1.5.5 Fls_CrcDataSizeType	165

6.1.6 Function Reference	165
6.1.6.1 Fls_Init()	165
6.1.6.2 Fls_Write()	165
6.1.6.3 Fls_Erase()	166
6.1.6.4 Fls_Read()	167
6.1.7 Variable Documentation	167
6.1.7.1 Fls_u32JobAddrIt	167
6.1.7.2 Fls_u32JobSectorIt	167
6.1.7.3 Fls_u32JobSectorEnd	168
6.1.7.4 Fls_eLLDJobResult	168
6.1.7.5 Fls_eLLDJob	168
6.1.7.6 Fls_pConfigPtr	168
6.2 QSPI IPV Driver	169
6.2.1 Detailed Description	169
6.2.2 Data Structure Documentation	173
6.2.2.1 struct Qspi_Ip_StateType	173
6.2.2.2 struct Qspi_Ip_ControllerAhbConfigType	173
6.2.2.3 struct Qspi_Ip_ControllerConfigType	173
6.2.2.4 struct Qspi_Ip_StatusConfigType	175
6.2.2.5 struct Qspi_Ip_EraseVarConfigType	175
6.2.2.6 struct Qspi_Ip_EraseConfigType	176
6.2.2.7 struct Qspi_Ip_ReadIdConfigType	176
6.2.2.8 struct Qspi_Ip_SuspendConfigType	176
6.2.2.9 struct Qspi_Ip_ResetConfigType	177
6.2.2.10 struct Qspi_Ip_LutConfigType	177
6.2.2.11 struct Qspi_Ip_InitOperationType	177
6.2.2.12 struct Qspi_Ip_InitConfigType	178
6.2.2.13 struct Qspi_Ip_MemoryConfigType	178
6.2.2.14 struct Qspi_Ip_MemoryConnectionType	179
6.2.3 Macro Definition Documentation	180
6.2.3.1 QSPI_IP_MAX_READ_SIZE	180
6.2.3.2 QSPI_IP_MAX_WRITE_SIZE	180
6.2.3.3 QSPI_IP_ERASE_TYPES	180
6.2.3.4 QSPI_IP_AHB_BUFFERS	180
6.2.3.5 QSPI_IP_LUT_INVALID	180
6.2.3.6 QSPI_IP_LUT_SEQ_END	180
6.2.3.7 QSPI_IP_PACK_LUT_REG	181
6.2.4 Types Reference	181
6.2.4.1 Qspi_Ip_InstrOpType	181
6.2.4.2 Qspi_Ip_InitCalloutPtrType	181
6.2.4.3 Qspi_Ip_ResetCalloutPtrType	181
6.2.4.4 Qspi_Ip_ErrorCheckCalloutPtrType	181
6.2.4.5 Qspi_Ip_EccCheckCalloutPtrType	182
6.2.5 Enum Reference	182
6.2.5.1 Qspi_Ip_StatusType	182
6.2.5.2 Qspi_Ip_ConnectionType	182
6.2.5.3 Qspi_Ip_OpType	182
6.2.5.4 Qspi_Ip_LutCommandsType	183
6.2.5.5 Qspi_Ip_LutPadsType	184
6.2.5.6 Qspi_Ip_ReadModeType	184
6.2.5.7 Qspi_Ip_DataRateType	185
6.2.5.8 Qspi_Ip_SampleDelayType	185
6.2.5.9 Qspi_Ip_SamplePhaseType	185
6.2.5.10 Qspi_Ip_FlashDataAlignType	186

6.2.5.11 Qspi_Ip_ClockSourceType	186
6.2.5.12 Qspi_Ip_ClockReferenceType	186
6.2.5.13 Qspi_Ip_LastCommandType	187
6.2.5.14 Qspi_Ip_FlashMemoryType	187
6.2.6 Function Reference	187
6.2.6.1 Qspi_Ip_Init()	187
6.2.6.2 Qspi_Ip_Deinit()	187
6.2.6.3 Qspi_Ip_EraseBlock()	188
6.2.6.4 Qspi_Ip_EraseChip()	188
6.2.6.5 Qspi_Ip_GetMemoryStatus()	188
6.2.6.6 Qspi_Ip_SetProtection()	189
6.2.6.7 Qspi_Ip_GetProtection()	189
6.2.6.8 Qspi_Ip_Reset()	189
6.2.6.9 Qspi_Ip_Enter0XX()	190
6.2.6.10 Qspi_Ip_Exit0XX()	190
6.2.6.11 Qspi_Ip_ProgramSuspend()	190
6.2.6.12 Qspi_Ip_ProgramResume()	190
6.2.6.13 Qspi_Ip_EraseSuspend()	191
6.2.6.14 Qspi_Ip_EraseResume()	191
6.2.6.15 Qspi_Ip_Read()	191
6.2.6.16 Qspi_Ip_ReadId()	192
6.2.6.17 Qspi_Ip_ProgramVerify()	192
6.2.6.18 Qspi_Ip_EraseVerify()	193
6.2.6.19 Qspi_Ip_Program()	193
6.2.6.20 Qspi_Ip_RunCommand()	193
6.2.6.21 Qspi_Ip_RunReadCommand()	194
6.2.6.22 Qspi_Ip_RunWriteCommand()	194
6.2.6.23 Qspi_Ip_AhbReadEnable()	195
6.2.6.24 Qspi_Ip_ControllerGetStatus()	195
6.2.6.25 Qspi_Ip_ControllerInit()	195
6.2.6.26 Qspi_Ip_ControllerDeinit()	196
6.2.6.27 Qspi_Ip_Abort()	196
6.2.6.28 Qspi_Ip_ReadSfdp()	197
6.2.6.29 Qspi_Ip_SetLut()	197
6.2.6.30 Qspi_Ip_WriteLuts_Privileged()	197
6.2.6.31 Qspi_Ip_SetAhbSeqId_Privileged()	198
6.2.6.32 Qspi_Ip_GetBaseAddress()	198
6.2.6.33 Qspi_Ip_IpCommand()	198
6.2.6.34 Qspi_Ip_IpRead()	199
6.2.6.35 Qspi_Ip_IpWrite()	199
6.3 FTFC IP Driver	201
6.3.1 Detailed Description	201
6.3.2 Data Structure Documentation	202
6.3.2.1 struct Fls_ExceptionDetailsType	202
6.3.2.2 struct Ftfc_ConfigType	202
6.3.3 Macro Definition Documentation	203
6.3.3.1 FLASH_CMD_PROGRAM_PHRASE	203
6.3.3.2 FTFC_WRITE_DOUBLE_WORD	203
6.3.3.3 FLS_SIZE_1BYTE	203
6.3.3.4 FLS_SIZE_2BYTE	203
6.3.3.5 FLS_SIZE_4BYTE	203
6.3.4 Types Reference	204
6.3.4.1 Fls_CompHandlerReturntype	204
6.3.4.2 Ftfc_StartFlashAccessNotifPtrType	204

6.3.4.3 Ftfc_FinishedFlashAccessNotifPtrType	204
6.3.5 Enum Reference	204
6.3.5.1 Ftfc_Fls_Ip_FlashBlocksNumberType	204
6.3.5.2 Ftfc_Fls_Ip_StatusType	205
6.3.6 Function Reference	205
6.3.6.1 Ftfc_Fls_Ip_Init()	205
6.3.6.2 Ftfc_Fls_Ip_Abort()	206
6.3.6.3 Ftfc_Fls_Ip_Read()	206
6.3.6.4 Ftfc_Fls_Ip_Compare()	207
6.3.6.5 Ftfc_Fls_Ip_GetBlockNumberFromAddress()	207
6.3.6.6 Ftfc_Fls_Ip_SectorErase()	208
6.3.6.7 Ftfc_Fls_Ip_SectorEraseStatus()	208
6.3.6.8 Ftfc_Fls_Ip_Write()	209
6.3.6.9 Ftfc_Fls_Ip_WriteStatus()	210
6.3.6.10 Ftfc_Fls_Ip_DsiHandler()	210
6.3.6.11 Ftfc_Fls_Ip_InvalidPrefetchBuff_Ram()	211

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.05.2025	NXP RTD Team	S32K1_ S32M24X Real-Time Drivers AUTOSAR 4.4 Version 3.↵ 0.0 QLP02

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR FLS for S32K1_S32M24X. AUTOSAR FLS driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR FLS driver requirements and APIs are described in the AUTOSAR FLS driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64

- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DET	Default Error Tracer
ECC	Error Correcting Code
VLE	Variable Length Encoding
N/A	Not Available
MCU	Microcontroller Unit
ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable Read-Only Memory
FEE	Flash EEPROM Emulation
FLS	Flash
RTD	Real Time Drivers
SWS	Software Specification
XML	Extensible Markup Language
RAM	Random Access Memory
QSPI	Quad Serial Peripheral Interface
SFDP	Serial Flash Discoverable Parameters
SDR	Single Data Rate
DDR	Double Data Rate
DTR	Double Transfer Rate
CRC	Cyclic Redundancy Check

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release 4.4
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

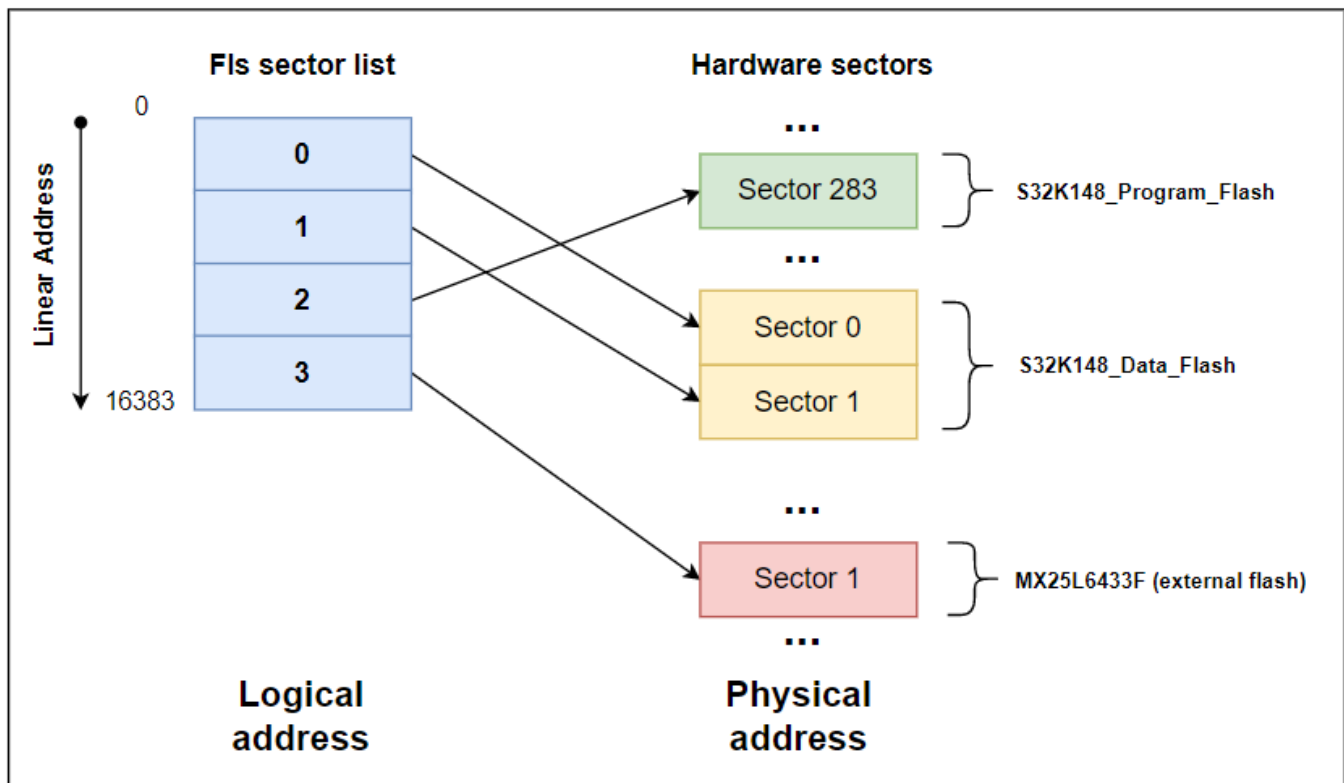
3.2 Driver Design Summary

3.2.1 Linear Address

- The FLS driver provides services for reading, writing and erasing flash memory and it combines configured flash memory sectors into one linear address space. The FLS module shall combine all available flash memory areas into one linear address space, it will always start at address 0 and continues without any gap.
- Sectors details Example, suppose user wants to configure following sectors:

FlsPhysicalSector	Fls Physical Start Address	Fls Sector Size
FlsSector_0	0 (0x0000)	4096 (0x1000)
FlsSector_1	4096 (0x1000)	4096 (0x1000)
FlsSector_2	8192 (0x2000)	4096 (0x1000)
FlsSector_3	12288 (0x3000)	4096 (0x1000)

The layout of configured sectors:



The FIsSector List should be configured in the following way:

FIsSector									
Index	Name	FIs Sector Index	FIs Physical Sector	FIs Number Of Sector	FIs Page Size	FIs Sector Size	FIs Sector Start Address		
0	FIsSector_0	0	FLS_DATA_ARRAY_0_BLOCK_3_S000	1	8	4096	0		
1	FIsSector_1	1	FLS_DATA_ARRAY_0_BLOCK_3_S001	1	8	4096	4096		
2	FIsSector_2	2	FLS_CODE_ARRAY_0_BLOCK_2_S383	1	8	4096	8192		
3	FIsSector_3	3	FLS_EXT_SECTOR	1	8	4096	12288		

FIs Sector Erase Asynch	FIs Page Write Asynch	FIs Hardware Channel	FIs Qspi Sector Channel	FIs Sector Hardware Byte Address
☒	☐	FLS_CH_INTERN	NOT_QSPI_CHANNEL	0x0
☐	☐	FLS_CH_INTERN	NOT_QSPI_CHANNEL	0x0
☐	☐	FLS_CH_INTERN	NOT_QSPI_CHANNEL	0x0
☐	☐	FLS_CH_QSPI	FLS_CH_EXTERN_QSPI_0_A1	0x1000

- As you can see FIs Sector Start Address for FIsSector_0 will be 0x0000 and FIs Sector Start Address for FIsSector_1 will be 0x1000 (4096) and so on.
- If user want to write FIsSector_1, user need to write to the logical address 0x1000 - 0x1FFF.
- If user want to erase it, user need to erase sector from address 0x1000 with size 0x1000.

Note

- The users do not need to calculate the FIs Sector Start Address and FIs Sector Size they can be automatically computed.
- If the microcontroller is in user mode, be sure that the Flash memory controller registers are accessible. For more information please refer to the 'Memory Protection Unit' and 'Register Protection' chapters in the device reference manual

- Care should be taken if using the first program flash sector (FLS_CODE_ARRAY_0_BLOCK_0_S000), as that sector contains the flash configuration field.
 - It is recommended to avoid using this sector, because the flash configuration field region has a default value different than the default erased value.
 - Writing in improper value into the flash configuration field region (0x0_0400 - 0x0_040F) might lead to protected sectors, secured device or permanently secured device.
 - An example of unprotecting the internal flash sectors is provided in the 'Driver usage and configuration tips' chapter.

3.2.2 Fls Programming Size

- Every write access to the underlying hardware memory will be done by using writes that are as big as allowed on the hardware
 - Internal flash: program phrase command of Flash Memory Module FTFx (8 bytes)
 - External flash: maximum of QSPI Tx buffer size (128 bytes)
 - [Fls QuadSPI driver architecture](#)

3.2.3 Fls QuadSPI driver architecture

This section describes the detail for a high-level overview of QuadSPI components in Fls driver, how they interact and how the driver should be used.

Table of content:

- High-level overview
- Use cases
- Supported memories

Related information:

- Clocking and IOMUX for QuadSPI (chapter "3.3 Hardware Resources" in User Manual)
- QuadSPI in multicore context (if supported by the platform, chapter "5.8 Multicore support" in Integration Manual)
- QuadSPI external memory assumptions (chapter "9 External assumptions for driver" in Integration Manual)

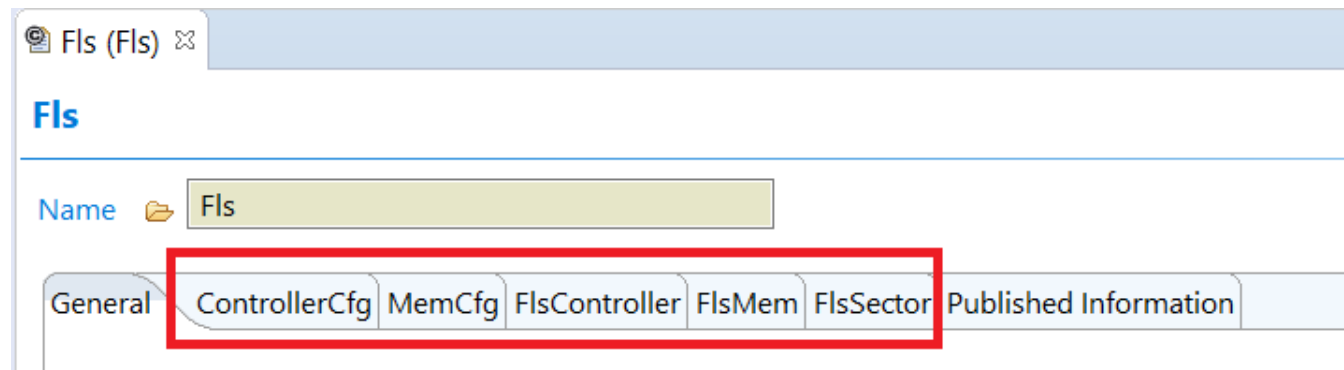
3.2.3.1 High-level overview

This sub-chapter describes the the architecture of the driver:

- The interaction between the HLD, Controller and Memory components
- What each part does and how they interact
- Examples of configuration

QuadSPI components in Fls driver

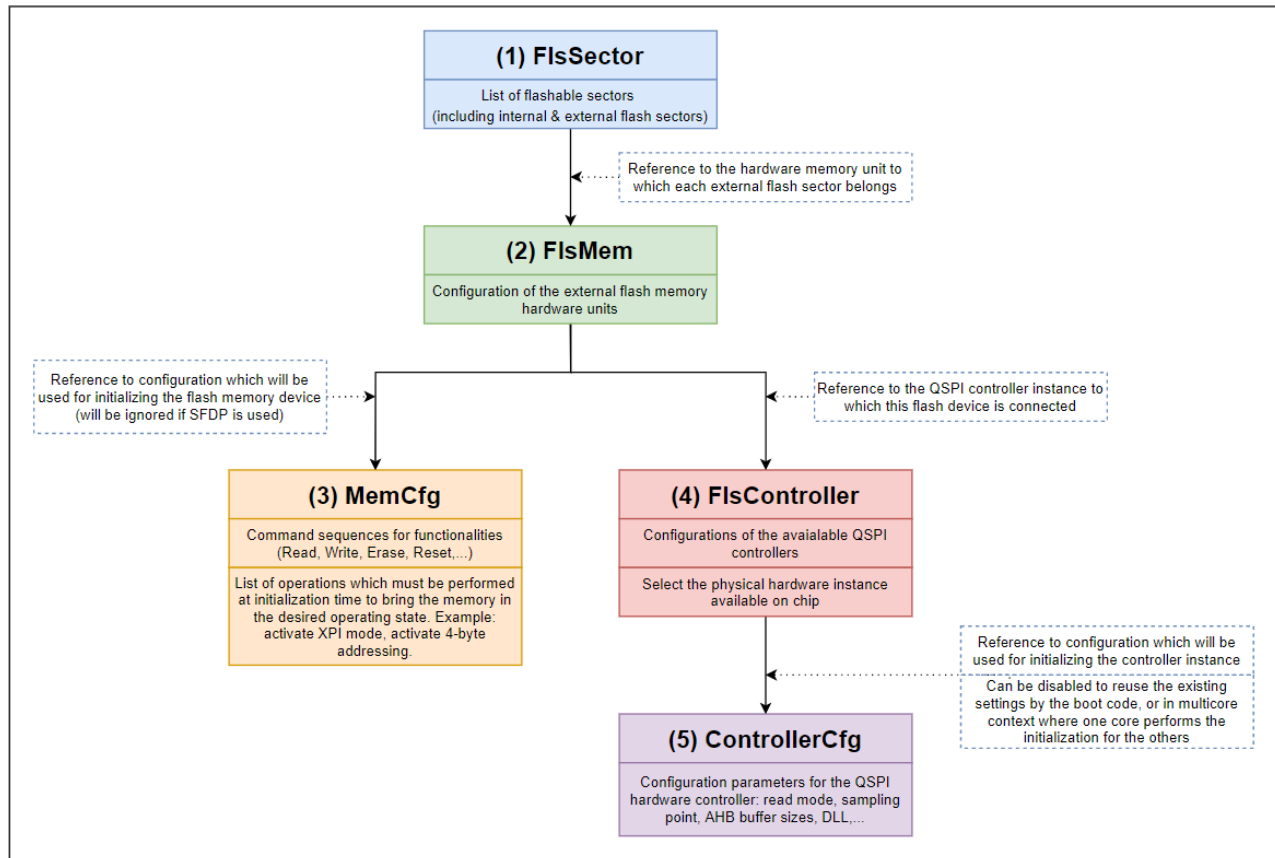
QuadSPI components user interface:



There are five components connect together in the order below:

#	Device name
1	FlsSector
2	FlsMem
3	MemCfg
4	FlsController
5	ControllerCfg

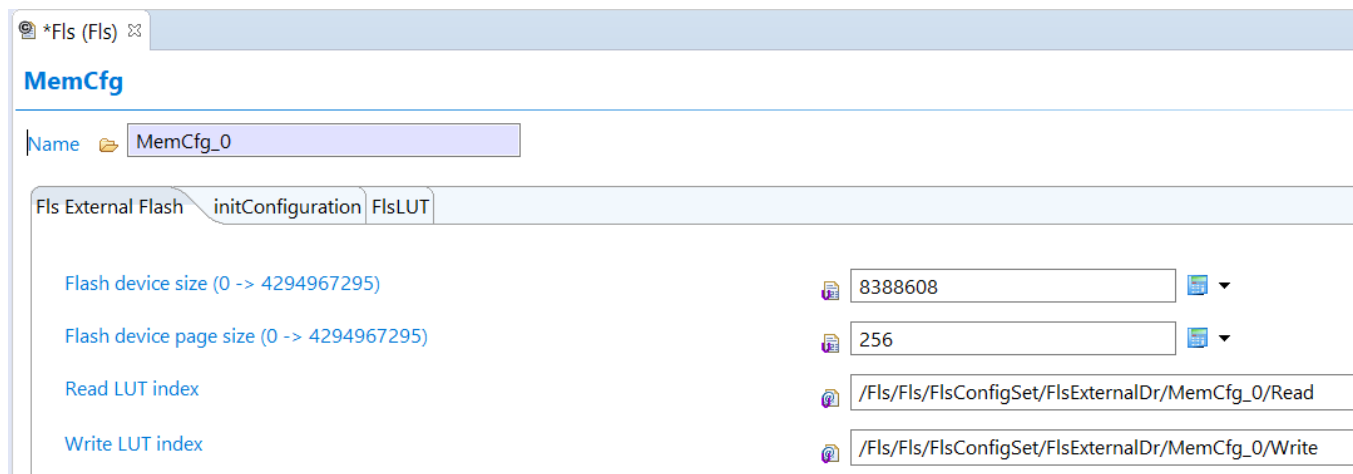
Connections between components:



MemCfg

This container contains all specific settings for the memory device:

- Memory characteristics: device size, page size
- LUT command sequences for basic functionality



It also provides a list of operations (**initConfiguration**) which must be performed at initialization time to bring

the memory in the desired operating state (for example: setting registers value). Here is an example of an operation to enable the Quad mode by set bit 6th in the Status register:

***Fls (Fls)**

initConfiguration

Name*

Initial device configuration

Operation type	QSPI_IP_OP_TYPE_RMW_REG
First LUT index	/Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0/ReadStatusRegister
Second LUT index	/Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0/WriteStatusRegister
Write Enable LUT index	/Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0/WriteEnable
Command address (0 -> 4294967295)	0
Register size (1 -> 4)	1
Bit-field offset (0 -> 32)	6
Bit-field width (0 -> 32)	1
Bit-field value (0 -> 4294967295)	1
Controller configuration	

In this example, QuadSPI driver will:

- Read 1 byte value of the status register by using the **First LUT index**
- Modify the **6th** bit to the desired value is **1**
- If needed, the **Write Enable LUT index** will be issued before a write command
- Write back that byte value to memory device by using the **Second LUT index**

Note

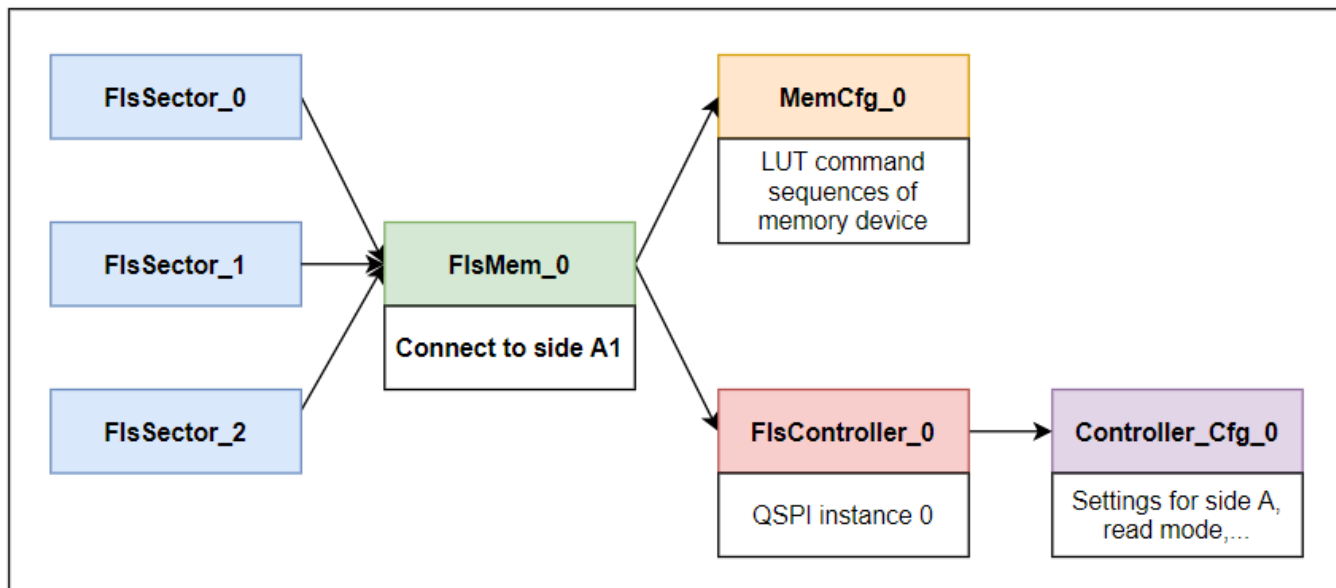
- When changing the value of non-volatile bits, users need to insert one more read operation (QSPI_IP←_OP_TYPE_READ_REG) right behind to wait for the write operation to complete

Examples of configuration

Example 1

Below is the diagram to depict the example from the section "3.2.1 Linear Address". Assume that the memory device connects to the side A1 of QuadSPI controller, we need:

- 01 hardware memory unit (reference to MemCfg_0): contains the LUT command set for initializing the memory device
- 01 controller configuration set (ControllerCfg_0): contains the configuration parameters for QuadSPI hardware instance 0



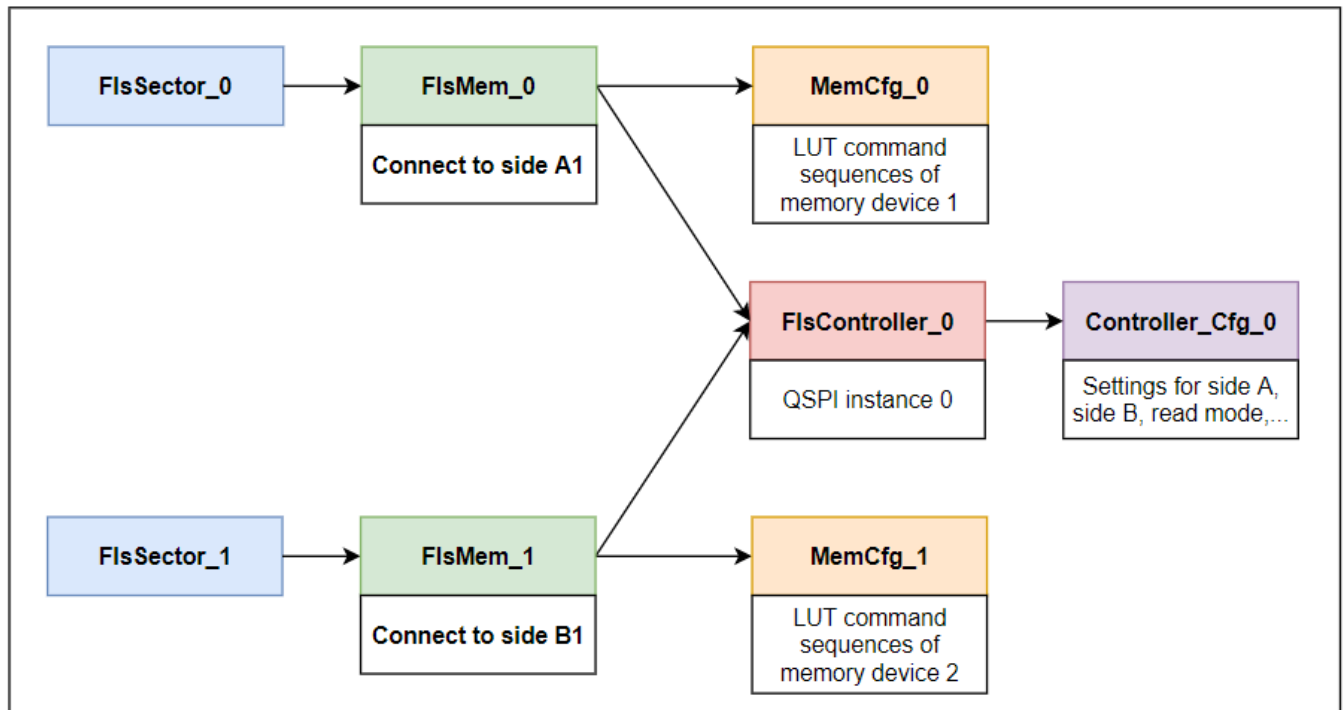
Example 2

Suppose we have 02 different external flash memory devices, one connects to side A and one connects to side B of the QuadSPI controller. And we want to configure 02 external sectors, the first sector in each memory device

FlsSectorList	Fls Sector Size	Fls Number Of Sector	Fls Logical Start Address	Fls Hardware Start Address	Fls Hardware Memory Unit
FlsSector_0	4096 (0x1000)	1	0 (0x0000)	0 (0x0000)	FlsMem_0
FlsSector_1	4096 (0x1000)	1	4096 (0x1000)	0 (0x0000)	FlsMem_1

In this example, we need:

- 02 hardware memory unit (reference to MemCfg_0 and MemCfg_1): contain the LUT command sets for initializing each memory device (if they have different command sets)
- 01 controller configuration set (ControllerCfg_0): contains the configuration parameters for QuadSPI hardware instance 0



As you can see:

- Any operations on the `FlsSector_0` will be mapped to the hardware sector 0 (0x0000 - 0xFFFF) of the memory device 1
- Any operations on the `FlsSector_1` will be mapped to the hardware sector 0 (0x0000 - 0xFFFF) of the memory device 2
- The QuadSPI controller communicates with each memory device by using the commands set from the corresponding **MemCfg**

3.2.3.2 Use cases

This sub-chapter provides various useful practical examples.

Software reset

The driver provides two ways to use the software reset command, for resetting the flash device:

- Software Reset (used by `Fls_Cancel()` and `Qspi_Ip_Reset()`, at any time during runtime)
- Initial Software Reset (used by `Fls_Init()`, only one time)

Driver

The screenshot displays the MemCfg tool interface for configuring the FLS driver. It shows two main sections: 'Software Reset' and 'Initial Software Reset'. Both sections have a 'Reset LUT index*' dropdown menu set to 2. Red arrows indicate the flow from these settings to the corresponding FLS LUT configuration windows. The 'RuntimeReset' LUT window shows a sequence of three instructions: QSPI_IP_LUT_INSTR_CMD, QSPI_IP_LUT_INSTR_STOP, and QSPI_IP_LUT_INSTR_CMD, with corresponding pads and operands. The 'InitReset' LUT window shows a similar sequence, but the first instruction is QSPI_IP_LUT_INSTR_CMD and the last is QSPI_IP_LUT_INSTR_CMD, with the last operand set to 0x99.

Note

- The number of reset commands is the number of sequences needed by the reset operation, separated by a stop phase
- A stop phase will be inserted automatically at the end of each command sequence

The **Initial Software Reset** procedure applies only at driver initialization. It might be different from the normal reset command, depending on the initial state of the flash. If not, set the same as reset command. In the above example, the memory device works in quad mode (4S-4S-4S), hence we need the reset commands in quad mode.

- The **Initial Software Reset** feature is useful in case we do not know the current state of the device memory (for example when bootrom leaves the memory in a certain state), and we need a reset sequence to bring it to the default state before performing initialization.

Here is an example of a combination of reset command sets (in three modes: SPI, QPI and OPI) to force memory device into its default state:

Initial Software Reset

Name

Reset LUT index

Number of reset commands (1 -> 255)

Fls (Fls)

FlsLUT

Name

Fls LUT FlsInstructionOperandPair

FlsInstructionOperandPair

Index	Name	Fls Instruction Operand Pair Index	Fls LUT Instruction	Fls LUT Pad	Fls LUT Operand
0	FlsInstructionOperandPair_0	0	QSPI_IP_LUT_INSTR_CMD	QSPI_IP_LUT_PADS_1	0x66
1	FlsInstructionOperandPair_1	1	QSPI_IP_LUT_INSTR_STOP	QSPI_IP_LUT_PADS_1	0x0
2	FlsInstructionOperandPair_2	2	QSPI_IP_LUT_INSTR_CMD	QSPI_IP_LUT_PADS_1	0x99
3	FlsInstructionOperandPair_3	3	QSPI_IP_LUT_INSTR_STOP	QSPI_IP_LUT_PADS_1	0x0
4	FlsInstructionOperandPair_4	4	QSPI_IP_LUT_INSTR_CMD	QSPI_IP_LUT_PADS_4	0x66
5	FlsInstructionOperandPair_5	5	QSPI_IP_LUT_INSTR_STOP	QSPI_IP_LUT_PADS_4	0x0
6	FlsInstructionOperandPair_6	6	QSPI_IP_LUT_INSTR_CMD	QSPI_IP_LUT_PADS_4	0x99
7	FlsInstructionOperandPair_7	7	QSPI_IP_LUT_INSTR_STOP	QSPI_IP_LUT_PADS_4	0x0
8	FlsInstructionOperandPair_8	8	QSPI_IP_LUT_INSTR_CMD	QSPI_IP_LUT_PADS_8	0x66
9	FlsInstructionOperandPair_9	9	QSPI_IP_LUT_INSTR_CMD	QSPI_IP_LUT_PADS_8	0x99
10	FlsInstructionOperandPair_10	10	QSPI_IP_LUT_INSTR_STOP	QSPI_IP_LUT_PADS_8	0x0
11	FlsInstructionOperandPair_11	11	QSPI_IP_LUT_INSTR_CMD	QSPI_IP_LUT_PADS_8	0x99
12	FlsInstructionOperandPair_12	12	QSPI_IP_LUT_INSTR_CMD	QSPI_IP_LUT_PADS_8	0x66

Note

- Fls_Cancel() will use **Software Reset** to abort the on-going write/erase operation
- This action causes loss of synchronization between QuadSPI controller and memory device (for example when working in DOPI mode with external DQS)
- The next section will describe the solution to deal with this situation

Controller configuration

There are several controller configuration points in the driver:

#	Controller configuration point	Location	Description
1	Controller initial configuration	FlsController	Configure QuadSPI controller
2	Controller configuration	MemCfg (operations list)	Re-configure QuadSPI controller during memory device initialization E.g: switch the controller to External DQS after activating DOPI mode
3	Configure controller on flash Init	MemCfg	Re-configure QuadSPI controller when resetting the memory device E.g: switch the controller to initial configuration before re-init the memory device

Driver

Controller initial configuration

This is the first initialization point which will be done once by `Fls_Init()`:

Fls (Fls) ✕

FlsController

Name  FlsController_0

Fls controller

Fls Controller Name  FLS_QSPI_0  

Controller initial configuration  /Fls/Fls/FlsConfigSet/FlsExternalDr/ControllerCfg_0_SDR_DQSLoopBack

This step can be skipped by leaving the reference node blank, the purpose is:

- Reuse the existing QuadSPI settings by the boot code
- Or in multicore context where one core performs the initialization for the others

Fls (Fls) ✕

FlsController

Name  FlsController_0

Fls controller

Fls Controller Name  FLS_QSPI_0  

Controller initial configuration  

Controller configuration

The second initialization point is in the list of operations of **MemCfg**. This step is needed after we configure the device memory to another mode that is no longer compatible with the controller configuration point #1 (E.g: after activating DOPI mode)

Fls (Fls) 

initConfiguration

Name  ext_dqs

Initial device configuration

Operation type  QSPI_IP_OP_TYPE_QSPI_CFG 

First LUT index 

Second LUT index 

Write Enable LUT index 

Command address (0 -> 4294967295)  0

Register size (1 -> 4)  1

Bit-field offset (0 -> 32)  0

Bit-field width (0 -> 32)  0

Bit-field value (0 -> 4294967295)  1

Controller configuration  /Fls/Fls/FlsConfigSet/FlsExternalDr/ControllerCfg_1_DDR_ExternalDQS

Configure controller on flash Init

The third configuration point is at the end of **MemCfg**. This step is needed when resetting the memory device, then we have to re-configure the controller to a mode that is compatible with the new state of memory device after reset. (E.g: when executing the reset sequence in DOPI mode)

Fls (Fls) 

MemCfg

Name  MemCfg_0

Fls External Flash  initConfiguration  FlsLUT

Initial Software Reset

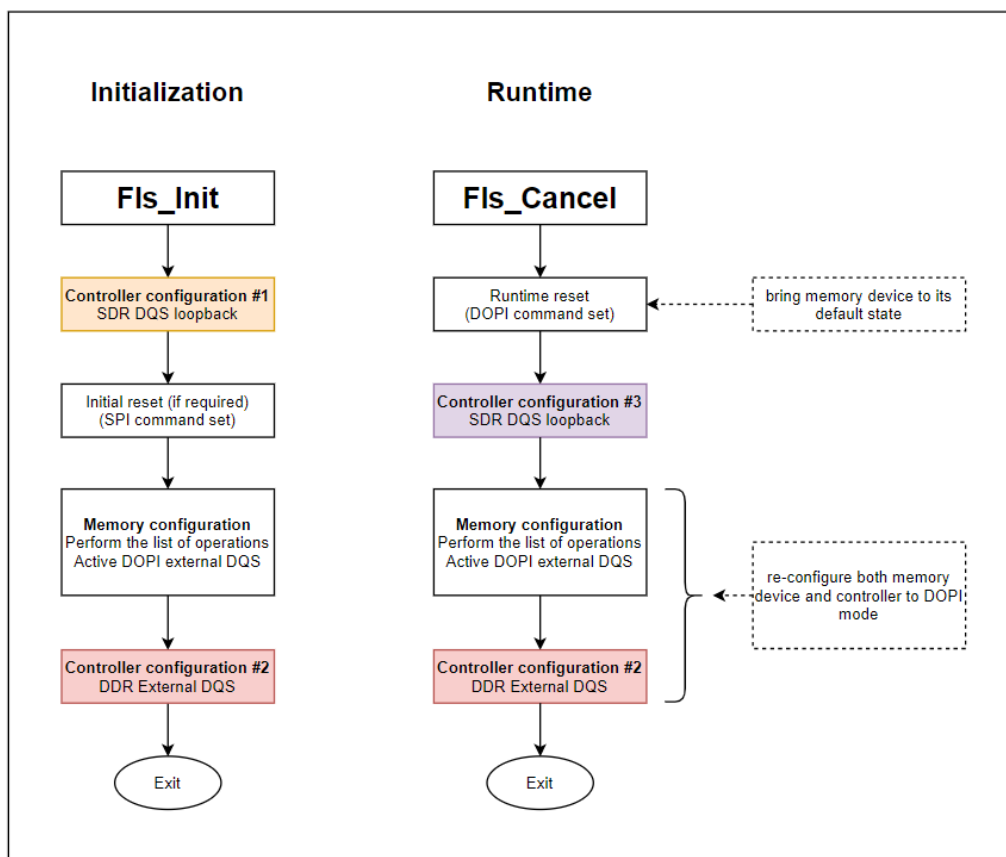
Name  initResetSettings

Reset LUT index  /Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0/ResetEnable_dopi 

Number of reset commands (1 -> 255)  2 

Configure controller on flash Init  /Fls/FlsConfigSet/FlsExternalDr/ControllerCfg_0_SDR_DQSLoopBack 

Below is the complete code flow (initialization time and runtime) for both QuadSPI controller and memory device to work in Double data rate - Octal I/O mode (DOPI).



Read/Write from unaligned addresses

Due to the nature of DDR protocol, both the starting address must be even address and data byte number must be even. Fls driver supports a feature to allow users to read/write with odd addresses and odd data length in DOPI mode, simply by setting the memory alignment value to 2 in the **FlsMem** configuration:

FlsMem

Name  FlsMem_0

Fls External Flash

Flash Device Name  Macronix 

Flash memory alignment (1 -> 16)  2 

Enable Ahb Direct Reads  ☒  Use SFDP autoconfiguration  ☐ 

Flash memory device initial configuration  /Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0

QSPI controller instance  /Fls/Fls/FlsConfigSet/FlsExternalDr/FlsController_0

Connection type  QSPI_IP_SIDE_A1 

Note

- For write operation, driver will send extra data with FFh to overwrite the overlapping memory area
- Users need to disable the development error detection feature in order to bypass the flash page boundary alignment checks:

General  ControllerCfg MemCfg FlsController FlsMem FlsSector Published Information

Fls General

Name  FlsGeneral

Fls Blank Check Api  ☒  Fls Cancel Api  ☒ 

Fls Compare Api  ☒  Fls Development Error Detect  ☐ 

- Or configure the size of flash page boundary to 1 to meet that requirement

General ControllerCfg MemCfg FlsController FlsMem FlsSector Published Information							
FlsSector							
Index	Name	Fls Sector Index	Fls Physical Sector	Fls Number Of Sector	Fls Page Size	Fls Sector Size	
0	FlsSector_0	0	FLS_EXT_SECTOR	3	1	4096	
1	FlsSector_1	1	FLS_EXT_SECTOR	2	16	4096	
2	FlsSector_2	2	FLS_EXT_SECTOR	1	16	4096	
3	FlsSector_3	3	FLS_EXT_SECTOR	1	16	4096	
4	FlsSector_4	4	FLS_EXT_SECTOR	1	16	4096	

AHB read

Users can enable the option **AHBReadEnable** in **FlsMem** to use the AHB read feature, this allows applica-

Driver

tion can read directly through Flash memory devices address mapping (QuadSPI's AHB region):

Fls (Fls)

FlsMem

Name

FlsMem_0

Fls External Flash

Flash Device Name

Macronix

Flash memory alignment (1 -> 16)

1

Enable Ahb Direct Reads

☒

Flash memory device initial configuration

/Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0

QSPI controller instance

/Fls/Fls/FlsConfigSet/FlsExternalDr/FlsController_0

Connection type

QSPI_IP_SIDE_A1

Besides, users need to configure the AHB buffers (master IDs and sizes):

Fls (Fls)

ControllerCfg

Name

ControllerCfg_0

Fls controller

FlsAhbBuffer

FlsAhbBuffer

Index	Name	Fls Ahb Buffer Instance	Fls Ahb Buffer Master ID	Fls Ahb Buffer Size	Fls Ahb Buffer All Masters Enable
0	FlsAhbBuffer_0	AHB_BUFFER_0	0	256	☐
1	FlsAhbBuffer_1	AHB_BUFFER_1	1	256	☐
2	FlsAhbBuffer_2	AHB_BUFFER_2	2	256	☐
3	FlsAhbBuffer_3	AHB_BUFFER_3	3	256	☒

Note

- The driver will configure AHB transfer sizes to match the buffer sizes
- **Qspi_Ip_ControllerGetStatus** can be used to wait for AHB commands complete to avoid conflict with subsequent IP commands

The LUT sequence of IP command read will be used for AHB command read:

MemCfg

Name: MemCfg_0

FLS External Flash | initConfiguration | FLSLUT

Flash device size (0 -> 4294967295): 8388608

Flash device page size (0 -> 4294967295): 256

Read LUT index: /Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0/Read

Write LUT index: /Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0/Write

FLSLUT

Name: Read

FLS LUT | FLSInstructionOperandPair

Index	Name	FLS Instruction Operand Pair Index	FLS LUT Instruction	FLS LUT Pad	FLS LUT Operand
0	FLSInstructionOperandPair_0	0	QSPI_IP_LUT_INSTR_CMD	QSPI_IP_LUT_PADS_1	0x3
1	FLSInstructionOperandPair_1	1	QSPI_IP_LUT_INSTR_ADDR	QSPI_IP_LUT_PADS_1	0x18
2	FLSInstructionOperandPair_2	2	QSPI_IP_LUT_INSTR_READ	QSPI_IP_LUT_PADS_1	0x10

Performance enhanced mode

The QuadSPI driver supports Continuous Read mode (0-X-X mode - no command for read instructions) which is implemented in some serial Flash memories:

MemCfg

Name: MemCfg_0

FLS External Flash | initConfiguration | FLSLUT

Flash device size (0 -> 4294967295): 8388608

Flash device page size (0 -> 4294967295): 256

Read LUT index: /Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0/FastRead

Write LUT index: /Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0/Write

0xx Read LUT index: /Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0/FastRead0xx

0xx Read LUT index - AHB version: /Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0/FastRead0xxAHB

Driver

There are two types of command, one for IP and one for AHB operations. Below is the example:

Name: FastRead					
FlsInstructionOperandPair					
Index	Name	Fls Instruction Operand Pair Index	Fls LUT Instruction	Fls LUT Pad	Fls LUT Operand
0	FlsInstructionOperandPair_0	0	QSPI_IP_LUT_INSTR_CMD	QSPI_IP_LUT_PADS_1	0xeb
1	FlsInstructionOperandPair_1	1	QSPI_IP_LUT_INSTR_ADDR	QSPI_IP_LUT_PADS_4	0x18
2	FlsInstructionOperandPair_2	2	QSPI_IP_LUT_INSTR_MODE	QSPI_IP_LUT_PADS_4	0xff
3	FlsInstructionOperandPair_3	3	QSPI_IP_LUT_INSTR_DUMMY	QSPI_IP_LUT_PADS_4	0x4
4	FlsInstructionOperandPair_4	4	QSPI_IP_LUT_INSTR_READ	QSPI_IP_LUT_PADS_4	0x10

Enhance indicator = 0xFF: Exit 0-X-X

Name: FastRead0xx					
FlsInstructionOperandPair					
Index	Name	Fls Instruction Operand Pair Index	Fls LUT Instruction	Fls LUT Pad	Fls LUT Operand
0	FlsInstructionOperandPair_0	0	QSPI_IP_LUT_INSTR_CMD	QSPI_IP_LUT_PADS_1	0xeb
1	FlsInstructionOperandPair_1	1	QSPI_IP_LUT_INSTR_ADDR	QSPI_IP_LUT_PADS_4	0x18
2	FlsInstructionOperandPair_2	2	QSPI_IP_LUT_INSTR_MODE	QSPI_IP_LUT_PADS_4	0xa5
3	FlsInstructionOperandPair_3	3	QSPI_IP_LUT_INSTR_DUMMY	QSPI_IP_LUT_PADS_4	0x4
4	FlsInstructionOperandPair_4	4	QSPI_IP_LUT_INSTR_READ	QSPI_IP_LUT_PADS_4	0x10
5	FlsInstructionOperandPair_5	5	QSPI_IP_LUT_INSTR_JMP_ON...	QSPI_IP_LUT_PADS_4	0x1

Enhance indicator = 0xA5: Enter 0-X-X

Jump to instruction 1 (ADDR)

Name: FastRead0xxAHB					
FlsInstructionOperandPair					
Index	Name	Fls Instruction Operand Pair Index	Fls LUT Instruction	Fls LUT Pad	Fls LUT Operand
0	FlsInstructionOperandPair_0	0	QSPI_IP_LUT_INSTR_ADDR	QSPI_IP_LUT_PADS_4	0x18
1	FlsInstructionOperandPair_1	1	QSPI_IP_LUT_INSTR_MODE	QSPI_IP_LUT_PADS_4	0xa5
2	FlsInstructionOperandPair_2	2	QSPI_IP_LUT_INSTR_DUMMY	QSPI_IP_LUT_PADS_4	0x4
3	FlsInstructionOperandPair_3	3	QSPI_IP_LUT_INSTR_READ	QSPI_IP_LUT_PADS_4	0x10

No read instruction

Enhance indicator = 0xA5: Enter 0-X-X

How they work:

1. (Optional) Call the **Qspi_Ip_AhbReadEnable** to enable AHB operation
2. Call the **Qspi_Ip_Enter0XX** to switch to 0-X-X read command sets, driver will perform a dummy read to activate 0-X-X mode
3. Call the **Qspi_Ip_Read** to read data from flash memory without the send of the instruction code
4. (Optional) Access the QuadSPI's AHB region to read data directly, **Qspi_Ip_ControllerGetStatus** can be used to wait for AHB commands complete to avoid conflict with subsequent IP commands
5. Call the **Qspi_Ip_Exit0XX** to disable 0-X-X mode and switch back to normal read command sets

Note

Qspi_Ip_ClearIpSeqPointer() and **Qspi_Ip_ClearAHBSeqPointer()** can be useful for devices which support burst modes for enhancing performance

SFDP feature

Fls driver contains the SFDP software, user can enable this option to attempt auto-configuration using the information read from the SFDP table

- SFDP (Serial Flash Discoverable Parameters) is a JEDEC standard - JESD216D

Fls (Fls) ✕

FlsMem

Name FlsMem_0

Fls External Flash

Flash Device Name	Macronix
Flash memory alignment (1 -> 16)	1
Enable Ahb Direct Reads	☒ Use SFDP autoconfiguration ☒
Flash memory device initial configuration	/Fls/Fls/FlsConfigSet/FlsExternalDr/MemCfg_0
QSPI controller instance	/Fls/Fls/FlsConfigSet/FlsExternalDr/FlsController_0
Connection type	QSPI_IP_SIDE_A1

Note

- The SFDP software only works for flash devices which support the SFDP feature
- Only standard JEDEC tables are interrogated, all vendor-specific tables will be skipped
- SFDP compliant devices must support 50 MHz operation for the Read SFDP command (instruction 5Ah). Some devices may support a wider frequency range, but user should run at 50 MHz or less and get valid results.
- The SFDP software does not support to read the tables directly in Octal-DDR mode (8D-8D-8D). It tries to issue 8D-8D-8D reset commands to force device to enter SPI mode before reading the SFDP tables. Therefore, user must enable the DDR mode in the QuadSPI controller settings for DDR instructions when working with devices boot up in Octal-DDR mode, or with Hyperflash devices which support the legacy SPI mode. (the Column Address setting is not required)

3.2.3.3 Supported memories

The following external device memories were tested by the Fls driver:

#	Vendor	Part No	Tested on release	SFDP	Single/Quad/Octal	Densities (bit)	Voltage	DQS	Frequency Flash Specification		Frequency QuadSPI supported	
									SDR Freq	DDR Freq	SDR Freq	DDR Freq
3	Macronix	MX25L6433FM2R-08G	S32K1XX	Y	Quad	64 Mb	3V	N	133MHz	-	80MHz	-

3.3 Hardware Resources

3.3.1 External flash

The Quad Serial Peripheral Interface (QuadSPI) block acts as an interface to external serial flash device. It supports SDR and HyperRAM modes upto 4 and 8 bidirectional data lines respectively.

For external flash: Configure pins for QSPI pin mux, driver strength enable, pull up enabled.

PTD12	PCR_PTD12	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0001	PTD12	PTD	Port D I/O	I/O
		0000_0010	FTM2_CH2	FTM2	FTM Channel	I/O
		0000_0011	LPI2C1_HREQ	LPI2C1	LPI2C Host Request Input	I
		0000_0100	ETM_TRACE_D1	TRACE		O
		0000_0101	MII_RMII_TX_EN	ENET	ENET Transmit Enable	O
		0000_0110	LPUART2_RTS	LPUART2	Request To Send	O
		0000_0111	QSPI_A_IO2	QuadSPI	QuadSPI Serial data for serial flash device A (fast)	I/O
		0000_0000	DISABLED		Signal Path Disabled	-
		0000_0001	PTC3	PTC	Port C I/O	I/O
PTC3	PCR_PTC3	0000_0010	FTM0_CH3	FTM0	FTM Channel	I/O
		0000_0011	CAN0_TX	CAN0	CAN Tx Channel	O
		0000_0100	LPUART0_TX	LPUART0	Transmit	I/O
		0000_0101	MII_TX_ER	ENET		O
		0000_0110	QSPI_A_CS	QuadSPI	QuadSPI Chip select for serial flash device A	O
		0000_0111	QSPI_B_IO3	QuadSPI	QuadSPI Serial data for serial flash / RAM device B	I/O
		-	ADC0_SE11	ADC0	ADC Single Ended Input	I
		-	CMP0_IN4	CMP0	Comparator Input Signal	I
		0000_0000	DISABLED		Signal Path Disabled	-
		0000_0001	PTD11	PTD	Port D I/O	I/O
PTD11	PCR_PTD11	0000_0010	FTM2_CH1	FTM2	FTM Channel	I/O
		0000_0011	FTM2_QD_PHA	FTM2	FTM quadrature Decode PhaseA	I
		0000_0100	ETM_TRACE_D2	TRACE		O
		0000_0101	MII_RMII_TX_CLK	ENET	ENET Transmit Clock	I/O
		0000_0110	LPUART2_CTS	LPUART2	Clear To Send (bar)	I
		0000_0111	QSPI_A_IO0	QuadSPI	QuadSPI Serial data for serial flash device A (fast)	I/O
		0000_0000	DISABLED		Signal Path Disabled	-
		0000_0001	PTC2	PTC	Port C I/O	I/O
		0000_0010	FTM0_CH2	FTM0	FTM Channel	I/O
		0000_0011	CAN0_RX	CAN0	CAN Rx channel	I
PTC2	PCR_PTC2	0000_0100	LPUART0_RX	LPUART0	Receive	I
		0000_0101	MII_RMII_TXD[0]	ENET	ENET Transmit Data	O
		0000_0110	ETM_TRACE_CLKOUT	TRACE		O
		0000_0111	QSPI_A_IO3	QuadSPI	QuadSPI Serial data for serial flash device A (fast)	I/O
		-	CMP0_IN6	CMP0	Comparator Input Signal	I
		0000_0000	DISABLED		Signal Path Disabled	-
		0000_0001	PTD7	PTD	Port D I/O	I/O
		0000_0010	LPUART2_TX	LPUART2	Transmit	I/O
		0000_0100	FTM2_FLT3	FTM2	FTM Fault Input	I
		0000_0101	MII_RMII_TXD[1]	ENET	ENET Transmit Data	O
PTD7	PCR_PTD7	0000_0110	ETM_TRACE_D0	TRACE		O
		0000_0111	QSPI_A_IO1	QuadSPI	QuadSPI Serial data for serial flash device A (fast)	I/O
		-	CMP0_IN6	CMP0	Comparator Input Signal	I
		0000_0000	DISABLED		Signal Path Disabled	-
		0000_0001	PTD10	PTD	Port D I/O	I/O
		0000_0010	FTM2_CH0	FTM2	FTM Channel	I/O
		0000_0011	FTM2_QD_PHB	FTM2	FTM quadrature Decode PhaseB	I
		0000_0100	ETM_TRACE_D3	TRACE		O
		0000_0101	MII_RX_CLK	ENET	ENET MII Receive Clock	I
		0000_0110	CLKOUT	SYSTEM	External Clock Output	O
PTD10	PCR_PTD10	0000_0111	QSPI_A_SCK	QuadSPI	QuadSPI Serial Clock for serial flash device A (fast)	I/O

Note:

- S32K148 (Except S32K148_lqfp100 has none) has one instance of QuadSPI. Other products in the S32K1xx series do not have QuadSPI.
- The following are not supported:
 - AHB Write
 - Data learning feature
 - Breakpoint and Watchpoint memory regions
 - QuadSPI in 100-pin LQFP

3.3.2 S32K1_S32M24X Flash Banks/Arrays, Sectors details

- The sizes of flash memory types on the chip are:

Chips	Program Flash	Data Flash (FlexNVM)
S32K116	128 KB (sector size 2k)	32 KB (sector size 2k)
S32K118	256 KB (sector size 2k)	32 KB (sector size 2k)
S32K142	256 KB (sector size 2k)	64 KB (sector size 2k)
S32K142W	256 KB (sector size 4k)	64 KB (sector size 2k)
S32K144	512 KB (sector size 4k)	64 KB (sector size 2k)
S32K144W	512 KB (sector size 4k)	64 KB (sector size 2k)
S32K146	1 MB (sector size 4k)	64 KB (sector size 2k)

Chips	Program Flash	Data Flash (FlexNVM)
S32K148	1.5 MB (sector size 4k)	512 KB (sector size 4k)
S32M241	128 KB (sector size 2k)	64 KB (sector size 2k)
S32M242	256 KB (sector size 2k)	64 KB (sector size 2k)
S32M243	256 KB (sector size 2k)	64 KB (sector size 2k)
S32M244	512 KB (sector size 4k)	64 KB (sector size 2k)

- For S32K116: has 128 KBytes of code flash (program flash) and 32 KBytes of data flash (FlexNVM)
 - 128K P Flash (each sector is 2K so $128K/2K = 64$ sectors)
 - 32K D Flash (each sector is 2K so $32K/2K = 16$ sectors)
 - There are 2 blocks (read partitions):
 - * P Flash: Block 0 (128K)
 - * D Flash: Block 1 (32K)

Sector name	Sector Size (KB)
FLS_DATA_ARRAY_0_BLOCK_1_S000	2
...	...
FLS_DATA_ARRAY_0_BLOCK_1_S015	2
FLS_CODE_ARRAY_0_BLOCK_0_S000	2
...	...
FLS_CODE_ARRAY_0_BLOCK_0_S063	2

- For S32K118: has 256 KBytes of code flash (program flash) and 32 KBytes of data flash (FlexNVM)
 - 256K P Flash (each sector is 2K so $256K/2K = 128$ sectors)
 - 32K D Flash (each sector is 2K so $32K/2K = 16$ sectors)
 - There are 2 blocks (read partitions):
 - * P Flash: Block 0 (256K)
 - * D Flash: Block 1 (32K)

Sector name	Sector Size (KB)
FLS_DATA_ARRAY_0_BLOCK_1_S000	2
...	...
FLS_DATA_ARRAY_0_BLOCK_1_S015	2
FLS_CODE_ARRAY_0_BLOCK_0_S000	2
...	...
FLS_CODE_ARRAY_0_BLOCK_0_S127	2

- For S32K142: has 256 KBytes of code flash (program flash) and 64 KBytes of data flash (FlexNVM)
 - 256K P Flash (each sector is 2K so $256K/2K = 128$ sectors)
 - 64K D Flash (each sector is 2K so $64K/2K = 32$ sectors)
 - There are 2 blocks (read partitions):
 - * P Flash: Block 0 (256K)

- * D Flash: Block 1 (64K)

Sector name	Sector Size (KB)
FLS_DATA_ARRAY_0_BLOCK_1_S000	2
...	...
FLS_DATA_ARRAY_0_BLOCK_1_S031	2
FLS_CODE_ARRAY_0_BLOCK_0_S000	2
...	...
FLS_CODE_ARRAY_0_BLOCK_0_S127	2

- For S32K142W: has 256 KBytes of code flash (program flash) and 64 KBytes of data flash (FlexNVM)
 - 256K P Flash (each sector is 4K so $256K/4K = 64$ sectors)
 - 64K D Flash (each sector is 2K so $64K/2K = 32$ sectors)
 - There are 2 blocks (read partitions):
 - * P Flash: Block 0 (256K)
 - * D Flash: Block 1 (64K)

Sector name	Sector Size (KB)
FLS_DATA_ARRAY_0_BLOCK_1_S000	2
...	...
FLS_DATA_ARRAY_0_BLOCK_1_S031	2
FLS_CODE_ARRAY_0_BLOCK_0_S000	4
...	...
FLS_CODE_ARRAY_0_BLOCK_0_S063	4

- For S32K144: has 512 KBytes of code flash (program flash) and 64 KBytes of data flash (FlexNVM)
 - 512K P Flash (each sector is 4K so $512/4K = 128$ sectors)
 - 64K D Flash (each sector is 2K so $64K/2K = 32$ sectors)
 - There are 2 blocks (read partitions):
 - * P Flash: Block 0 (512K)
 - * D Flash: Block 1 (64K)

Sector name	Sector Size (KB)
FLS_DATA_ARRAY_0_BLOCK_1_S000	2
...	...
FLS_DATA_ARRAY_0_BLOCK_1_S031	2
FLS_CODE_ARRAY_0_BLOCK_0_S000	4
...	...
FLS_CODE_ARRAY_0_BLOCK_0_S127	4

- For S32K144W: has 512 KBytes of code flash (program flash) and 64 KBytes of data flash (FlexNVM)
 - 512K P Flash (each sector is 4K so $512/4K = 128$ sectors)
 - 64K D Flash (each sector is 2K so $64K/2K = 32$ sectors)
 - There are 2 blocks (read partitions):

- * P Flash: Block 0 (512K)
- * D Flash: Block 1 (64K)

Sector name	Sector Size (KB)
FLS_DATA_ARRAY_0_BLOCK_1_S000	2
...	...
FLS_DATA_ARRAY_0_BLOCK_1_S031	2
FLS_CODE_ARRAY_0_BLOCK_0_S000	4
...	...
FLS_CODE_ARRAY_0_BLOCK_0_S127	4

- For S32K146: has 1 MBytes of code flash (program flash) and 64 KBytes of data flash (FlexNVM)
 - 1M P Flash (each sector is 4K so $1\text{M}/4\text{K} = 256$ sectors)
 - 64K D Flash (each sector is 2K so $64\text{K}/2\text{K} = 128$ sectors)
 - There are 3 blocks (read partitions):
 - * P Flash: Block 0 (512K), Block 1 (512K)
 - * D Flash: Block 2 (64K)

Sector name	Sector Size (KB)
FLS_DATA_ARRAY_0_BLOCK_2_S000	2
...	...
FLS_DATA_ARRAY_0_BLOCK_2_S031	2
FLS_CODE_ARRAY_0_BLOCK_0_S000	4
...	...
FLS_CODE_ARRAY_0_BLOCK_1_S255	4

- For S32K148: has 1.5 MBytes of code flash (program flash) and 512 KBytes of data flash (FlexNVM)
 - 1.5M P Flash (each sector is 4K so $1.5\text{M}/4\text{K} = 384$ sectors)
 - 512K D Flash (each sector is 4K so $512\text{K}/4\text{K} = 128$ sectors)
 - There are 4 blocks (read partitions):
 - * P Flash: Block 0 (512K), Block 1 (512K), Block 2 (512K)
 - * D Flash: Block 3 (512K)

Sector name	Sector Size (KB)
FLS_DATA_ARRAY_0_BLOCK_3_S000	4
...	...
FLS_DATA_ARRAY_0_BLOCK_3_S127	4
FLS_CODE_ARRAY_0_BLOCK_0_S000	4
...	...
FLS_CODE_ARRAY_0_BLOCK_2_S383	4

- For S32M241: has 128 KBytes of code flash (program flash) and 64 KBytes of data flash (FlexNVM)
 - 128K P Flash (each sector is 2K so $128\text{K}/2\text{K} = 64$ sectors)
 - 64K D Flash (each sector is 2K so $64\text{K}/2\text{K} = 32$ sectors)

- There are 2 blocks (read partitions):
 - * P Flash: Block 0 (128K)
 - * D Flash: Block 1 (64K)

Sector name	Sector Size (KB)
FLS_DATA_ARRAY_0_BLOCK_1_S000	2
...	...
FLS_DATA_ARRAY_0_BLOCK_1_S031	2
FLS_CODE_ARRAY_0_BLOCK_0_S000	2
...	...
FLS_CODE_ARRAY_0_BLOCK_0_S063	2

- For S32M242: has 256 KBytes of code flash (program flash) and 64 KBytes of data flash (FlexNVM)
 - 256K P Flash (each sector is 2K so $256K/2K = 128$ sectors)
 - 64K D Flash (each sector is 2K so $64K/2K = 32$ sectors)
 - There are 2 blocks (read partitions):
 - * P Flash: Block 0 (256K)
 - * D Flash: Block 1 (64K)

Sector name	Sector Size (KB)
FLS_DATA_ARRAY_0_BLOCK_1_S000	2
...	...
FLS_DATA_ARRAY_0_BLOCK_1_S031	2
FLS_CODE_ARRAY_0_BLOCK_0_S000	2
...	...
FLS_CODE_ARRAY_0_BLOCK_0_S127	2

- For S32M243: has 256 KBytes of code flash (program flash) and 64 KBytes of data flash (FlexNVM)
 - 256K P Flash (each sector is 4K so $256K/4K = 64$ sectors)
 - 64K D Flash (each sector is 2K so $64K/2K = 32$ sectors)
 - There are 2 blocks (read partitions):
 - * P Flash: Block 0 (256K)
 - * D Flash: Block 1 (64K)

Sector name	Sector Size (KB)
FLS_DATA_ARRAY_0_BLOCK_1_S000	2
...	...
FLS_DATA_ARRAY_0_BLOCK_1_S031	2
FLS_CODE_ARRAY_0_BLOCK_0_S000	2
...	...
FLS_CODE_ARRAY_0_BLOCK_0_S063	2

- For S32M244: has 512 KBytes of code flash (program flash) and 64 KBytes of data flash (FlexNVM)
 - 512K P Flash (each sector is 4K so $512K/4K = 128$ sectors)

- 64K D Flash (each sector is 2K so $64K/2K = 32$ sectors)
- There are 2 blocks (read partitions):
 - * P Flash: Block 0 (512K)
 - * D Flash: Block 1 (64K)

Sector name	Sector Size (KB)
FLS_DATA_ARRAY_0_BLOCK_1_S000	2
...	...
FLS_DATA_ARRAY_0_BLOCK_1_S031	2
FLS_CODE_ARRAY_0_BLOCK_0_S000	4
...	...
FLS_CODE_ARRAY_0_BLOCK_0_S127	4

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR FLS Driver software specification in some places. There are also some additional requirements (on top of requirements detailed in AUTOSAR FLS Driver software specification) which need to be satisfied for correct operation.

- Deviations Status Column Description

Term	Definition
N/S	Not In Scope
N/F	Not Fully Implemented
N/I	Not Implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope for the driver.

- Driver Deviations Table

Requirement	Status	Description	Notes
SWS_Fls_00145	N/S	If possible, e.g. with interrupt controlled implementations, the FLS module shall start the first round of the erase job directly within the function Fls_↔Erase to reduce overall runtime.	Fls driver does not support interrupt. Will be removed after ticket: AAI-902 is implemented.
SWS_Fls_00146	N/S	If possible, e.g. with interrupt controlled implementations, the FLS module shall start the first round of the write job directly within the function Fls_↔Write to reduce overall runtime.	Fls driver does not support interrupt. Will be removed after ticket: AAI-902 is implemented.
SWS_Fls_00232	N/S	The configuration parameter FlsUse↔Interrupts shall switch between interrupt and polling controlled job processing if this is supported by the flash memory hardware.	Fls driver does not support interrupt. Will be removed after ticket: AAI-902 is implemented.

Requirement	Status	Description	Notes
SWS_Fls_00233	N/S	The FLS module's implementer shall locate the interrupt service routine in Fls_Irq.c.	Fls driver does not support interrupt. Will be removed after ticket: AAI-902 is implemented.
SWS_Fls_00234	N/S	If interrupt controlled job processing is supported and enabled with the configuration parameter FlsUseInterrupts, the interrupt service routine shall <continue>	Fls driver does not support interrupt. Will be removed after ticket: AAI-902 is implemented.

- As a deviation from standard: Fls_[VariantName]_PBcfg.c files will contain the definition for all parameters that are variant aware, independent of the configuration class that will be selected (PC, LT, PB) Fls_Cfg.c, Fls_Cfg.h file will contain the definition for all parameters that are not variant aware

3.5 Driver Limitations

3.5.1 For internal flash.

- With Fls_Write, both u32TargetAddress and u32Length must be double words aligned. If not, a STATUS_FTFCS_IP_ERROR_INPUT_PARAM error code will be thrown at IP layer leading to FLS_ERASE_WRITE_FAILED at HLD.

3.5.2 For external flash

- The Flash initialization for external flash (QSPI) does not enable QSPI clock gate, users must configure the MISCTRL0 register to enable Qspi clock gate before Flash initialization or QSPI Ip initialization (see MISCTRL0 register, SIM chapter, Reference Manual)

– Example: `IP_SIM->MISCTRL0 |= SIM_MISCTRL0_QSPI_CLK_SEL_MASK;`

Note:

- On S32K148 derivative, PCCRMR[R2] should be programmed as 00b (NonCacheable) as S32K148 FlexNVMS region is not cacheable
- For more information, please refer to the LMEM chapter in the Reference manual and "Data Cache Restrictions" chapter in the Integration Manual
- The SFDP feature only supports standard JEDEC tables, all vendor-specific tables will be skipped

3.6 Driver usage and configuration tips

3.6.1 Introduction

For internal flash sectors, it's possible to modify the behavior of sector erase / page write using two configuration parameters (FlsSectorEraseAsynch, FlsPageWriteAsynch) in FlsSector TAB.

FlsSector

Name  FlsSector_0

Fls Sector

Fls Sector Index	 0  
Fls Physical Sector	 FLS_EXT_SECTOR  
Fls Number Of Sector (1 -> 65535)*	 1 
Fls Page Size	 16  
Fls Sector Size	 4096  
Fls Sector Start Address	 0  
Fls Sector Erase Asynch	 ☒  Fls Page Write Asynch  ☒ 
Fls Hardware Channel	 FLS_CH_QSPI 
Fls Qspi Sector Channel	 FLS_CH_EXTERN_QSPI_0_A1 
 External flash device instance	 /Fls/FlsConfigSet/FlsExternalDr/FlsMem_0
Fls Sector Hardware Byte Address (0x0 -> 0xfffffffff)	 0x11000 

If FlsSectorEraseAsynch/FlsPageWriteAsynch are enabled sector erase / page write job in the Fls_MainFunction are executed asynchronously, it means that Fls_MainFunction will not wait (not blocking) for completion of high voltage operation.

If FlsSectorEraseAsynch/FlsPageWriteAsynch are disabled sector erase / page write job are executed synchronously, which means sector erase / page write job are blocking and any high voltage operation will be completed during one Fls_Mainfunction.

3.6.2 Fls Enable Check Configuration CRC

The CRC configuration check that takes an amount of time, which may not be necessary if the image is already authenticated. So the driver will support the CRC check optional for the FLS Configuration. With the default to OFF (disabled). User can enable this function on configuration interface

Fls General

Name*  FlsGeneral

☐ FlsEcucPartitionRef

Enable development error check at IP level  ☐ 

Fls Enable Check Configuration CRC*  ☒ 

3.6.3 Avoiding RWW problem

To avoid RWW (Read While Write) problems on the internal flash, the FLS driver provides the FlsAcLoad←

OnJobStart configuration parameter. If it is set to true the Fls driver will load the flash access code routine to RAM whenever an erase or write job is started and unload (overwrite) it after that job has been finished or cancelled.

FlsAcLoadOnJobStart functionality can be used only in case of Sync Mode, in which case the flash access code is loaded to RAM and therefore the flash driver shouldn't have RWW problems; if FlsAcLoadOnJobStart is set to false the sector erased / page written must belong to flash array / partition different from flash array / partition the application is executing from.

In case of Async operations it is only possible to erase / write to flash array different from flash array the application is executing from. This mode is usable only if the platform supports different Read While Write partitions or if the entire code is executed from RAM, during the flash modify operation.

Note:

1. For internal flash, the flash driver use the sector erase / page write access code to clear the FSTAT:CCIF bit and wait for completion of high voltage operation (and therefore incompatible with Async operation).
2. The flash module is further divided into partitions/blocks that determine locations for valid read-while-write (RWW) operations(Ex: Program flash block 0 and Data flash/FlexNvm). While the embedded flash memory is performing a 'write' (program or erase) to a given partition, it can simultaneously perform a read from any other partition.
3. FlsAcCallback should be in located in RAM if FlsACLoadOnJob is true to avoid RWW problem.

3.6.4 Flash memory physical sectors unlock example

For unprotecting internal flash sectors, the flash field configuration locations corresponding to sector protection have to be erased (Addresses 0x0_0408 - 0x0_040B and 0x0_040F), reprogrammed if needed and the chip reset.

Care has to be taken when programming the flash configuration field, so that FSEC location (0x0_040C) is reprogrammed to value 0xFE after erase, and all configuration locations are erased or programmed as needed.

Code example for resetting configuration field using direct register access.

```

/* Erase flash sector containing configuration field */
FTFC->FCCOB0 = 0x09;    /* Erase sector */
FTFC->FCCOB1 = 0x00;    /* Address 0x0_0000*/
FTFC->FCCOB2 = 0x00;    /* Address */
FTFC->FCCOB3 = 0x00;    /* Address */

/* Program flash configuration field */
FTFC->FCCOB0 = 0x07;    /* Program phrase */
FTFC->FCCOB1 = 0x08;    /* Address 0x0_0408*/
FTFC->FCCOB2 = 0x04;    /* Address */
FTFC->FCCOB3 = 0x00;    /* Address */
FTFC->FCCOB4 = 0xFF;    /* Data for flash location 0x0_040B, FPROT3 */
FTFC->FCCOB5 = 0xFF;    /* Data for flash location 0x0_040A, FPROT2 */
FTFC->FCCOB6 = 0xFF;    /* Data for flash location 0x0_0409, FPROT1 */
FTFC->FCCOB7 = 0xFF;    /* Data for flash location 0x0_0408, FPROT0 */
FTFC->FCCOB8 = 0xFF;    /* Data for flash location 0x0_040F, FDPROT */
FTFC->FCCOB9 = 0xFF;    /* Data for flash location 0x0_040E, FEPROT */
FTFC->FCCOBA = 0xFF;    /* Data for flash location 0x0_040D, FOPT */
FTFC->FCCOBB = 0xFE;    /* Data for flash location 0x0_040C, FSEC */

/* Reset */

```

3.6.5 Development Error Description

Error Code	Value	Condition triggering the error
FLS_E_PARAM_CONFIG	1	API service called with wrong parameter
FLS_E_PARAM_ADDRESS	2	u32TargetAddress is not in range and aligned to first byte of flash sector
FLS_E_PARAM_LENGTH	3	u32TargetAddress is not in range and aligned to last byte of flash sector
FLS_E_PARAM_DATA	4	API service called with wrong parameter
FLS_E_UNINIT	5	API service called without module initialization
FLS_E_BUSY	6	API service called while driver still busy
FLS_E_PARAM_POINTER	10	API service called with NULL_PTR passed

3.7 Runtime errors

- The driver supports runtime generation of the errors listed in the table:

Error code	Function	Condition triggering the error
FLS_E_VERIFY_ERASE_FAILED	Fls_MainFunction()	Verify erasing operation failed before writing a flash block
		Verify erasing operation failed after erasing a flash block
FLS_E_VERIFY_WRITE_FAILED	Fls_MainFunction()	Verify writing operation failed after writing a flash block

Error code	Function	Condition triggering the error
FLS_E_TIMEOUT	Fls_Init()	Access timeout value to the flash controller has been exceeded
	Fls_MainFunction()	Maximum read / write / compare / erase time has been exceeded

- The driver supports Transient faults generation of the errors listed in the table:

Error Code	Function	Condition triggering the error
FLS_E_ERASE_FAILED	Fls_MainFunction()	A flash erase job fails due to a hardware error
FLS_E_WRITE_FAILED		A flash write job fails due to a hardware error
FLS_E_READ_FAILED		A flash read job fails due to a hardware error
FLS_E_COMPARE_FAILED		A flash compare job fails due to a hardware error
FLS_E_UNEXPECTED_FLASH_ID	Fls_Init()	The hardware ID of the external flash device mismatched the corresponding configuration parameter

- Development Error Description

Error Code	Value	Condition triggering the error
FLS_E_PARAM_CONFIG	1	API service called with wrong parameter
FLS_E_PARAM_ADDRESS	2	u32TargetAddress is not in range and aligned to first byte of flash sector
FLS_E_PARAM_LENGTH	3	u32TargetAddress is not in range and aligned to last byte of flash sector
FLS_E_PARAM_DATA	4	NULL_PTR == SourceAddressPtr
FLS_E_UNINIT	5	API service called without module initialization
FLS_E_BUSY	6	PI service called while driver still busy
FLS_E_PARAM_POINTER	10	NULL_PTR passed

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

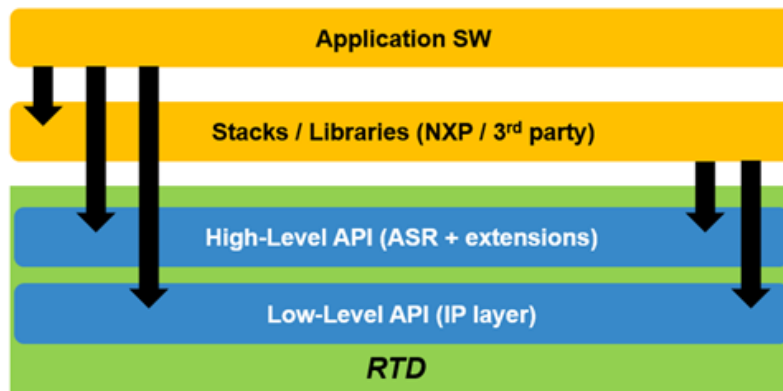


Figure 3.1 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized
 2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
 3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
 4. Initialization of specific drivers
 5. Runtime functionality
 6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

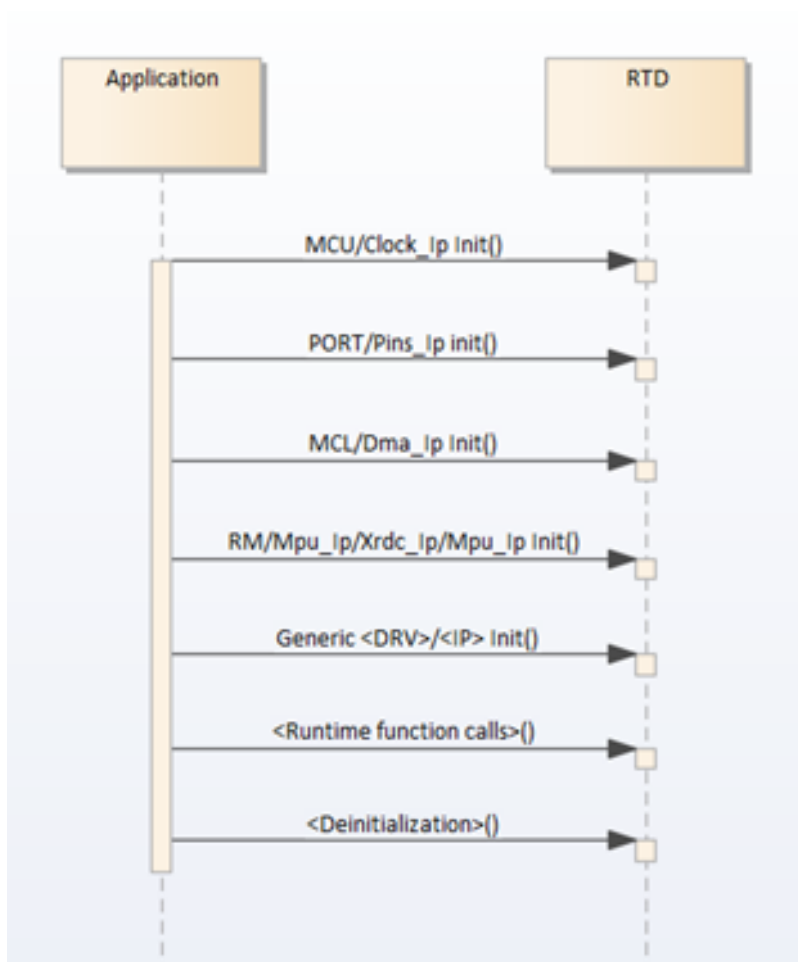


Figure 3.2 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

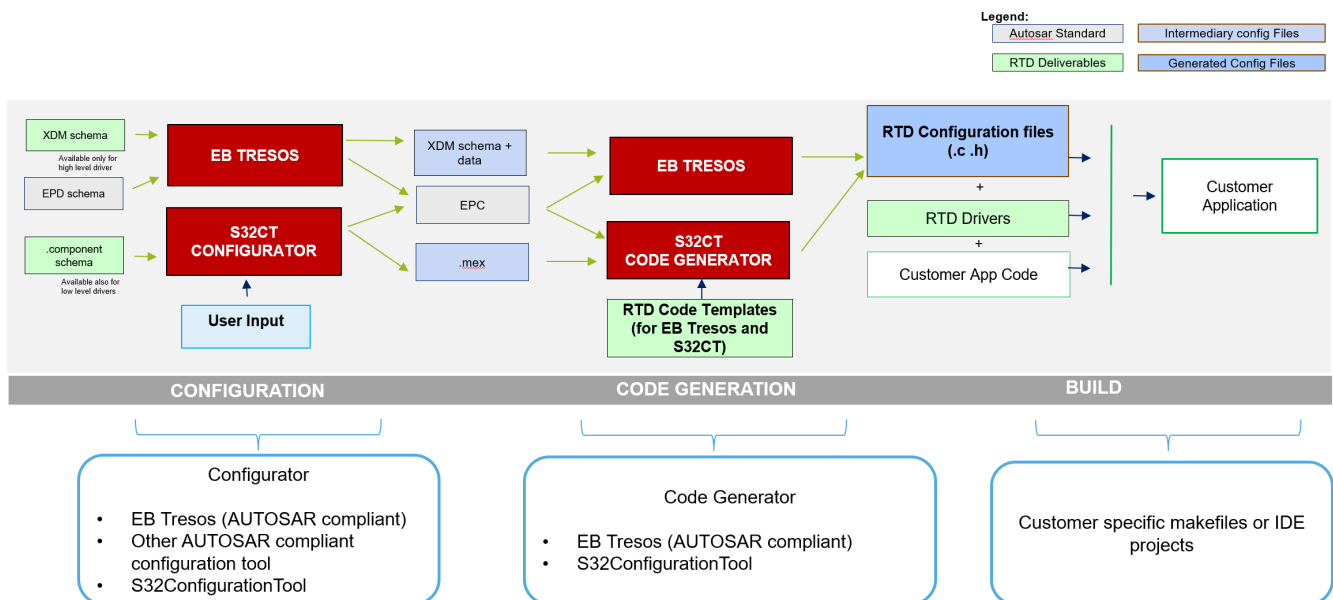


Figure 4.1 Configuration generation flow

- [Fls](#)
- [FTFC_IP](#)
- [QSPI_IP](#)

4.1 Fls

- Module [Fls](#)
 - Container [FlsConfigSet](#)
 - * Parameter [FlsAcErase](#)
 - * Parameter [FlsAcWrite](#)
 - * Parameter [FlsAcErasePointer](#)
 - * Parameter [FlsAcWritePointer](#)
 - * Parameter [FlsCallCycle](#)
 - * Parameter [FlsDefaultMode](#)
 - * Parameter [FlsACCallback](#)
 - * Parameter [FlsJobEndNotification](#)
 - * Parameter [FlsJobErrorNotification](#)
 - * Parameter [FlsStartFlashAccessNotif](#)
 - * Parameter [FlsFinishedFlashAccessNotif](#)
 - * Parameter [FlsReadFunctionCallout](#)
 - * Parameter [FlsQspiInitCallout](#)
 - * Parameter [FlsQspiResetCallout](#)
 - * Parameter [FlsQspiErrorCheckCallout](#)
 - * Parameter [FlsQspiEccCheckCallout](#)
 - * Parameter [FlsMaxReadFastMode](#)
 - * Parameter [FlsMaxReadNormalMode](#)
 - * Parameter [FlsMaxWriteFastMode](#)
 - * Parameter [FlsMaxWriteNormalMode](#)
 - * Parameter [FlsProtection](#)
 - * Container [FlsExternalDriver](#)
 - Reference [FlsSpiReference](#)
 - Container [ControllerCfg](#)
 - Parameter [FlsHwUnitReadMode](#)
 - Parameter [FlsSerialFlashA1Size](#)
 - Parameter [FlsSerialFlashA2Size](#)
 - Parameter [FlsSerialFlashB1Size](#)
 - Parameter [FlsSerialFlashB2Size](#)
 - Parameter [FlsHwUnitSamplingModeA](#)
 - Parameter [FlsHwUnitSamplingModeB](#)
 - Parameter [IdleSignalDriveIOFB3HighLvl](#)
 - Parameter [IdleSignalDriveIOFB2HighLvl](#)

- Parameter [IdleSignalDriveIOFA3HighLvl](#)
- Parameter [IdleSignalDriveIOFA2HighLvl](#)
- Parameter [FlsHwUnitSamplingEdge](#)
- Parameter [FlsHwUnitSamplingDly](#)
- Parameter [FlsHwUnitDqsLatencyEnable](#)
- Parameter [FlsHwUnitTdh](#)
- Parameter [FlsHwUnitTcsh](#)
- Parameter [FlsHwUnitTcss](#)
- Parameter [FlsHwUnitColumnAddressWidth](#)
- Parameter [FlsHwUnitWordAddressable](#)
- Container [FlsAhbBuffer](#)
 - Parameter [FlsAhbBufferInstance](#)
 - Parameter [FlsAhbBufferMasterId](#)
 - Parameter [FlsAhbBufferSize](#)
 - Parameter [FlsAhbBufferAllMasters](#)
- Container [FlsHwUnitSpecificSettings](#)
 - Parameter [FlsHwUnitInputClockSelect](#)
 - Parameter [FlsHwUnitInternalRefClockSelect](#)
 - Parameter [FlsHwUnitProgrammableDivider](#)
 - Parameter [FlsHwUnitInvertClkDqsA](#)
 - Parameter [FlsHwUnitInvertClkDqsB](#)
 - Parameter [FlsHwUnitFineDelayA](#)
 - Parameter [FlsHwUnitFineDelayB](#)
- Container [MemCfg](#)
 - Parameter [MemCfgSize](#)
 - Parameter [MemCfgPageSize](#)
 - Reference [MemCfgReadLUT](#)
 - Reference [MemCfgWriteLUT](#)
 - Reference [MemCfgRead0xxLUT](#)
 - Reference [MemCfgRead0xxLUTAHB](#)
 - Reference [ctrlAutoCfgPtr](#)
 - Container [MemCfgReadIdSettings](#)
 - Parameter [MemCfgReadIdSize](#)
 - Parameter [FlsQspiDeviceId](#)
 - Reference [MemCfgReadIdLUT](#)
 - Container [MemCfgEraseSettings](#)

- Parameter [MemCfgErase1Size](#)
- Parameter [MemCfgErase2Size](#)
- Parameter [MemCfgErase3Size](#)
- Parameter [MemCfgErase4Size](#)
- Reference [MemCfgErase1LUT](#)
- Reference [MemCfgErase2LUT](#)
- Reference [MemCfgErase3LUT](#)
- Reference [MemCfgErase4LUT](#)
- Reference [ChipEraseLUT](#)
- Container [statusConfig](#)
 - Parameter [regSize](#)
 - Parameter [busyOffset](#)
 - Parameter [busyValue](#)
 - Parameter [writeEnableOffset](#)
 - Parameter [blockProtectionOffset](#)
 - Parameter [blockProtectionWidth](#)
 - Parameter [blockProtectionValue](#)
 - Reference [statusRegInitReadLut](#)
 - Reference [statusRegReadLut](#)
 - Reference [statusRegWriteLut](#)
 - Reference [writeEnableSRLut](#)
 - Reference [writeEnableLut](#)
- Container [suspendSettings](#)
 - Reference [eraseSuspendLut](#)
 - Reference [eraseResumeLut](#)
 - Reference [programSuspendLut](#)
 - Reference [programResumeLut](#)
- Container [resetSettings](#)
 - Parameter [resetCmdCount](#)
 - Reference [resetCmdLut](#)
- Container [initResetSettings](#)
 - Parameter [resetCmdCount](#)
 - Reference [resetCmdLut](#)
- Container [initConfiguration](#)
 - Parameter [opType](#)
 - Parameter [addr](#)

- Parameter [size](#)
- Parameter [shift](#)
- Parameter [width](#)
- Parameter [value](#)
- Reference [command1Lut](#)
- Reference [command2Lut](#)
- Reference [weLut](#)
- Reference [ctrlCfgPtr](#)
- Container [FlsLUT](#)
 - Parameter [FlsLUTIndex](#)
- Container [FlsInstructionOperandPair](#)
 - Parameter [FlsInstrOperPairIndex](#)
 - Parameter [FlsLUTInstruction](#)
 - Parameter [FlsLUTPad](#)
 - Parameter [FlsLUTOperand](#)
- Container [FlsController](#)
 - Parameter [ControllerName](#)
 - Reference [FlsControllerCfgRef](#)
- Container [FlsMem](#)
 - Parameter [FlsMemName](#)
 - Parameter [MemAlignment](#)
 - Parameter [AHBReadEnable](#)
 - Parameter [FlsMemUseSfdp](#)
 - Parameter [connectionType](#)
 - Reference [FlsMemCfgRef](#)
 - Reference [qspiInstance](#)
- * Container [FlsSectorList](#)
 - Container [FlsSector](#)
 - Parameter [FlsSectorIndex](#)
 - Parameter [FlsPhysicalSector](#)
 - Parameter [FlsNumberOfSectors](#)
 - Parameter [FlsPageSize](#)
 - Parameter [FlsSectorSize](#)
 - Parameter [FlsSectorStartaddress](#)
 - Parameter [FlsSectorEraseAsynch](#)
 - Parameter [FlsPageWriteAsynch](#)

- Parameter [FlsHwCh](#)
- Parameter [FlsSectorHwAddress](#)
- Reference [flashInstance](#)
- Container [AutosarExt](#)
 - * Parameter [FlsEnableUserModeSupport](#)
 - * Parameter [FlsQspiLockLUT](#)
 - * Parameter [FlsQspiHangRecovery](#)
 - * Parameter [FlsSynchronizeCache](#)
 - * Parameter [FlsInvalidPrefetchBufFromRam](#)
 - * Parameter [FlsInternalSectorsConfigured](#)
 - * Parameter [FlsExternalSectorsConfigured](#)
- Container [FlsGeneral](#)
 - * Parameter [FlsEnableDevAssert](#)
 - * Parameter [FlsEnableCheckCfgCrc](#)
 - * Parameter [FlsAcLoadOnJobStart](#)
 - * Parameter [FlsCleanCacheAfterLoadAc](#)
 - * Parameter [FlsBaseAddress](#)
 - * Parameter [FlsBlankCheckApi](#)
 - * Parameter [FlsCancelApi](#)
 - * Parameter [FlsCompareApi](#)
 - * Parameter [FlsDevErrorDetect](#)
 - * Parameter [FlsDriverIndex](#)
 - * Parameter [FlsGetJobResultApi](#)
 - * Parameter [FlsGetStatusApi](#)
 - * Parameter [FlsSetModeApi](#)
 - * Parameter [FlsTotalSize](#)
 - * Parameter [FlsUseInterrupts](#)
 - * Parameter [FlsVersionInfoApi](#)
 - * Parameter [FlsECCCheck](#)
 - * Parameter [FlsECCHandlingProtectionHook](#)
 - * Parameter [FlsEraseVerificationEnabled](#)
 - * Parameter [FlsWriteVerificationEnabled](#)
 - * Parameter [FlsMaxEraseBlankCheck](#)
 - * Parameter [FlsTimeoutSupervisionEnabled](#)
 - * Parameter [FlsTimeoutMethod](#)
 - * Parameter [FlsAsyncWriteTimeout](#)

- * Parameter [FlsAsyncEraseTimeout](#)
- * Parameter [FlsSyncWriteTimeout](#)
- * Parameter [FlsSyncEraseTimeout](#)
- * Parameter [FlsAbortTimeout](#)
- * Parameter [FlsQspiIpTimeoutOsifCounterType](#)
- * Parameter [FlsQspiSyncReadTimeout](#)
- * Parameter [FlsQspiAsyncWriteTimeout](#)
- * Parameter [FlsQspiAsyncEraseTimeout](#)
- * Parameter [FlsQspiSyncWriteTimeout](#)
- * Parameter [FlsQspiSyncEraseTimeout](#)
- * Parameter [FlsQspiCommandCompleteTimeout](#)
- * Parameter [FlsQspiResetTimeout](#)
- * Parameter [FlsQspiFlashInitTimeout](#)
- * Parameter [FlsQspiSoftwareResetDelay](#)
- * Parameter [FlsQspiTxBufferResetDelay](#)
- * Parameter [FlsQspiWriteEnableRetries](#)
- * Reference [FlsEcucPartitionRef](#)
- Container [FlsPublishedInformation](#)
 - * Parameter [FlsAcLocationErase](#)
 - * Parameter [FlsAcLocationWrite](#)
 - * Parameter [FlsAcSizeErase](#)
 - * Parameter [FlsAcSizeWrite](#)
 - * Parameter [FlsEraseTime](#)
 - * Parameter [FlsErasedValue](#)
 - * Parameter [FlsECCValue](#)
 - * Parameter [FlsExpectedHwId](#)
 - * Parameter [FlsSpecifiedEraseCycles](#)
 - * Parameter [FlsWriteTime](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)

Configuration Parameters

- * Parameter [VendorApiInfix](#)
- * Parameter [VendorId](#)

4.1.1 Module Fls

Configuration of the Fls (internal or external flash driver) module.

Included containers:

- [FlsConfigSet](#)
- [AutosarExt](#)
- [FlsGeneral](#)
- [FlsPublishedInformation](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.1.2 Container FlsConfigSet

Container for runtime configuration parameters of the flash driver.

Implementation Type: Fls_ConfigType.

Included subcontainers:

- [FlsExternalDriver](#)
- [FlsSectorList](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter FlsAcErase

Address offset in RAM to which the erase flash access code shall be loaded.

Used as function pointer to access the erase flash access code.

Note: To use Fls Access Code Erase be sure Fls Access Code Erase Pointer is NULL or NULL_PTR.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	536740096
max	4294967295
min	0

4.1.4 Parameter FlsAcWrite

Address offset in RAM to which the write flash access code shall be loaded.

Used as function pointer to access the write flash access code.

Note: To use Fls Access Code Write be sure Fls Access Code Write Pointer is NULL or NULL_PTR.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	536740096
max	4294967295
min	0

4.1.5 Parameter FlsAcErasePointer

Vendor specific: Pointer in RAM to which the erase flash access code shall be loaded.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.1.6 Parameter FlsAcWritePointer

Vendor specific: Pointer in RAM to which the write flash access code shall be loaded.

Used as function pointer to access the write flash access code.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.1.7 Parameter FlsCallCycle

Cycle time of calls of the flash driver main function

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0.2
max	1.0
min	0.0

4.1.8 Parameter FlsDefaultMode

This parameter is the default FLS device mode after initialization.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	MEMIF_MODE_SLOW
literals	['MEMIF_MODE_FAST', 'MEMIF_MODE_SLOW']

4.1.9 Parameter FlsACCallback

Vendor specific: Mapped to the Access Code Callback provided by some upper layer module, typically the Wdg module.

Note: Disable the Access Code Callback to have it set as NULL_PTR.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	Fls_AC_Callback

4.1.10 Parameter FlsJobEndNotification

Mapped to the job end notification routine provided by some upper layer module, typically the Fee module.

Note: Disable the end notification to have it set as NULL_PTR

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Property	Value
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	Fee_JobEndNotification

4.1.11 Parameter FlsJobErrorNotification

Mapped to the job error notification routine provided by some upper layer module, typically the Fee module.

Note: Disable the error notification to have it set as NULL_PTR

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	Fee_JobErrorNotification

4.1.12 Parameter FlsStartFlashAccessNotif

Vendor specific: Start flash access. If configured, this notification will be called before any flash memory access.

It is called before flash memory read accesses(in read, compare, verify write, verify erase jobs) and before flash memory program operations(in write and erase jobs).

The purpose of this notification together with FlsFinishedFlashAccess, is to ensure that, if needed, no other executed code(other tasks, cores, masters) will access the affected flash area simultaneously with the access initiated by the driver. For more details, see Integration manual, chapter 5. Module requirements.

Note: Disable the error notification to have it set as NULL_PTR

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	Fls_StartFlashAccessNotif

4.1.13 Parameter FlsFinishedFlashAccessNotif

Vendor specific: Finished flash access. If configured, this notification will be called after any flash memory access.

It is called after flash memory read accesses(in read, compare, verify write, verify erase jobs).

The purpose of this notification together with FlsStartFlashAccess, is to ensure that, if needed, no other executed code(other tasks, cores, masters) will access the affected flash area simultaneously with the access initiated by the driver. For more details, see Integration manual, chapter 5. Module requirements.

Note: Disable the error notification to have it set as NULL_PTR

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	Fls_FinishedFlashAccessNotif

4.1.14 Parameter FlsReadFunctionCallout

Vendor specific: The callout for the user to check for ECC errors for Internal Flash memories.

In this callout, the user can schedule a task that reads from flash memory to a read source buffer and check/handle for an ECC exception.

If an exception occurs, a descriptor regarding the faulty line number that caused the ECC and the state of the task should be updated.

Note: Inside a task, the flow is not endangered in case of an ECC exception, as the task can be forcibly terminated in that case. (please see the chapter 'ECC Management on Flash' in IM for more information)

Configuration Parameters

- *Disable: Read and Compare functions will be handled by driver*
- *Enable: Read and Compare functions will be handled by users.*

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	FlsReadFunctionCallout

4.1.15 Parameter FlsQspiInitCallout

Vendor specific: Callout function called by the driver at the end of the QSPI Init phase.

The intended purpose of this callout is to provide to the application the possibility of performing additional configuration to the QSPI hardware IP or to the external memories connected(for ex: sending the lock/unlock sequences for the external flash sectors, altering QSPI IP timing, etc.)

Note: Disable the callout in order to have it set as NULL_PTR.

Note: The callout can be configured only if the FlsExternalDriver is enabled.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	FlsQspiInitCallout

4.1.16 Parameter FlsQspiResetCallout

Vendor specific: Callout function called by the driver at the beginning of a new job. The intended purpose of this callout is to provide to the application the possibility of resetting the external memory to an idle and error free state.

If the callout is disabled, at the beginning of a new job the `Fls_MainFunction` will check the external memory status and if not, poll and wait for it to become idle.

If the callout is enabled and the memory is not idle, the `Fls_MainFunction` will also call the configured function to allow the application to send extra commands to the external memory (software reset, abort any suspended operation, error flags clearing, etc.)

Note: Disable the callout in order to have it set as `NULL_PTR`.

Note: The callout can be configured only if the `FlsExternalDriver` is enabled.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FlsQspiResetCallout

4.1.17 Parameter `FlsQspiErrorCheckCallout`

Vendor specific: Callout function called by the driver at the end of each program and erase job

The intended purpose of this callout is to provide to the application the possibility of interrogating the error status of the memory after each program and erase job.

The application should check any error or status bits available and reset the memory after interrogation in case an error condition was detected.

If the callout is enabled, at the end of each job, the callout is called and the return value is checked to determine if there was any error during the memory operation.

Return values: `E_OK(0)` `E_NOT_OK(1)`.

If `E_OK(0)` is received, the job is considered successful.

If `E_NOT_OK(1)` is received, the job is considered unsuccessful and marked as failed.

If the callout is disabled, the job is assumed as successful from the memory status point of view.

Note: Disable the callout in order to have it set as `NULL_PTR`.

Note: The callout can be configured only if the `FlsExternalDriver` is enabled.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	FlsQspiErrorCheckCallout

4.1.18 Parameter FlsQspiEccCheckCallout

Vendor specific: Callout function called by the driver at the end of each read operation

The intended purpose of this callout is to provide to the application the possibility of interrogating the ECC status of the memory after each read operation.

The callout provide the hardware channel, start address and the size of the read operation.

The application should check there is any ECC error in the current read data

If the callout is enabled, at the end of each read operation, the callout is called and the return value is checked to determine if there was any error during the memory operation.

Return values: E_OK(0) E_NOT_OK(1).

If E_OK(0) is received, the job is considered successful.

If E_NOT_OK(1) is received, the job is considered unsuccessful and marked as failed.

If the callout is disabled, the job is assumed as successful from the memory status point of view.

Note: Disable the callout in order to have it set as NULL_PTR.

Note: The callout can be configured only if the FlsExternalDriver is enabled.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	FlsQspiEccCheckCallout

4.1.19 Parameter FlsMaxReadFastMode

The maximum number of bytes to read or compare in one cycle of the flash driver's job processing function in fast mode.

Note: If external sectors are configured and if FlsHwUnitWordAddressable is set, the FlsMaxReadFastMode must be an even value(two bytes aligned).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	1048576
max	4294967295
min	0

4.1.20 Parameter FlsMaxReadNormalMode

The maximum number of bytes to read or compare in one cycle of the flash driver's job processing function in normal mode.

Note: If external sectors are configured and if FlsHwUnitWordAddressable is set, the FlsMaxReadNormalMode must be an even value(two bytes aligned).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	1024
max	4294967295
min	0

4.1.21 Parameter FlsMaxWriteFastMode

The maximum number of bytes to write in one cycle of the flash driver's job processing function in fast mode.

Note: If external sectors are configured, the FlsMaxWriteFastMode must be an integer multiple of the FlsPageSize.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	256
max	4294967295
min	0

4.1.22 Parameter FlsMaxWriteNormalMode

The maximum number of bytes to write in one cycle of the flash driver's job processing function in normal mode.

Note: If external sectors are configured, the FlsMaxWriteFastMode must be an integer multiple of the FlsPageSize.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	8
max	4294967295
min	0

4.1.23 Parameter FlsProtection

Erase/write protection settings. Note: Not supported by the driver.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.24 Container FlsExternalDriver

This container is present for external Flash drivers only. Internal Flash drivers do not use the parameter listed in this container, hence its multiplicity is 0 for internal drivers.

Included subcontainers:

- [ControllerCfg](#)
- [MemCfg](#)
- [FlsController](#)
- [FlsMem](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.25 Reference FlsSpiReference

Reference to SPI sequence. Not used in current implementation.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	true
destination	/AUTOSAR/EcucDefs/Spi/SpiDriver/SpiSequence

4.1.26 Container ControllerCfg

Vendor specific: Container for defining configurations for QSPI controllers.

These configurations are not tied to a particular controller.

Included subcontainers:

- [FlsAhbBuffer](#)
- [FlsHwUnitSpecificSettings](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.27 Parameter FlsHwUnitReadMode

Vendor specific: The hardware unit read mode:

QSPI_IP_DATA_RATE_SDR (single data rate) which samples incoming data on a single edge.

QSPI_IP_DATA_RATE_DDR (double data rate) which samples incoming data on both edges.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_DATA_RATE_SDR
literals	['QSPI_IP_DATA_RATE_SDR', 'QSPI_IP_DATA_RATE_DDR']

4.1.28 Parameter FlsSerialFlashA1Size

Vendor specific: Size of flash device connected to side A1 of the controller. Set to 0 if no flash device is connected.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.29 Parameter FlsSerialFlashA2Size

Vendor specific: Size of flash device connected to side A2 of the controller. Set to 0 if no flash device is connected.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.30 Parameter FlsSerialFlashB1Size

Vendor specific: Size of flash device connected to side B1 of the controller. Set to 0 if no flash device is connected.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE

Configuration Parameters

Property	Value
defaultValue	0
max	4294967295
min	0

4.1.31 Parameter FlsSerialFlashB2Size

Vendor specific: Size of flash device connected to side B2 of the controller. Set to 0 if no flash device is connected.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.32 Parameter FlsHwUnitSamplingModeA

Vendor specific: It selects DQS clock for sampling read data at Flash A QuadSPI port:

QSPI_IP_READ_MODE_INTERNAL_DQS = DQS internal (Default).

QSPI_IP_READ_MODE_LOOPBACK = Pad loopback.

QSPI_IP_READ_MODE_LOOPBACK_DQS = DQS pad loopback.

QSPI_IP_READ_MODE_EXTERNAL_DQS = External DQS.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_READ_MODE_INTERNAL_DQS

Property	Value
literals	['QSPI_IP_READ_MODE_INTERNAL_DQS', 'QSPI_IP_READ_MODE←_LOOPBACK']

4.1.33 Parameter FlsHwUnitSamplingModeB

Vendor specific: It selects DQS clock for sampling read data at Flash B QuadSPI port:

QSPI_IP_READ_MODE_INTERNAL_DQS = DQS internal (Default).

QSPI_IP_READ_MODE_LOOPBACK = Pad loopback.

QSPI_IP_READ_MODE_LOOPBACK_DQS = DQS pad loopback.

QSPI_IP_READ_MODE_EXTERNAL_DQS = External DQS.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_READ_MODE_INTERNAL_DQS
literals	['QSPI_IP_READ_MODE_INTERNAL_DQS', 'QSPI_IP_READ_MODE←_LOOPBACK', 'QSPI_IP_READ_MODE_EXTERNAL_DQS']

4.1.34 Parameter IdleSignalDriveIOFB3HighLvl

Vendor specific: Idle Signal Drive IOFB[3] Flash B. This bit determines the logic level the IOFB[3] output of the QuadSPI module is driven to in the inactive state.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.35 Parameter IdleSignalDriveIOFB2HighLvl

Vendor specific: Idle Signal Drive IOFB[2] Flash B. This bit determines the logic level the IOFB[2] output of the QuadSPI module is driven to in the inactive state.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.36 Parameter IdleSignalDriveIOFA3HighLvl

Vendor specific: Idle Signal Drive IOFA[3] Flash A. This bit determines the logic level the IOFA[3] output of the QuadSPI module is driven to in the inactive state.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.37 Parameter IdleSignalDriveIOFA2HighLvl

Vendor specific: Idle Signal Drive IOFA[2] Flash A. This bit determines the logic level the IOFA[2] output of the QuadSPI module is driven to in the inactive state.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.38 Parameter FlsHwUnitSamplingEdge

Vendor specific: Full-speed phase selection for SDR instructions.

This field selects the edge of the sampling clock valid for full-speed commands.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_SAMPLE_PHASE_NON_INVERTED
literals	['QSPI_IP_SAMPLE_PHASE_NON_INVERTED', 'QSPI_IP_SAMPLE_PHASE_INVERTED']

4.1.39 Parameter FlsHwUnitSamplingDly

Vendor specific: Full-speed delay selection for internal/pad loop back DQS sampling.

This field selects the delay in accordance with the reference edge for the valid sample point.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Configuration Parameters

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_SAMPLE_DELAY_SAME_DQS
literals	['QSPI_IP_SAMPLE_DELAY_SAME_DQS', 'QSPI_IP_SAMPLE_DELAY_SAME_DQS', 'QSPI_IP_SAMPLE_DELAY_SAME_DQS', 'QSPI_IP_SAMPLE_DELAY_SAME_DQS']

4.1.40 Parameter FlsHwUnitDqsLatencyEnable

Vendor specific: DQS Latency Enable. Feature used to support external devices which add latency cycles in the DQS signal. When no signal is provided by the external devices, data is not sampled, thus the memory stretches the timing.

DQS Latency is applicable only for DQS sampling mode: FLS_EXTERNAL_DQS.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.41 Parameter FlsHwUnitTdh

Vendor specific: TDH: Serial flash data in hold time. Should be set to QSPI_IP_FLASH_DATA_ALIGN_REFCLK for QSPI_IP_DATA_RATE_SDR mode.

TDH parameter delays data sent to flash, in order to meet the input hold time requirement of flash.

QSPI_IP_FLASH_DATA_ALIGN_REFCLK = Data aligned with the posedge of Internal reference clock of QuadSPI.

QSPI_IP_FLASH_DATA_ALIGN_2X_REFCLK = Data aligned with 2x serial flash half clock.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_FLASH_DATA_ALIGN_REFCLK
literals	['QSPI_IP_FLASH_DATA_ALIGN_REFCLK', 'QSPI_IP_FLASH_DATA_ALIGN_2X_REFCLK']

4.1.42 Parameter FlsHwUnitTcsh

Vendor specific: TCSH: Serial flash CS hold time in terms of serial flash clock cycles.

A bigger value will release the CS signal later after the transaction ends.

The actual delay between chip select and clock is defined as:

$TCSH = 1 \text{ SCK clk if } N = 0/1 \text{ else, } N \text{ SCK clk if } N > 1$, where N is the setting of TCSH

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	3
max	15
min	0

4.1.43 Parameter FlsHwUnitTcss

Vendor specific: TCSS: Serial flash CS setup time in terms of serial flash clock cycles.

A bigger value will pull the CS signal earlier before the transaction starts.

The actual delay between chip select and clock is defined as:

$TCSS = 0.5 \text{ SCK clk if } N = 0/1 \text{ else, } N + 0.5 \text{ SCK clk if } N > 1$, where N is the setting of TCSS.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	3
max	15
min	0

4.1.44 Parameter FlsHwUnitColumnAddressWidth

Vendor specific: Column Address Space. Defines the width of the column address.

Example: If the column address is for example [2:0] of QSPI_SFAR/AHB address, then CAS must be 3. If there is no column address separation in any serial flash this value must be programmed to 0.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.1.45 Parameter FlsHwUnitWordAddressable

Vendor specific: Defines whether the serial flash is a byte addressable flash or a word addressable flash.

According to this bit configuration the address is re-mapped to the flash interface.

DISABLED: Byte addressable serial flash mode.

ENABLED: Word (2 byte) addressable serial flash mode. If the incoming address is 0x2004, the controller re-maps this address to access the flash location 0x1002.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.46 Container FlsAhbBuffer

Container for the configuration of the AHB read buffers. Holds the configuration for each AHB buffer configured for AHB read mode.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	4
upperMultiplicity	4
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.47 Parameter FlsAhbBufferInstance

Vendor specific: Selects the AHB buffer instance for which this configuration applies.

If an instance is not present, the corresponding AHB buffer will be configured with size 0.

The size of the AHB_BUFFER_3 instance will be configured to at least the selected size, or more, up until the maximum value is reached. For more details about the maximum available size see chip specific details.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	AHB_BUFFER_1
literals	['AHB_BUFFER_0', 'AHB_BUFFER_1', 'AHB_BUFFER_2', 'AHB_BUFFER_3']

4.1.48 Parameter FlsAhbBufferMasterId

Vendor specific: The ID of the AHB master associated with this buffer. Any AHB access with this master port number is routed to this buffer. It must be ensured that the master IDs associated with all buffers must be different.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.1.49 Parameter FlsAhbBufferSize

Vendor specific: The size allocated to this AHB Buffer instance. The minimum size is 8 bytes, the maximum size is the entire AHB Buffer.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.50 Parameter FlsAhbBufferAllMasters

Vendor specific: When set, buffer3 acts as an all-master buffer. Any AHB access with a master port number not matching with the master ID of buffer0 or buffer1 or buffer2 is routed to buffer3.

When set, the Master ID parameter for this buffer is ignored.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.51 Container FlsHwUnitSpecificSettings

Vendor specific: Container for clock options and chip settings.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.52 Parameter FlsHwUnitInputClockSelect

Vendor specific: Select source for AHB read interface clock, module clock and bus interface clock.

SCLKCFG[6] bit.

Option 1: QSPI_IP_CLK_SRC_BUS_CLK.

Option 2: QSPI_IP_CLK_SRC_SYS_CLK.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

Configuration Parameters

Property	Value
defaultValue	QSPI_IP_CLK_SRC_SYS_CLK
literals	['QSPI_IP_CLK_SRC_SYS_CLK', 'QSPI_IP_CLK_SRC_BUS_CLK']

4.1.53 Parameter FlsHwUnitInternalRefClockSelect

Vendor specific: Internal reference clock (async clock domain) source selection for Quadspi.

SCLKCFG[4] bit.

Option 1: QSPI_IP_CLK_REF_PLL_DIV1.

Option 2: QSPI_IP_CLK_REF_FIRC_DIV1.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_CLK_REF_FIRC_DIV1
literals	['QSPI_IP_CLK_REF_PLL_DIV1', 'QSPI_IP_CLK_REF_FIRC_DIV1']

4.1.54 Parameter FlsHwUnitProgrammableDivider

Vendor specific: Programmable divider configuration selection. Based on this divider value, the external memory clock and internal sampling clock is derived.

SOC CFG[31:29] bits.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2
max	8

Property	Value
min	1

4.1.55 Parameter FlsHwUnitInvertClkDqsA

Vendor specific: Reference clock selection for DQS for Flash-A.

SCLKCFG[3] bit.

True : Inverted Clock from 'Fls Hw Unit Sampling Mode A' selected as DQS.

False: Clock from 'Fls Hw Unit Sampling Mode A' selected as DQS.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.56 Parameter FlsHwUnitInvertClkDqsB

Vendor specific: Reference clock selection for DQS for Flash-B.

SCLKCFG[3] bit.

True : Inverted Clock from 'Fls Hw Unit Sampling Mode B' selected as DQS.

False: Clock from 'Fls Hw Unit Sampling Mode B' selected as DQS.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.57 Parameter FlsHwUnitFineDelayA

Vendor specific: Fine delay chain configuration for Flash A.

SOCCFG[7:0] bits.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	127
min	0

4.1.58 Parameter FlsHwUnitFineDelayB

Vendor specific: Fine delay chain configuration for Flash B.

SOCCFG[15:8] bits.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	127
min	0

4.1.59 Container MemCfg

Vendor specific: Container for defining configurations for serial flash devices.

These configurations are not tied to a particular device instance.

Included subcontainers:

- [MemCfgReadIdSettings](#)
- [MemCfgEraseSettings](#)
- [statusConfig](#)
- [suspendSettings](#)
- [resetSettings](#)
- [initResetSettings](#)
- [initConfiguration](#)
- [FlsLUT](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.60 Parameter MemCfgSize

Vendor specific: The size in bytes of this flash device.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	8388608
max	4294967295
min	0

4.1.61 Parameter MemCfgPageSize

Vendor specific: The page size in bytes of this flash device.

Page size is the maximum amount of data that the flash device can write in a single write operation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Configuration Parameters

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	256
max	4294967295
min	0

4.1.62 Reference MemCfgReadLUT

Vendor specific: Reference to the LUT Sequence ID which will be used for read operations

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.63 Reference MemCfgWriteLUT

Vendor specific: Reference to the LUT Sequence ID which will be used for write operations

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False

Property	Value
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.64 Reference MemCfgRead0xxLUT

Vendor specific: Reference to the LUT Sequence ID which will be used for optimized read operations, if supported by the device.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.65 Reference MemCfgRead0xxLUTAHB

Vendor specific: Reference to the LUT Sequence ID which will be used for optimized read operations through AHB reads, if supported by the device.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.66 Reference ctrlAutoCfgPtr

Vendor specific: Reference to configuration which will be used for initializing the controller when the flash device is initialized.

This is needed for devices which need to change controller configuration during device initialization (e.g. switch to External DQS after activating DOPI mode).

Configuration Parameters

Resetting the flash device will re-apply this configuration.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/ControllerCfg

4.1.67 Container MemCfgReadIdSettings

Vendor specific: Container for Read Device/Manufacturer ID command

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.68 Parameter MemCfgReadIdSize

Vendor specific: The size in bytes of the information returned by the readId command.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	3

Property	Value
max	4
min	0

4.1.69 Parameter FlsQspiDeviceId

Vendor specific: External memory ID. If the associated "FLS_E_UNEXPECTED_FLASH_ID" error is enabled, at Init, the configured value is checked against the value read from memory.

The memory ID is read from the memory using the configured READ_ID LUT sequence.

Example for a Macronix device:

Configured value of FlsQspiDeviceId = 0x3A:81:C2, meaning Memory density: 0x3A, Memory type: 0x81, Manufacturer ID: 0xC2.

The configured READ_ID LUT sequence schedules a read id command (ex: RDID 0x9F) with read length 3 bytes.

Note: This parameter can be configured only when Read Id LUT index reference is used.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0x3A:81:C2

4.1.70 Reference MemCfgReadIdLUT

Vendor specific: Reference to the LUT Sequence ID which will be used for reading device/manufacture Id.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.71 Container MemCfgEraseSettings

Vendor specific: Container for erase commands supported by the device

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.72 Parameter MemCfgErase1Size

Vendor specific: The size in bytes of the erased area: 2^{size} ; e.g. 0x0C means 4 Kbytes

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	12
max	32
min	1

4.1.73 Parameter MemCfgErase2Size

Vendor specific: The size in bytes of the erased area: 2^{size} ; e.g. 0x0C means 4 Kbytes

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	12
max	32
min	1

4.1.74 Parameter MemCfgErase3Size

Vendor specific: The size in bytes of the erased area: $2^{\wedge}$ size; e.g. 0x0C means 4 Kbytes

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	12
max	32
min	1

4.1.75 Parameter MemCfgErase4Size

Vendor specific: The size in bytes of the erased area: $2^{\wedge}$ size; e.g. 0x0C means 4 Kbytes

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	12
max	32
min	1

4.1.76 Reference MemCfgErase1LUT

Vendor specific: Reference to the LUT Sequence ID for erase type 1.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.77 Reference MemCfgErase2LUT

Vendor specific: Reference to the LUT Sequence ID for erase type 2.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.78 Reference MemCfgErase3LUT

Vendor specific: Reference to the LUT Sequence ID for erase type 3.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.79 Reference MemCfgErase4LUT

Vendor specific: Reference to the LUT Sequence ID for erase type 4.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.80 Reference ChipEraseLUT

Vendor specific: Reference to the LUT Sequence ID for chip erase command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.81 Container statusConfig

Vendor specific: Container for settings related to the status register of the flash device

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.82 Parameter regSize

Vendor specific: The size in bytes of the status register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4
min	1

4.1.83 Parameter busyOffset

Vendor specific: Position of "busy" bit inside status register. This bit indicates whether the device is busy with a high voltage operation or not.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	31
min	0

4.1.84 Parameter busyValue

Vendor specific: Value of "busy" bit which indicates that the device is busy; can be 0 or 1

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	1
min	0

4.1.85 Parameter writeEnableOffset

Vendor specific: Position of "write enable" bit inside the status register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	31
min	0

4.1.86 Parameter blockProtectionOffset

Vendor specific: Offset of block protection bits inside the status register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2
max	31
min	0

4.1.87 Parameter blockProtectionWidth

Vendor specific: Width of block protection bitfield inside the status register

A value of 0 disables protection setting.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	4
max	32
min	0

4.1.88 Parameter blockProtectionValue

Vendor specific: Value of block protection bitfield inside the status register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.1.89 Reference statusRegInitReadLut

Vendor specific: Reference to the LUT Sequence ID for Read status register command.

This sequence is used during the initialization stage.

For example if the initial state of the flash is SPI, this should be a SPI sequence.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.90 Reference statusRegReadLut

Vendor specific: Reference to the LUT Sequence ID for Read status register command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Configuration Parameters

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.91 Reference statusRegWriteLut

Vendor specific: Reference to the LUT Sequence ID for Write status register command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.92 Reference writeEnableSRLut

Vendor specific: Reference to the LUT Sequence ID for Status register write enable command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.93 Reference writeEnableLut

Vendor specific: Reference to the LUT Sequence ID for Write enable command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.94 Container suspendSettings

Vendor specific: Container related to write/erase suspend and resume commands, if supported by the device.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.95 Reference eraseSuspendLut

Vendor specific: Reference to the LUT Sequence ID for Erase suspend command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.96 Reference eraseResumeLut

Vendor specific: Reference to the LUT Sequence ID for Erase resume command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.97 Reference programSuspendLut

Vendor specific: Reference to the LUT Sequence ID for Program suspend command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.98 Reference programResumeLut

Vendor specific: Reference to the LUT Sequence ID for Program resume command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false

Property	Value
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.99 Container resetSettings

Vendor specific: Container related to software reset command, for resetting the flash device.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.100 Parameter resetCmdCount

Vendor specific: Number of commands in the reset sequence

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	255
min	1

4.1.101 Reference resetCmdLut

Vendor specific: Reference to the LUT Sequence ID for the first command from the reset sequence.

Configuration Parameters

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.102 Container initResetSettings

Vendor specific: Container related to software reset command, for resetting the flash device. This reset procedure applies only at driver initialization. It might be different from the normal reset command, depending on the initial state of the flash. If not, set the same as reset command.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.103 Parameter resetCmdCount

Vendor specific: Number of commands in the reset sequence

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	1

Property	Value
max	255
min	1

4.1.104 Reference resetCmdLut

Vendor specific: Reference to the LUT Sequence ID for the first command from the reset sequence.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.105 Container initConfiguration

Vendor specific: This container describes the list of operations which must be performed at initialization time to bring the memory in the desired operating state.

Example: activate XPI mode, activate 4-byte addressing.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	255
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.106 Parameter opType

Vendor specific: Operation type can be one of the following:

QSPI_IP_OP_TYPE_CMD - Simple command

QSPI_IP_OP_TYPE_WRITE_REG - Write value in external flash register

QSPI_IP_OP_TYPE_RMW_REG - RMW command on external flash register

Configuration Parameters

QSPI_IP_OP_TYPE_READ_REG - Read external flash register until expected value is read

QSPI_IP_OP_TYPE_QSPI_CFG - Re-configure QSPI controller

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_OP_TYPE_CMD
literals	['QSPI_IP_OP_TYPE_CMD', 'QSPI_IP_OP_TYPE_WRITE_REG', 'QSPI_IP_OP_TYPE_RMW_REG', 'QSPI_IP_OP_TYPE_READ_REG', 'QSPI_IP_OP_TYPE_QSPI_CFG']

4.1.107 Parameter addr

Vendor specific: Address, if used in command

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.108 Parameter size

Vendor specific: Size in bytes of configuration register, where it applies.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4
min	1

4.1.109 Parameter shift

Vendor specific: Offset of configuration field, where it applies.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	32
min	0

4.1.110 Parameter width

Vendor specific: Width of configuration field, where it applies.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Configuration Parameters

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	32
min	0

4.1.111 Parameter value

Vendor specific: Value to set/expect in the bit-field.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4294967295
min	0

4.1.112 Reference command1Lut

Vendor specific: Index of first command sequence in Lut; for RMW type this is the read command

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.113 Reference command2Lut

Vendor specific: Index of second command sequence in Lut, only used for RMW type, this is the write command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.114 Reference weLut

Vendor specific: Index of write enable command, if needed before a write command. Only used for Write and RMW operations.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.1.115 Reference ctrlCfgPtr

Vendor specific: Reference to configuration which will be used for initializing the controller.

Valid only for QSPI_IP_OP_TYPE_QSPI_CFG operations

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/ControllerCfg

4.1.116 Container FlsLUT

Vendor specific: Container for the configuration of the Look Up Table holding all the Instruction/Operands sequences.

A sequence consists of a series of up to 8 instruction/operands pairs, which can occupy up to 4 LUTs, which are executed whenever a command is triggered to the external flash memory.

Included subcontainers:

- [FlsInstructionOperandPair](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	65534
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.117 Parameter FlsLUTIndex

Vendor specific: Fls LUT Index is an invariant index, used to order the LUT entries and loop over them in the correct, configured order. Its value should be equal with the position of the configured LUT inside the configured LUT list (the same value as the shown index).

Rationale: The generated .epc configuration might reorder the LUT elements (alphabetically), thus the default index parameter changes, becoming out of sync with the real intended order (the values are not generated in the intended order, they are sorted).

Range:

min = 0

max = 65534

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	65534
min	0

4.1.118 Container FlsInstructionOperandPair

Vendor specific: One command set which holds one memory command-operand pair.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.119 Parameter FlsInstrOperPairIndex

Vendor specific: Fls Instruction Operand Pair Index is an invariant index, used to order the Instr.Oper. entries and loop over them in the correct, configured order. Its value should be equal with the position of the configured pair inside the configured pair list (the same value as the shown index).

Rationale: The generated .epc configuration might reorder the instr.oper. pairs (alphabetically), thus the index parameter changes, becoming out of sync with the real intended order (the values are not generated in the intended order, they are sorted).

Range:

min = 0

max = 65534

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Configuration Parameters

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	65534
min	0

4.1.120 Parameter FlsLUTInstruction

Vendor specific: The instruction type used to identify the command used by the QSPI IP when sending the command to the memory.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_LUT_INSTR_CMD
literals	['QSPI_IP_LUT_INSTR_CMD', 'QSPI_IP_LUT_INSTR_ADDR', 'QSPI_IP_LUT_INSTR_DUMMY', 'QSPI_IP_LUT_INSTR_MODE', 'QSPI_IP_LUT_INSTR_MODE2', 'QSPI_IP_LUT_INSTR_MODE4', 'QSPI_IP_LUT_INSTR_READ', 'QSPI_IP_LUT_INSTR_WRITE', 'QSPI_IP_LUT_INSTR_JMP_ON_CS', 'QSPI_IP_LUT_INSTR_ADDR_DDR', 'QSPI_IP_LUT_INSTR_MODE_DDR', 'QSPI_IP_LUT_INSTR_MODE2_DDR', 'QSPI_IP_LUT_INSTR_MODE4_DDR', 'QSPI_IP_LUT_INSTR_READ_DDR', 'QSPI_IP_LUT_INSTR_WRITE_DDR', 'QSPI_IP_LUT_INSTR_CMD_DDR', 'QSPI_IP_LUT_INSTR_CADDR', 'QSPI_IP_LUT_INSTR_CADDR_DDR', 'QSPI_IP_LUT_INSTR_STOP']

4.1.121 Parameter FlsLUTPad

Vendor specific: Number of pads/pins used for the current command.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_LUT_PADS_1
literals	['QSPI_IP_LUT_PADS_1', 'QSPI_IP_LUT_PADS_2', 'QSPI_IP_LUT_PADS_4', 'QSPI_IP_LUT_PADS_8']

4.1.122 Parameter FlsLUTOperand

Vendor specific: The operand of the instruction command sent to memory.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.1.123 Container FlsController

Container for selecting the start configuration of QSPI controllers.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.124 Parameter ControllerName

Vendor specific: The name of the configured hardware unit name. The configured parameters will apply to this hardware unit name only.

The name of the hardware unit name represents the physical hardware unit available on chip.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FLS_QSPI_0
literals	['FLS_QSPI_0']

4.1.125 Reference FlsControllerCfgRef

Vendor specific: Reference to configuration which will be used for initializing the controller.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/ControllerCfg

4.1.126 Container FlsMem

Container for selecting the start configuration and connection information of serial flash devices.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF

Property	Value
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.127 Parameter FlsMemName

Vendor specific: The name of the configured flash device. The configured parameters will apply to this device only.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	Device_0

4.1.128 Parameter MemAlignment

Vendor specific: The address alignment required by the external flash (1, 2 or 4 bytes, ...), needed in the OCTA DTR Mode (DOP) or hyperbus devices.

For read operation:

- The driver will decrease the address if it is not aligned, and increasing the size to compensate.
- After the actual read, the driver ignores the first few bytes before starting the copy/comparison to the user data.

For write operation: send extra data with FFh to overwrite the overlapping memory area

? If there is a need to program from odd starting address, keep the even input address and the input data shall start with FFh.

? If there is a need to program with odd ending address, simply provide extra data with FFh in the last falling edge of clock.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	16
min	1

4.1.129 Parameter AHBReadEnable

Vendor specific: When set, Qspi_Ip_AhbReadEnable() will be called from Fls_Init() to allow reads via AHB.

The application can read directly through Flash memory devices address mapping.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.130 Parameter FlsMemUseSfdp

Vendor specific: Select this option to attempt auto-configuration using the information read from the SFDP table

This only works for flash devices which support the SFDP feature.

SFDP (Serial Flash Discoverable Parameters) is a JEDEC standard - JESD216D.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.131 Parameter connectionType

Vendor specific: Connection type of the flash device to the controller:

QSPI_IP_SIDE_A1 - Serial flash connected on side A1

QSPI_IP_SIDE_A2 - Serial flash connected on side A2

QSPI_IP_SIDE_B1 - Serial flash connected on side B1

QSPI_IP_SIDE_B2 - Serial flash connected on side B2

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_SIDE_A1
literals	['QSPI_IP_SIDE_A1', 'QSPI_IP_SIDE_A2', 'QSPI_IP_SIDE_B1', 'QSPI_IP_SIDE_B2']

4.1.132 Reference FlsMemCfgRef

Vendor specific: Reference to configuration which will be used for initializing the flash memory device.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destinations	['/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg']

4.1.133 Reference qspiInstance

Vendor specific: QSPI controller instance to which this flash device is connected.

Configuration Parameters

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/FlsController

4.1.134 Container FlsSectorList

List of flashable sectors and pages.

Included subcontainers:

- [FlsSector](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.135 Container FlsSector

Configuration description of a flashable sector

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD

4.1.136 Parameter FlsSectorIndex

Vendor specific: Fls Sector Index is an invariant index, used to order flash sectors and loop over them in the correct, configured order. Its value should be equal with the position of the configured sector inside the configured sector list (the same value as the shown index).

Rationale: The generated .epc configuration might reorder the flash sectors(alphabetically), thus the index parameter changes, becoming out of sync with the real intended order (for example: Fls Sector Start Addresses).

Range:

min = 0

max = 65534

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	65534
min	0

4.1.137 Parameter FlsPhysicalSector

Vendor specific: Physical flash device sector.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FLS_DATA_ARRAY_0_BLOCK_3_S000

[illegible]

Property	Value
----------	-------

4.1.138 Parameter FlsNumberOfSectors

Number of continuous sectors with the above characteristics.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	1
max	65535
min	1

4.1.139 Parameter FlsPageSize

Size of one page of this sector. Implementation Type: Fls_LengthType.

For internal flash, page size is 8 byte

For external flash, page size is chip specific.

For example: In Macronix devices, the ECC algorithm uses a Hamming code that can correct a single bit error per 16-Byte page.

It is recommended that data be programmed in multiples of 16 bytes using the Page Program command instead of programming a byte or a word at a time using the Program command.

Each group of 16 bytes must fall within the same 16-Byte boundary.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	8
max	4294967295
min	0

4.1.140 Parameter FlsSectorSize

Size of this sector. Implementation Type: Fls_LengthType.

Note: Size of the sector should be a multiple of the page size.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	4096
max	4294967295
min	0

4.1.141 Parameter FlsSectorStartaddress

Start address of this sector.

Implementation Type: Fls_AddressType.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.142 Parameter FlsSectorEraseAsynch

Vendor specific: Enable asynchronous execution of the erase job in the Fls_MainFunction function which doesn't wait (block) for completion of the sector erase operation. The flash driver doesn't use the erase access code to the erase flash sector in asynchronous mode so it can be used only on flash sectors which belong to flash array different from flash array the

application is executing from.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.143 Parameter FlsPageWriteAsynch

Vendor specific: Enable asynchronous execution of the write job in the Fls_MainFunction function which doesn't wait (block) for completion of the page write operation(s). The flash driver doesn't use the write access code to the write flash page(s) in asynchronous mode so it can be used only on flash sectors which belong to flash array different from flash array the application is executing from.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.144 Parameter FlsHwCh

Vendor specific: The hardware channel type of the current sector: internal flash or external QSPI flash sector.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	FLS_CH_INTERN
literals	['FLS_CH_INTERN', 'FLS_CH_QSPI']

4.1.145 Parameter FlsSectorHwAddress

Vendor specific: Hardware address of this sector, as needed by the external flash device (usually starting from 0).

Applicable only to external sectors. This value is used to access the hardware sector on the attached device and will be sent as parameter of flash comands, so it should be completed to meet the requirements of the external flash memory type and configured operating mode.

Internally, this address is added to the MCU base addresses of each channel, configured in SF{A/B}{1/2}AD registers, in order to select the corresponding external device hw channel.

Example: FlsSectorHwAddress = 0x100

Sector hardware channel = FLS_CH_EXTERN_QSPI_0_A2

FlsSerialFlashA1TopAddr = 0x24000000

The address used by the driver internally will be 0x24000100, thus selecting external flash device A2 and accessing internal location 0x100 of the memory.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	4294967295
min	0

4.1.146 Reference flashInstance

Vendor specific: External flash device instance to which this sector belongs.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Fls/FlsConfigSet/FlsExternalDriver/FlsMem

4.1.147 Container AutosarExt

Vendor specific: This container contains the global Non-Autosar configuration parameters of the Fls driver.

This container is a MultipleConfigurationContainer, i.e. this container and its sub-containers exist once per configuration set.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.148 Parameter FlsEnableUserModeSupport

Vendor specific: When this parameter is enabled, the FLS module will adapt to run from User Mode, with the following measures: configuring REG_PROT for Fls IPs so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1

for more information and availability on this platform, please see chapter User Mode Support in IM

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.149 Parameter FlsQspiLockLUT

Vendor specific: Enable the Lock/Unlock of the LUT for the external QuadSPI memory.

If Enabled, the LUT is unlocked at the beginning of the Init phase and locked at the end of it.

If Disabled, the LUT has to be unlocked if the Init phase is supposed to populate it.

Note: not used in the driver code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.150 Parameter FlsQspiHangRecovery

Vendor specific: Enable the hang recovery feature for the external QuadSPI controller.

If Enabled, the driver will perform the software reset sequence in case of being stuck in BUSY state.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.151 Parameter FlsSynchronizeCache

Vendor specific:

Synchronize the memory by invalidating the cache after each flash hardware operation.

The FLS driver needs to maintain the memory coherency by means of three methods:

- 1. Disable data cache, or*
- 2. Configure the flash region upon which the driver operates, as non-cacheable, or*
- 3. Enable the FlsSynchronizeCache feature.*

Depending on the application configuration, one option may be more beneficial than other.

Enabled: The FLS driver will call Mcl cache API functions in order to invalidate the cache after each high voltage operation(write,erase) and before each read operation, in order to ensure that the cache and the modified flash memory are in sync.

If enabled, the driver will attempt to invalidate only the modified lines from the cache.

If the size of the region to be invalidated is greater than half of the cache size, then the entire cache is invalidated.

Note: If enabled, the McLmemEnableCacheApi parameter has to be enabled and the MCL plugin included as a dependency.

Disabled: The upper layers have to ensure that the flash region upon which the driver operates is not cached.

This can be obtained by either disabling the data cache or by configuring the memory region as non-cacheable.

Note: This feature is applicable only if supported on the current platform.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.152 Parameter FlsInvalidPrefetchBufFromRam

Vendor specific: Allow to allocate the function invalidation the prefetch buffer of internal flash on RAM or FLASH.

Rationale: On some platforms the RAM may not be executable due to security restrictions.

If Enabled: the invalidate prefetch buffer functionality is executable from RAM.

If Disabled: the invalidate prefetch buffer functionality is executable from FLASH.

Note: For more information please see the chapter Tips for FLS integration in the Integration Manual.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.153 Parameter FlsInternalSectorsConfigured

Vendor specific:

Boolean parameter which must be enabled if internal flash sectors are configured.

Enabled: At least one internal flash sector is configured in any variant.

Disabled: No internal flash sector is configured in any variant, only external flash sectors are present.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.154 Parameter FlsExternalSectorsConfigured

Vendor specific:

Boolean parameter which must be enabled if external flash sectors are configured.

Enabled: At least one external flash sector is configured in any variant.

Disabled: No external flash sector is configured in any variant, only internal flash sectors are present.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.155 Container FlsGeneral

Container for general parameters of the flash driver. These parameters are always pre-compile.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.156 Parameter FlsEnableDevAssert

Vendor specific:

true: Development error checking at IP level is enabled.

false: Development error checking at IP level is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.157 Parameter FlsEnableCheckCfgCrc

Vendor specific:

Configuration Parameters

true: Enable calculates CRC for Fls Configuration

false: Disable calculates CRC for Fls Configuration.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.158 Parameter FlsAcLoadOnJobStart

The flash driver shall load the flash access code to RAM whenever an erase or write job is started and unload (overwrite) it after that job has been finished or canceled.

true: Flash access code loaded on job start / unloaded on job end or error.

false: Flash access code not loaded to / unloaded from RAM at all.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.159 Parameter FlsCleanCacheAfterLoadAc

Vendor specific: Pre-processor switch to allow to clean cache after loading AccessCode to RAM to ensure the synchronization between cache and RAM memory.

This action might be needed in case the AccessCode function is copied to a cacheable area.

true: cleans cache after loading AccessCode function to RAM to write cache data to the actual RAM memory

false: does not clean cache after loading AccessCode function to RAM

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.160 Parameter FlsBaseAddress

The flash memory start address (see also FLS118).

FLS169: This parameter defines the lower boundary for read / write / erase and compare jobs. Note: Not needed / supported by the driver.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	4294967295
min	0

4.1.161 Parameter FlsBlankCheckApi

Compile switch to enable and disable the Fls_BlankCheck function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.162 Parameter FlsCancelApi

Compile switch to enable and disable the Fls_Cancel function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.163 Parameter FlsCompareApi

Compile switch to enable and disable the Fls_Compare function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.164 Parameter FlsDevErrorDetect

Pre-processor switch to enable and disable development error detection.

true: Development error detection enabled.

false: Development error detection disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.165 Parameter FlsDriverIndex

Index of the driver, used by FEE.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	254
min	0

4.1.166 Parameter FlsGetJobResultApi

Compile switch to enable and disable the Fls_GetJobResult function.

Configuration Parameters

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.167 Parameter FlsGetStatusApi

Compile switch to enable and disable the Fls_GetStatus function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.168 Parameter FlsSetModeApi

Compile switch to enable and disable the Fls_SetMode function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.169 Parameter FlsTotalSize

The total amount of flash memory in bytes (see also FLS118). FLS170: This parameter in conjunction with FLS_BASE_ADDRESS defines the upper boundary for read / write / erase and compare jobs.

Note: Not needed / supported by the driver.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.170 Parameter FlsUseInterrupts

Not supported by the driver.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

Configuration Parameters

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.171 Parameter FlsVersionInfoApi

Pre-processor switch to enable / disable the API to read out the modules version information.

true: Version info API enabled.

false: Version info API disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.172 Parameter FlsECCCheck

Vendor specific: Pre-processor switch to enable / disable the API to report data storage (ECC) errors to the flash driver.

This is the first ECC handling approach which modifies the program counter to skip the instruction causing the fault.

Please read the chapter Exception Handler in case of ECC error in IM for more information.

true : The ECC check by HardfaultHandler API is enabled.

false: The ECC check by HardfaultHandler API is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.173 Parameter FlsECCHandlingProtectionHook

Vendor specific: Pre-processor switch to enable / disable the API to report data storage (ECC) errors to the flash driver.

This is the second ECC handling approach which is compatible with Autosar Os.

Please read the chapter Exception Handler in case of ECC error in IM for more information.

true : The ECC check by AutosarOs API is enabled.

false: The ECC check by AutosarOs API is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.174 Parameter FlsEraseVerificationEnabled

Pre-processor switch to enable / disable the erase blank check. After a flash block has been erased, the erase blank check compares the contents of the addressed memory area against the value of an erased flash cell to check that the block has been completely erased.

true: Memory region is checked to be erased.

false: Memory region is not checked to be erased.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.175 Parameter FlsWriteVerificationEnabled

Pre-processor switch to enable / disable the write verify check. After writing a flash block, the write verify check compares the contents of the reprogrammed memory area against the contents of the provided application buffer to check that the block has

Configuration Parameters

been completely reprogrammed.

true: Written data is compared directly after write.

false: Written data is not compared directly after write.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.176 Parameter FlsMaxEraseBlankCheck

Vendor specific: The maximum number of bytes to blank check in one cycle of the flash driver's job processing function. Affects only the flash blocks that have enabled asynchronous execution of the erase job (FlsSectorEraseAsynch=true).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	256
max	65536
min	8

4.1.177 Parameter FlsTimeoutSupervisionEnabled

Compile switch to enable timeout supervision.

true: timeout supervision for read/erase/write/compare jobs enabled.

false: timeout supervision for read/erase/write/compare jobs disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.178 Parameter FlsTimeoutMethod

Vendor specific: Counter type used in timeout detection for FLS service request.

Based on selected counter type the timeout value will be interpreted as follows:

OSIF_COUNTER_DUMMY - Counts the number of iterations of the waiting loop. The actual timeout depends on many factors: operation type, compiler optimizations, interrupts or other tasks in the system, etc.

OSIF_COUNTER_SYSTEM - Microseconds.

OSIF_COUNTER_CUSTOM - Defined by user implementation of timing services

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM']

4.1.179 Parameter FlsAsyncWriteTimeout

Vendor specific: Fls Async Write Timeout is the timeout value for write operation in asynchronous mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2147483647
max	2147483647
min	0

4.1.180 Parameter FlsAsyncEraseTimeout

Vendor specific: Fls Async Erase Timeout is the timeout value for erase operation in asynchronous mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2147483647
max	2147483647
min	0

4.1.181 Parameter FlsSyncWriteTimeout

Vendor specific: Fls Sync Write Timeout is the timeout value for write operation in synchronous mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	2147483647
max	2147483647
min	0

4.1.182 Parameter FlsSyncEraseTimeout

Vendor specific: Fls Sync Erase Timeout is the timeout value for erase operation in synchronous mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	2147483647
max	2147483647
min	0

4.1.183 Parameter FlsAbortTimeout

Vendor specific: Fls Abort Timeout is the timeout value for aborting an ongoing operation.

The timeout is used also in Fls_Cancel API and Abort Erase suspend, if enabled and if the flash hardware channel does not support an immediate abort feature.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	32767
max	2147483647
min	0

4.1.184 Parameter FlsQspiIpTimeoutOsifCounterType

Vendor specific: Counter type used in timeout detection for QSPI operations.

Based on selected counter type the timeout value will be interpreted as follows:

OSIF_COUNTER_DUMMY - Counts the number of iterations of the waiting loop. The actual timeout depends on many factors: operation type, compiler optimizations, interrupts or other tasks in the system, etc.

OSIF_COUNTER_SYSTEM - Microseconds.

OSIF_COUNTER_CUSTOM - Defined by user implementation of timing services

Note: Qspi always uses timeout

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM']

4.1.185 Parameter FlsQspiSyncReadTimeout

Vendor specific: Fls Qspi Sync Read Timeout is the timeout value for read operation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2147483647
max	2147483647
min	0

4.1.186 Parameter FlsQspiAsyncWriteTimeout

Vendor specific: Fls Qspi Async Write Timeout is the timeout value for QSPI write operation in asynchronous mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	2147483647
max	2147483647
min	0

4.1.187 Parameter FlsQspiAsyncEraseTimeout

Vendor specific: Fls Qspi Async Erase Timeout is the timeout value for QSPI erase operation in asynchronous mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	2147483647
max	2147483647
min	0

4.1.188 Parameter FlsQspiSyncWriteTimeout

Vendor specific: Fls Qspi Sync Write Timeout is the timeout value for QSPI write operation in synchronous mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2147483647
max	2147483647
min	0

4.1.189 Parameter FlsQspiSyncEraseTimeout

Vendor specific: Fls Qspi Sync Erase Timeout is the timeout value for QSPI erase operation in synchronous mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2147483647
max	2147483647
min	0

4.1.190 Parameter FlsQspiCommandCompleteTimeout

Vendor specific: Fls Qspi Command Complete Timeout is the timeout value for waiting for a QSPI command to be completed.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	2147483647
min	0

4.1.191 Parameter FlsQspiResetTimeout

Vendor specific: Fls Qspi Reset Timeout is the timeout for waiting for the external device to become available after a software reset.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	2147483647
min	0

4.1.192 Parameter FlsQspiFlashInitTimeout

Vendor specific: Fls Flash Init Timeout is the timeout for completing the initialization operation sequence for the external flash at startup.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	2147483647
min	0

4.1.193 Parameter FlsQspiSoftwareResetDelay

Vendor specific: Fls Qspi Reset Delay is the waiting time after changing the value of the QSPI software reset bits in MCR register.

Note: The default value is calculated in the number of CPU cycles for the worst case scenario (with maximum possible CPU frequency and minimum possible flash clock frequency).

See the note of MCR register of QSPI chapter in Reference manual for more information.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	2147483647
min	0

4.1.194 Parameter FlsQspiTxBufferResetDelay

Vendor specific: Fls Qspi TX Buffer Reset Delay is the waiting time after changing the value of the QSPI TX FIFO/buffer reset bits in MCR register.

Note: The default value is calculated in the number of CPU cycles for the worst case scenario (with maximum possible CPU frequency and minimum possible flash clock frequency).

See the note of MCR register of QSPI chapter in Reference manual for more information.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	2147483647
min	0

4.1.195 Parameter FlsQspiWriteEnableRetries

Vendor specific: Number of attempts when sending the Write Enable command to the external flash.

The driver will read back the status register after each attempt and check the Busy bit.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	3
max	100
min	0

4.1.196 Reference FlsEcucPartitionRef

Maps the Flash driver to zero or one ECUC partition to make the driver API available in this partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.197 Container FlsPublishedInformation

Additional published parameters not covered by CommonPublishedInformation container.

Note that these parameters do not have any configuration class setting, since they are published information.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.198 Parameter FlsAcLocationErase

Position in RAM, to which the erase flash access code has to be loaded.

Only relevant if the erase flash access code is not position independent. If this information is not provided it is assumed that the erase flash access code is position independent and that therefore the RAM position can be freely configured.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	4294967295
min	0

4.1.199 Parameter FlsAcLocationWrite

Vendor specific: Position in RAM, to which the write flash access code has to be loaded.

Only relevant if the write flash access code is not position independent. If this information is not provided it is assumed that the write flash access code is position independent and that therefore the RAM position can be freely configured.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION

Property	Value
defaultValue	0
max	4294967295
min	0

4.1.200 Parameter FlsAcSizeErase

Number of bytes in RAM needed for the erase flash access code.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	4294967295
min	0

4.1.201 Parameter FlsAcSizeWrite

Number of bytes in RAM needed for the write flash access code.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	4294967295
min	0

4.1.202 Parameter FlsEraseTime

Maximum time to erase one complete flash sector [sec].

Configuration Parameters

Note: This value can be found on DS as the maximum erase time occurs after the specified number of program/erase cycles .

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	5.0
max	5.0
min	0.0

4.1.203 Parameter FlsErasedValue

The contents of an erased flash memory cell.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4294967295
max	4294967295
min	0

4.1.204 Parameter FlsECCValue

Vendor specific: The contents of an ECC flash memory line.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	1427461397
max	4294967295
min	0

4.1.205 Parameter FlsExpectedHwId

Unique identifier of the hardware device that is expected by this driver (the device for which this driver has been implemented).

Only relevant for external flash drivers.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0

4.1.206 Parameter FlsSpecifiedEraseCycles

Number of erase cycles specified for the flash device (usually given in the device data sheet).

FLS198: If the number of specified erase cycles depends on the operating environment (temperature, voltage, ...) during reprogramming of the flash device, the minimum number for which a data retention of at least 15 years over the temperature range from -40C .. +125C can be guaranteed shall be given.

Note: If there are different numbers of specified erase cycles for different flash sectors of the device this parameter has to be extended to a parameter list (similar to the sector list above).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	100000
max	4294967295
min	0

4.1.207 Parameter FlsWriteTime

Maximum time to program one complete flash page [sec].

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	5.0E-4
max	5.0E-4
min	0.0

4.1.208 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.209 Parameter ArReleaseMajorVersion

Vendor specific: Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.210 Parameter ArReleaseMinorVersion

Vendor specific: Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.211 Parameter ArReleaseRevisionVersion

Vendor specific: Patch version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Configuration Parameters

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.212 Parameter ModuleId

Vendor specific: Module ID of this module.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	92
max	92
min	92

4.1.213 Parameter SwMajorVersion

Vendor specific: Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.214 Parameter SwMinorVersion

Vendor specific: Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.215 Parameter SwPatchVersion

Vendor specific: Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.216 Parameter VendorApiInfix

Vendor specific: In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_<VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

Configuration Parameters

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.1.217 Parameter VendorId

Vendor specific: Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.2 FTFC_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Ftfc_Ip](#)
 - Container [FlsConfigSet](#)
 - * Parameter [FlsStartFlashAccessNotif](#)
 - * Parameter [FlsFinishedFlashAccessNotif](#)
 - Container [FlsGeneral](#)

- * Parameter [IpDevErrorDetect](#)
- * Parameter [FlsECCCheck](#)
- * Parameter [FlsECCHandlingProtectionHook](#)
- * Parameter [FlsEraseVerificationEnabled](#)
- * Parameter [FlsWriteVerificationEnabled](#)
- * Parameter [FlsTimeoutSupervisionEnabled](#)
- * Parameter [FlsTimeoutMethod](#)
- * Parameter [FlsAsyncWriteTimeout](#)
- * Parameter [FlsAsyncEraseTimeout](#)
- * Parameter [FlsSyncWriteTimeout](#)
- * Parameter [FlsSyncEraseTimeout](#)
- * Parameter [FlsAbortTimeout](#)
- Container [AutosarExt](#)
 - * Parameter [FlsEnableUserModeSupport](#)
 - * Parameter [FlsSynchronizeCache](#)
 - * Parameter [FlsInvalidPrefetchBufFromRam](#)
- Container [FlsPublishedInformation](#)
 - * Parameter [FlsErasedValue](#)
 - * Parameter [FlsECCValue](#)

4.2.1 Module [Ftfc_Ip](#)

Ftfc_Ip config set

Included containers:

- [FlsConfigSet](#)
- [FlsGeneral](#)
- [AutosarExt](#)
- [FlsPublishedInformation](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.2.2 Parameter [IpDevErrorDetect](#)

Vendor specific:

Configuration Parameters

true: Development error checking at IP level is enabled.

false: Development error checking at IP level is disabled.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.3 QSPI_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Qspi_Ip](#)
 - Container [FlsConfigSet](#)
 - * Parameter [FlsQspiInitCallout](#)
 - * Parameter [FlsQspiResetCallout](#)
 - * Parameter [FlsQspiErrorCheckCallout](#)
 - * Parameter [FlsQspiEccCheckCallout](#)
 - * Container [FlsExternalDriver](#)
 - Container [FlsMem](#)
 - Container [MemCfg](#)
 - Container [FlsController](#)
 - Container [ControllerCfg](#)
 - Container [FlsGeneral](#)
 - * Parameter [IpDevErrorDetect](#)
 - * Parameter [FlsQspiIpTimeoutOsifCounterType](#)
 - * Parameter [FlsQspiSyncReadTimeout](#)
 - * Parameter [FlsQspiCommandCompleteTimeout](#)
 - * Parameter [FlsQspiResetTimeout](#)
 - * Parameter [FlsQspiFlashInitTimeout](#)
 - * Parameter [FlsQspiSoftwareResetDelay](#)

- * Parameter [FlsQspiTxBufferResetDelay](#)
- * Parameter [FlsQspiWriteEnableRetries](#)
- Container [AutosarExt](#)
 - * Parameter [FlsEnableUserModeSupport](#)

4.3.1 Module Qspi_Ip

Qspi_Ip config set

Included containers:

- [FlsConfigSet](#)
- [FlsGeneral](#)
- [AutosarExt](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.3.2 Parameter IpDevErrorDetect

Vendor specific:

true: Development error checking at IP level is enabled.

false: Development error checking at IP level is disabled.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

FLS Driver	145
QSPI IPV Driver	169
FTFC IP Driver	201

Chapter 6

Module Documentation

6.1 FLS Driver

6.1.1 Detailed Description

Data Structures

- struct [Fls_Flash_InternalSectorInfoType](#)
FLASH physical sector description. [More...](#)
- struct [Fls_QspiCfgConfigType](#)
Fls Qspi CfgConfig Type. [More...](#)
- struct [Fls_ConfigType](#)
Fls Config Type. [More...](#)

Macros

- `#define FLS_DEVICE_INSTANCE_INVALID`
- `#define FLS_API_VENDOR_ID`
Version Check parameters.
- `#define FLS_MODULE_ID`
AUTOSAR module identification.
- `#define FLS_INSTANCE_ID`
AUTOSAR module instance identification.
- `#define FLS_E_PARAM_CONFIG`
Development error codes (passed to DET).
- `#define FLS_E_PARAM_ADDRESS`
API service called with wrong address parameter.
- `#define FLS_E_PARAM_LENGTH`
API service called with wrong length parameter.
- `#define FLS_E_PARAM_DATA`
API service called with wrong data parameter.
- `#define FLS_E_UNINIT`
API service called without module initialization.

- `#define FLS_E_BUSY`
API service called while driver still busy.
- `#define FLS_E_PARAM_POINTER`
API service called with NULL pointer.
- `#define FLS_E_VERIFY_ERASE_FAILED`
Runtime error codes (passed to DET).
- `#define FLS_E_VERIFY_WRITE_FAILED`
Write verification (compare) failed.
- `#define FLS_E_TIMEOUT`
Timeout exceeded.
- `#define FLS_E_ERASE_FAILED`
Transient Faults codes (passed to DET).
- `#define FLS_E_WRITE_FAILED`
Flash write failed (HW)
- `#define FLS_E_READ_FAILED`
Flash read failed (HW)
- `#define FLS_E_COMPARE_FAILED`
Flash compare failed (HW)
- `#define FLS_INIT_ID`
All service IDs (passed to DET).
- `#define FLS_ERASE_ID`
service ID of function: Fls_Erase. (passed to DET)
- `#define FLS_WRITE_ID`
service ID of function: Fls_Write. (passed to DET)
- `#define FLS_CANCEL_ID`
service ID of function: Fls_Cancel. (passed to DET)
- `#define FLS_GETJOBRESULT_ID`
service ID of function: Fls_GetJobResult. (passed to DET)
- `#define FLS_MAINFUNCTION_ID`
service ID of function: Fls_MainFunction. (passed to DET)
- `#define FLS_READ_ID`
service ID of function: Fls_Read. (passed to DET)
- `#define FLS_COMPARE_ID`
service ID of function: Fls_Compare. (passed to DET)
- `#define FLS_SETMODE_ID`
service ID of function: Fls_SetMode. (passed to DET)
- `#define FLS_GETVERSIONINFO_ID`
service ID of function: Fls_GetVersionInfo. (passed to DET)
- `#define FLS_BLANK_CHECK_ID`
service ID of function: Fls_BlankCheck. (passed to DET)
- `#define FLS_SECTOR_ERASE_ASYNC`

- *All sector flags.*
- `#define FLS_PAGE_WRITE_ASYNC`
fls page write async
- `#define FLS_START_SEC_CODE`
Start of Fls section CODE.
- `#define FLS_STOP_SEC_CODE`
Stop of Fls section CODE.
- `#define FLS_IPW_CFG_INVALID`

Types Reference

- `typedef uint32 Fls_SectorIndexType`
Logical sector index.
- `typedef uint32 Fls_AddressType`
Fls Address Type.
- `typedef uint32 Fls_LengthType`
Fls Length Type.
- `typedef uint32 Fls_SectorCountType`
Fls Sector Count Type.
- `typedef uint8 Fls_BlockNumberOfSectorType`
Fls BLock Count Type.
- `typedef Ftfc_ConfigType Fls_InternalConfigType`
Fls Internal Flash Type.
- `typedef void(* Fls_JobEndNotificationPtrType) (void)`
Fls Job End Notification Pointer Type.
- `typedef void(* Fls_JobErrorNotificationPtrType) (void)`
Fls Job Error Notification Pointer Type.
- `typedef void(* Fls_ACCallbackPtrType) (void)`
Pointer type of Fls_AC_Callback function.
- `typedef void(* Fls_AcErasePtrType) (void(*CallBack) (void))`
Define pointer type of erase access code function.
- `typedef void(* Fls_AcWritePtrType) (void(*CallBack) (void))`
Define pointer type of write access code function.

Enum Reference

- `enum Fls_HwChType`
Flash sector channel type.
- `enum Fls_JobType`
Type of job currently executed by Fls_MainFunction.
- `enum Fls_LLDReturnType`
Result of low-level flash operation.
- `enum Fls_LLDJobType`

Type of job currently executed by Fls_LLDMainFunction.

- enum [Fls_CrcDataSizeType](#)

Size of data to be processed by CRC.

Function Reference

- void [Fls_Init](#) (const [Fls_ConfigType](#) *ConfigPtr)

The function initializes Fls module.

- Std_ReturnType [Fls_Write](#) ([Fls_AddressType](#) TargetAddress, const uint8 *SourceAddressPtr, [Fls_LengthType](#) Length)

Write one or more complete flash pages to the flash device.

- Std_ReturnType [Fls_Erase](#) ([Fls_AddressType](#) TargetAddress, [Fls_LengthType](#) Length)

Erase one or more complete flash sectors.

- Std_ReturnType [Fls_Read](#) ([Fls_AddressType](#) SourceAddress, uint8 *TargetAddressPtr, [Fls_LengthType](#) Length)

Reads from flash memory.

Variables

- [Fls_AddressType](#) [Fls_u32JobAddrIt](#)

Logical address of data block currently processed by Fls_MainFunction.

- volatile [Fls_SectorIndexType](#) [Fls_u32JobSectorIt](#)

Index of flash sector currently processed by a job.

- [Fls_SectorIndexType](#) [Fls_u32JobSectorEnd](#)

Index of last flash sector by current job.

- volatile MemIf_JobResultType [Fls_eLLDJobResult](#)

Result of last flash hardware job.

- [Fls_LLDJobType](#) [Fls_eLLDJob](#)

Type of current flash hardware job - used for asynchronous operating mode.

- const [Fls_ConfigType](#) * [Fls_pConfigPtr](#)

Pointer to current flash module configuration set.

6.1.2 Data Structure Documentation

6.1.2.1 struct [Fls_Flash_InternalSectorInfoType](#)

FLASH physical sector description.

Definition at line 341 of file [Fls_Types.h](#).

Data Fields

uint32	pSectorStartAddressPtr	FLASH physical sector start address.
uint32	u32SectorId	Corresponding number in sector location to calc cfgCRC.

6.1.2.2 struct Fls_QspiCfgConfigType

Fls Qspi CfgConfig Type.

Fls Qspi CfgConfig Type

Definition at line 351 of file Fls_Types.h.

Data Fields

- const uint8(* [u8SectFlashUnit](#))[]
External flash unit assigned to each sector. Size: u32SectorCount.
- const uint8 [u8FlashUnitsCount](#)
Number of serial flash instances.
- const [Qspi_Ip_MemoryConnectionType](#)(* [paFlashConnectionCfg](#))[]
Connection for each external memory device to available controllers. Size: u8FlashUnitsCount.
- const uint8(* [u8FlashConfig](#))[]
Configuration index used for each flash unit. Size: u8FlashUnitsCount.
- const boolean(* [paAHBReadCfg](#))[]
AHB direct reads configurations. Size: u8FlashUnitsCount.
- const uint8 [u8FlashConfigCount](#)
Number of serial flash configurations.
- const [Qspi_Ip_MemoryConfigType](#)(* [paFlashCfg](#))[]
External memory devices configurations. Size: u8FlashConfigCount.
- const uint8 [u8QspiUnitsCount](#)
Number of QSPI hardware instances.
- const uint8(* [u8QspiInstance](#))[]
Hardware instances for each QSPI unit. Size: u8QspiUnitsCount.
- const uint8(* [u8QspiConfig](#))[]
Configuration for each QSPI unit. Size: u8QspiUnitsCount.
- const uint8 [u8QspiConfigCount](#)
Number of QSPI configurations.
- const [Qspi_Ip_ControllerConfigType](#)(* [paQspiUnitCfg](#))[]
QSPI configurations. Size: u8QspiConfigCount.

Field Documentation

u8SectFlashUnit

```
const uint8(* u8SectFlashUnit)[]
```

External flash unit assigned to each sector. Size: u32SectorCount.

Definition at line 356 of file Fls_Types.h.

u8FlashUnitsCount

```
const uint8 u8FlashUnitsCount
```

Number of serial flash instances.

Definition at line 360 of file Fls_Types.h.

paFlashConnectionCfg

```
const Qspi_Ip_MemoryConnectionType(* paFlashConnectionCfg)[]
```

Connection for each external memory device to available controllers. Size: u8FlashUnitsCount.

Definition at line 364 of file Fls_Types.h.

u8FlashConfig

```
const uint8(* u8FlashConfig)[]
```

Configuration index used for each flash unit. Size: u8FlashUnitsCount.

Definition at line 368 of file Fls_Types.h.

paAHBReadCfg

```
const boolean(* paAHBReadCfg)[]
```

AHB direct reads configurations. Size: u8FlashUnitsCount.

Definition at line 372 of file Fls_Types.h.

u8FlashConfigCount

```
const uint8 u8FlashConfigCount
```

Number of serial flash configurations.

Definition at line 377 of file Fls_Types.h.

paFlashCfg

```
const Qspi_Ip_MemoryConfigType(* paFlashCfg)[]
```

External memory devices configurations. Size: u8FlashConfigCount.

Definition at line 381 of file Fls_Types.h.

u8QspiUnitsCount

```
const uint8 u8QspiUnitsCount
```

Number of QSPI hardware instances.

Definition at line 386 of file Fls_Types.h.

u8QspiInstance

```
const uint8(* u8QspiInstance)[]
```

Hardware instances for each QSPI unit. Size: u8QspiUnitsCount.

Definition at line 390 of file Fls_Types.h.

u8QspiConfig

```
const uint8(* u8QspiConfig)[]
```

Configuration for each QSPI unit. Size: u8QspiUnitsCount.

Definition at line 394 of file Fls_Types.h.

u8QspiConfigCount

```
const uint8 u8QspiConfigCount
```

Number of QSPI configurations.

Definition at line 398 of file Fls_Types.h.

paQspiUnitCfg

```
const Qspi_Ip_ControllerConfigType(* paQspiUnitCfg)[]
```

QSPI configurations. Size: u8QspiConfigCount.

Definition at line 402 of file Fls_Types.h.

6.1.2.3 struct Fls_ConfigType

Fls Config Type.

Fls module initialization data structure

Definition at line 410 of file Fls_Types.h.

Data Fields

- [Fls_AcErasePtrType acErasePtr](#)
pointer to erase access code function in RAM or ROM
- [Fls_AcWritePtrType acWritePtr](#)
pointer to write access code function in RAM or ROM
- [Fls_ACCallbackPtrType acCallBackPtr](#)

- pointer to ac callback function*
- [Fls_JobEndNotificationPtrType jobEndNotificationPtr](#)
pointer to job end notification function
- [Fls_JobErrorNotificationPtrType jobErrorNotificationPtr](#)
pointer to job error notification function
- [MemIf_ModeType eDefaultMode](#)
default FLS device mode after initialization (MEMIF_MODE_FAST, MEMIF_MODE_SLOW)
- [Fls_LengthType u32MaxReadFastMode](#)
max number of bytes to read in one cycle of Fls_MainFunction (fast mode)
- [Fls_LengthType u32MaxReadNormalMode](#)
max number of bytes to read in one cycle of Fls_MainFunction (normal mode)
- [Fls_LengthType u32MaxWriteFastMode](#)
max number of bytes to write in one cycle of Fls_MainFunction (fast mode)
- [Fls_LengthType u32MaxWriteNormalMode](#)
max number of bytes to write in one cycle of Fls_MainFunction (normal mode)
- [Fls_SectorCountType u32SectorCount](#)
number of configured logical sectors
- [const Fls_AddressType\(* paSectorEndAddr \)\[\]](#)
pointer to array containing last logical address of each configured sector
- [const Fls_LengthType\(* paSectorSize \)\[\]](#)
pointer to array containing sector size of each configured sector
- [const Fls_Flash_InternalSectorInfoType *const \(* pSectorList \)\[\]](#)
pointer to array containing physical sector ID of each configured sector
- [const uint8\(* paSectorFlags \)\[\]](#)
pointer to array containing flags set of each configured sector
- [const Fls_LengthType\(* paSectorPageSize \)\[\]](#)
pointer to array containing page size information of each configured sector
- [const Fls_HwChType\(* paHwCh \)\[\]](#)
Pointer to array containing the hardware channel(internal, external_qspi, external_emmc) of each configured sector.
- [const uint32\(* paSectorHwAddress \)\[\]](#)
Pointer to array containing the configured hardware start address of each external sector.
- [const Fls_QspiCfgConfigType * pFlsQspiCfgConfig](#)
Pointer to configuration structure of QSPI.
- [const Fls_InternalConfigType * pFlsInternalCfgConfig](#)
Pointer to configuration structure internal flash.

Field Documentation

acErasePtr

`Fls_AcErasePtrType` acErasePtr

pointer to erase access code function in RAM or ROM

Definition at line 416 of file Fls_Types.h.

acWritePtr

`Fls_AcWritePtrType` acWritePtr

pointer to write access code function in RAM or ROM

Definition at line 420 of file Fls_Types.h.

acCallbackPtr

`Fls_AcCallbackPtrType` acCallbackPtr

pointer to ac callback function

Definition at line 425 of file Fls_Types.h.

jobEndNotificationPtr

`Fls_JobEndNotificationPtrType` jobEndNotificationPtr

pointer to job end notification function

Definition at line 429 of file Fls_Types.h.

jobErrorNotificationPtr

`Fls_JobErrorNotificationPtrType` jobErrorNotificationPtr

pointer to job error notification function

Definition at line 433 of file Fls_Types.h.

eDefaultMode

`MemIf_ModeType` eDefaultMode

default FLS device mode after initialization (MEMIF_MODE_FAST, MEMIF_MODE_SLOW)

Definition at line 443 of file Fls_Types.h.

u32MaxReadFastMode

`Fls_LengthType` `u32MaxReadFastMode`

max number of bytes to read in one cycle of `Fls_MainFunction` (fast mode)

Definition at line 447 of file `Fls_Types.h`.

u32MaxReadNormalMode

`Fls_LengthType` `u32MaxReadNormalMode`

max number of bytes to read in one cycle of `Fls_MainFunction` (normal mode)

Definition at line 451 of file `Fls_Types.h`.

u32MaxWriteFastMode

`Fls_LengthType` `u32MaxWriteFastMode`

max number of bytes to write in one cycle of `Fls_MainFunction` (fast mode)

Definition at line 455 of file `Fls_Types.h`.

u32MaxWriteNormalMode

`Fls_LengthType` `u32MaxWriteNormalMode`

max number of bytes to write in one cycle of `Fls_MainFunction` (normal mode)

Definition at line 459 of file `Fls_Types.h`.

u32SectorCount

`Fls_SectorCountType` `u32SectorCount`

number of configured logical sectors

Definition at line 463 of file `Fls_Types.h`.

paSectorEndAddr

```
const Fls_AddressType (* paSectorEndAddr) [ ]
```

pointer to array containing last logical address of each configured sector

Definition at line 467 of file `Fls_Types.h`.

paSectorSize

```
const Fls_LengthType (* paSectorSize) [ ]
```

pointer to array containing sector size of each configured sector

Definition at line 471 of file Fls_Types.h.

pSectorList

```
const Fls_Flash_InternalSectorInfoType* const (* pSectorList) [ ]
```

pointer to array containing physical sector ID of each configured sector

Definition at line 475 of file Fls_Types.h.

paSectorFlags

```
const uint8 (* paSectorFlags) [ ]
```

pointer to array containing flags set of each configured sector

Definition at line 481 of file Fls_Types.h.

paSectorPageSize

```
const Fls_LengthType (* paSectorPageSize) [ ]
```

pointer to array containing page size information of each configured sector

Definition at line 485 of file Fls_Types.h.

paHwCh

```
const Fls_HwChType (* paHwCh) [ ]
```

Pointer to array containing the hardware channel(internal, external_qspi, external_emmc) of each configured sector.

Definition at line 489 of file Fls_Types.h.

paSectorHwAddress

```
const uint32 (* paSectorHwAddress) [ ]
```

Pointer to array containing the configured hardware start address of each external sector.

Definition at line 493 of file Fls_Types.h.

pFlsQspiCfgConfig

```
const Fls_QspiCfgConfigType* pFlsQspiCfgConfig
```

Pointer to configuration structure of QSPI.

Definition at line 496 of file Fls_Types.h.

pFlsInternalCfgConfig

```
const Fls_InternalConfigType* pFlsInternalCfgConfig
```

Pointer to configuration structure internal flash.

Definition at line 500 of file Fls_Types.h.

6.1.3 Macro Definition Documentation

6.1.3.1 FLS_DEVICE_INSTANCE_INVALID

```
#define FLS_DEVICE_INSTANCE_INVALID
```

Invalid device instance

Definition at line 138 of file Fls.h.

6.1.3.2 FLS_API_VENDOR_ID

```
#define FLS_API_VENDOR_ID
```

Version Check parameters.

Definition at line 56 of file Fls_Api.h.

6.1.3.3 FLS_MODULE_ID

```
#define FLS_MODULE_ID
```

AUTOSAR module identification.

Definition at line 102 of file Fls_Api.h.

6.1.3.4 FLS_INSTANCE_ID

```
#define FLS_INSTANCE_ID
```

AUTOSAR module instance identification.

Definition at line 106 of file Fls_Api.h.

6.1.3.5 FLS_E_PARAM_CONFIG

```
#define FLS_E_PARAM_CONFIG
```

Development error codes (passed to DET).

API service called with wrong config parameter

Definition at line 114 of file Fls_Api.h.

6.1.3.6 FLS_E_PARAM_ADDRESS

```
#define FLS_E_PARAM_ADDRESS
```

API service called with wrong address parameter.

Definition at line 118 of file Fls_Api.h.

6.1.3.7 FLS_E_PARAM_LENGTH

```
#define FLS_E_PARAM_LENGTH
```

API service called with wrong length parameter.

Definition at line 122 of file Fls_Api.h.

6.1.3.8 FLS_E_PARAM_DATA

```
#define FLS_E_PARAM_DATA
```

API service called with wrong data parameter.

Definition at line 126 of file Fls_Api.h.

6.1.3.9 FLS_E_UNINIT

```
#define FLS_E_UNINIT
```

API service called without module initialization.

Definition at line 130 of file Fls_Api.h.

6.1.3.10 FLS_E_BUSY

```
#define FLS_E_BUSY
```

API service called while driver still busy.

Definition at line 134 of file Fls_Api.h.

6.1.3.11 FLS_E_PARAM_POINTER

```
#define FLS_E_PARAM_POINTER
```

API service called with NULL pointer.

Definition at line 138 of file Fls_Api.h.

6.1.3.12 FLS_E_VERIFY_ERASE_FAILED

```
#define FLS_E_VERIFY_ERASE_FAILED
```

Runtime error codes (passed to DET).

Erase verification (blank check) failed

Definition at line 146 of file Fls_Api.h.

6.1.3.13 FLS_E_VERIFY_WRITE_FAILED

```
#define FLS_E_VERIFY_WRITE_FAILED
```

Write verification (compare) failed.

Definition at line 150 of file Fls_Api.h.

6.1.3.14 FLS_E_TIMEOUT

```
#define FLS_E_TIMEOUT
```

Timeout exceeded.

Definition at line 154 of file Fls_Api.h.

6.1.3.15 FLS_E_ERASE_FAILED

```
#define FLS_E_ERASE_FAILED
```

Transient Faults codes (passed to DET).

Flash erase failed (HW)

Definition at line 162 of file Fls_Api.h.

6.1.3.16 FLS_E_WRITE_FAILED

```
#define FLS_E_WRITE_FAILED
```

Flash write failed (HW)

Definition at line 166 of file Fls_Api.h.

6.1.3.17 FLS_E_READ_FAILED

```
#define FLS_E_READ_FAILED
```

Flash read failed (HW)

Definition at line 170 of file Fls_Api.h.

6.1.3.18 FLS_E_COMPARE_FAILED

```
#define FLS_E_COMPARE_FAILED
```

Flash compare failed (HW)

Definition at line 174 of file Fls_Api.h.

6.1.3.19 FLS_INIT_ID

```
#define FLS_INIT_ID
```

All service IDs (passed to DET).

service ID of function: Fls_Init. (passed to DET)

Definition at line 186 of file Fls_Api.h.

6.1.3.20 FLS_ERASE_ID

```
#define FLS_ERASE_ID
```

service ID of function: Fls_Erase. (passed to DET)

Definition at line 190 of file Fls_Api.h.

6.1.3.21 FLS_WRITE_ID

```
#define FLS_WRITE_ID
```

service ID of function: Fls_Write. (passed to DET)

Definition at line 194 of file Fls_Api.h.

6.1.3.22 FLS_CANCEL_ID

```
#define FLS_CANCEL_ID
```

service ID of function: Fls_Cancel. (passed to DET)

Definition at line 198 of file Fls_Api.h.

6.1.3.23 FLS_GETJOBRESULT_ID

```
#define FLS_GETJOBRESULT_ID
```

service ID of function: Fls_GetJobResult. (passed to DET)

Definition at line 202 of file Fls_Api.h.

6.1.3.24 FLS_MAINFUNCTION_ID

```
#define FLS_MAINFUNCTION_ID
```

service ID of function: Fls_MainFunction. (passed to DET)

Definition at line 206 of file Fls_Api.h.

6.1.3.25 FLS_READ_ID

```
#define FLS_READ_ID
```

service ID of function: Fls_Read. (passed to DET)

Definition at line 210 of file Fls_Api.h.

6.1.3.26 FLS_COMPARE_ID

```
#define FLS_COMPARE_ID
```

service ID of function: Fls_Compare. (passed to DET)

Definition at line 214 of file Fls_Api.h.

6.1.3.27 FLS_SETMODE_ID

```
#define FLS_SETMODE_ID
```

service ID of function: Fls_SetMode. (passed to DET)

Definition at line 218 of file Fls_Api.h.

6.1.3.28 FLS_GETVERSIONINFO_ID

```
#define FLS_GETVERSIONINFO_ID
```

service ID of function: Fls_GetVersionInfo. (passed to DET)

Definition at line 222 of file Fls_Api.h.

6.1.3.29 FLS_BLANK_CHECK_ID

```
#define FLS_BLANK_CHECK_ID
```

service ID of function: Fls_BlankCheck. (passed to DET)

Definition at line 226 of file Fls_Api.h.

6.1.3.30 FLS_SECTOR_ERASE_ASYNC

```
#define FLS_SECTOR_ERASE_ASYNC
```

All sector flags.

fls sector erase async

Definition at line 235 of file Fls_Api.h.

6.1.3.31 FLS_PAGE_WRITE_ASYNC

```
#define FLS_PAGE_WRITE_ASYNC
```

fls page write async

Definition at line 239 of file Fls_Api.h.

6.1.3.32 FLS_START_SEC_CODE

```
#define FLS_START_SEC_CODE
```

Start of Fls section CODE.

Definition at line 251 of file Fls_Api.h.

6.1.3.33 FLS_STOP_SEC_CODE

```
#define FLS_STOP_SEC_CODE
```

Stop of Fls section CODE.

Definition at line 531 of file Fls_Api.h.

6.1.3.34 FLS_IPW_CFG_INVALID

```
#define FLS_IPW_CFG_INVALID
```

Invalid configuration, specifies unused device

Definition at line 107 of file Fls_IPW.h.

6.1.4 Types Reference

6.1.4.1 Fls_SectorIndexType

```
typedef uint32 Fls_SectorIndexType
```

Logical sector index.

Definition at line 252 of file Fls_Types.h.

6.1.4.2 Fls_AddressType

```
typedef uint32 Fls_AddressType
```

Fls Address Type.

Address offset from the configured flash base address to access a certain flash memory area.

Definition at line 268 of file Fls_Types.h.

6.1.4.3 Fls_LengthType

```
typedef uint32 Fls_LengthType
```

Fls Length Type.

Number of bytes to read,write,erase,compare

Definition at line 274 of file Fls_Types.h.

6.1.4.4 Fls_SectorCountType

```
typedef uint32 Fls_SectorCountType
```

Fls Sector Count Type.

Number of configured sectors

Definition at line 280 of file Fls_Types.h.

6.1.4.5 Fls_BlockNumberOfSectorType

```
typedef uint8 Fls_BlockNumberOfSectorType
```

Fls BLock Count Type.

Block number of sectors type

Definition at line 286 of file Fls_Types.h.

6.1.4.6 Fls_InternalConfigType

```
typedef Ftfc_ConfigType Fls_InternalConfigType
```

Fls Internal Flash Type.

Configuration structure of internal flash.

Definition at line 292 of file Fls_Types.h.

6.1.4.7 Fls__JobEndNotificationPtrType

```
typedef void(* Fls__JobEndNotificationPtrType) (void)
```

Fls Job End Notification Pointer Type.

Pointer type of Fls__JobEndNotification function

Definition at line 299 of file Fls__Types.h.

6.1.4.8 Fls__JobErrorNotificationPtrType

```
typedef void(* Fls__JobErrorNotificationPtrType) (void)
```

Fls Job Error Notification Pointer Type.

Pointer type of Fls__JobErrorNotification function

Definition at line 305 of file Fls__Types.h.

6.1.4.9 Fls__ACCallbackPtrType

```
typedef void(* Fls__ACCallbackPtrType) (void)
```

Pointer type of Fls__AC__Callback function.

Definition at line 311 of file Fls__Types.h.

6.1.4.10 Fls__AcErasePtrType

```
typedef void(* Fls__AcErasePtrType) (void(*CallBack)(void))
```

Define pointer type of erase access code function.

Definition at line 317 of file Fls__Types.h.

6.1.4.11 Fls__AcWritePtrType

```
typedef void(* Fls__AcWritePtrType) (void(*CallBack)(void))
```

Define pointer type of write access code function.

Definition at line 322 of file Fls__Types.h.

6.1.5 Enum Reference

6.1.5.1 Fls_HwChType

enum [Fls_HwChType](#)

Flash sector channel type.

Definition at line 158 of file Fls_Types.h.

6.1.5.2 Fls_JobType

enum [Fls_JobType](#)

Type of job currently executed by Fls_MainFunction.

Enumerator

FLS_JOB_ERASE	erase one or more complete flash sectors
FLS_JOB_WRITE	write one or more complete flash pages
FLS_JOB_READ	read one or more bytes from flash memory
FLS_JOB_COMPARE	compare data buffer with content of flash memory
FLS_JOB_BLANK_CHECK	check content of erased flash memory area

Definition at line 167 of file Fls_Types.h.

6.1.5.3 Fls_LLDReturnType

enum [Fls_LLDReturnType](#)

Result of low-level flash operation.

Enumerator

FLASH_E_OK	operation succeeded
FLASH_E_FAILED	operation failed due to hardware error
FLASH_E_BLOCK_INCONSISTENT	data buffer doesn't match with content of flash memory
FLASH_E_PENDING	operation is pending
FLASH_E_PARTITION_ERR	FlexNVM partition ratio error.

Definition at line 194 of file Fls_Types.h.

6.1.5.4 Fls_LLDJobType

enum [Fls_LLDJobType](#)

Type of job currently executed by Fls_LLDMainFunction.

Enumerator

FLASH_JOB_NONE	no job executed by Fls_LLDMainFunction
FLASH_JOB_ERASE	erase one flash sector

Enumerator

FLASH_JOB_ERASE_TEMP	complete erase and start an interleaved erase flash sector
FLASH_JOB_WRITE	write one or more complete flash pages
FLASH_JOB_ERASE_BLANK_CHECK	erase blank check of flash sector

Definition at line 206 of file Fls_Types.h.

6.1.5.5 Fls_CrcDataSizeType

enum `Fls_CrcDataSizeType`

Size of data to be processed by CRC.

Enumerator

FLS_CRC_8_BITS	crc 8 bits
FLS_CRC_16_BITS	crc 16 bits

Definition at line 236 of file Fls_Types.h.

6.1.6 Function Reference

6.1.6.1 Fls_Init()

```
void Fls_Init (
    const Fls_ConfigType * ConfigPtr )
```

The function initializes Fls module.

The function sets the internal module variables according to given configuration set.

Parameters

in	<i>ConfigPtr</i>	Pointer to flash driver configuration set.
----	------------------	--

Precondition

ConfigPtr must not be `NULL_PTR` and the module status must not be `MEMIF_BUSY`.

6.1.6.2 Fls_Write()

```
Std_ReturnType Fls_Write (
    Fls_AddressType TargetAddress,
    const uint8 * SourceAddressPtr,
    Fls_LengthType Length )
```

Write one or more complete flash pages to the flash device.

Starts a write job asynchronously. The actual job is performed by `Fls_MainFunction`.

Module Documentation

Parameters

in	<i>TargetAddress</i>	Target address in flash memory. This address offset will be added to the flash memory base address.
in	<i>SourceAddressPtr</i>	Pointer to source data buffer.
in	<i>Length</i>	Number of bytes to write.

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Write command has been accepted.
<i>E_NOT_OK</i>	Write command has not been accepted.

Precondition

The module has to be initialized and not busy.

Postcondition

Fls_Write changes module status and some internal variables (Fls_u32JobSectorIt, Fls_u32JobAddrIt, Fls_u32JobAddrEnd, Fls_pJobDataSrcPtr, Fls_eJob, Fls_eJobResult).

6.1.6.3 Fls_Erase()

```
Std_ReturnType Fls_Erase (
    Fls_AddressType TargetAddress,
    Fls_LengthType Length )
```

Erase one or more complete flash sectors.

Starts an erase job asynchronously. The actual job is performed by the Fls_MainFunction.

Parameters

in	<i>TargetAddress</i>	Target address in flash memory.
in	<i>Length</i>	Number of bytes to erase.

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Erase command has been accepted.
<i>E_NOT_OK</i>	Erase command has not been accepted.

Precondition

The module has to be initialized and not busy.

Postcondition

`Fls_Erase` changes module status and some internal variables (`Fls_u32JobSectorIt`, `Fls_u32JobAddrIt`, `Fls_u32JobAddrEnd`, `Fls_Job`, `Fls_eJobResult`).

6.1.6.4 Fls_Read()

```
Std_ReturnType Fls_Read (
    Fls_AddressType SourceAddress,
    uint8 * TargetAddressPtr,
    Fls_LengthType Length )
```

Reads from flash memory.

Starts a read job asynchronously. The actual job is performed by `Fls_MainFunction`.

Parameters

in	<i>SourceAddress</i>	Source address in flash memory. This address offset will be added to the flash memory base address.
in	<i>Length</i>	Number of bytes to read.
out	<i>TargetAddressPtr</i>	Pointer to target data buffer.

Returns

`Std_ReturnType`

Return values

<i>E_OK</i>	Read command has been accepted
<i>E_NOT_OK</i>	Read command has not been accepted

Precondition

The module has to be initialized and not busy.

Postcondition

`Fls_Read` changes module status and some internal variables (`Fls_u32JobSectorIt`, `Fls_u32JobAddrIt`, `Fls_u32JobAddrEnd`, `Fls_pJobDataDestPtr`, `Fls_eJob`, `Fls_eJobResult`).

6.1.7 Variable Documentation**6.1.7.1 Fls_u32JobAddrIt**

```
Fls_AddressType Fls_u32JobAddrIt [extern]
```

Logical address of data block currently processed by `Fls_MainFunction`.

6.1.7.2 Fls_u32JobSectorIt

```
volatile Fls_SectorIndexType Fls_u32JobSectorIt [extern]
```


Index of flash sector currently processed by a job.

Used by all types of job

6.1.7.3 Fls_u32JobSectorEnd

```
Fls_SectorIndexType Fls_u32JobSectorEnd [extern]
```

Index of last flash sector by current job.

Used to check status of all external flash chips before start jobs or is the last sector in Erase job

6.1.7.4 Fls_eLLDJobResult

```
volatile MemIf_JobResultType Fls_eLLDJobResult [extern]
```

Result of last flash hardware job.

6.1.7.5 Fls_eLLDJob

```
Fls_LLDJobType Fls_eLLDJob [extern]
```

Type of current flash hardware job - used for asynchronous operating mode.

6.1.7.6 Fls_pConfigPtr

```
const Fls_ConfigType* Fls_pConfigPtr [extern]
```

Pointer to current flash module configuration set.

6.2 QSPI IPV Driver

6.2.1 Detailed Description

Data Structures

- struct [Qspi_Ip_StateType](#)
Driver configuration structure. [More...](#)
- struct [Qspi_Ip_ControllerAhbConfigType](#)
AHB configuration structure. [More...](#)
- struct [Qspi_Ip_ControllerConfigType](#)
Driver configuration structure. [More...](#)
- struct [Qspi_Ip_StatusConfigType](#)
Status register configuration structure. [More...](#)
- struct [Qspi_Ip_EraseVarConfigType](#)
Describes one type of erase. [More...](#)
- struct [Qspi_Ip_EraseConfigType](#)
Erase capabilities configuration structure. [More...](#)
- struct [Qspi_Ip_ReadIdConfigType](#)
Read Id capabilities configuration structure. [More...](#)
- struct [Qspi_Ip_SuspendConfigType](#)
Suspend capabilities configuration structure. [More...](#)
- struct [Qspi_Ip_ResetConfigType](#)
Soft Reset capabilities configuration structure. [More...](#)
- struct [Qspi_Ip_LutConfigType](#)
List of LUT sequences. [More...](#)
- struct [Qspi_Ip_InitOperationType](#)
Initialization operation. [More...](#)
- struct [Qspi_Ip_InitConfigType](#)
Initialization sequence. [More...](#)
- struct [Qspi_Ip_MemoryConfigType](#)
Driver configuration structure. [More...](#)
- struct [Qspi_Ip_MemoryConnectionType](#)
Flash-controller connections configuration structure. [More...](#)

Macros

- `#define QSPI_IP_MAX_READ_SIZE`
- `#define QSPI_IP_MAX_WRITE_SIZE`
- `#define QSPI_IP_ERASE_TYPES`
- `#define QSPI_IP_AHB_BUFFERS`
Number of AHB buffers in the device.
- `#define QSPI_IP_LUT_INVALID`
- `#define QSPI_IP_LUT_SEQ_END`

- `#define QSPI_IP_PACK_LUT_REG(ops0, ops1)`

Types Reference

- `typedef uint16 Qspi_Ip_InstrOpType`
Operation in a LUT sequence.
- `typedef Qspi_Ip_StatusType(* Qspi_Ip_InitCalloutPtrType) (uint32 instance)`
Init callout pointer type.
- `typedef Qspi_Ip_StatusType(* Qspi_Ip_ResetCalloutPtrType) (uint32 instance)`
Reset callout pointer type.
- `typedef Qspi_Ip_StatusType(* Qspi_Ip_ErrorCheckCalloutPtrType) (uint32 instance)`
Error Check callout pointer type.
- `typedef Qspi_Ip_StatusType(* Qspi_Ip_EccCheckCalloutPtrType) (uint32 instance, uint32 startAddress, uint32 dataLength)`
Ecc Check callout pointer type.

Enum Reference

- `enum Qspi_Ip_StatusType`
qspi return codes
- `enum Qspi_Ip_ConnectionType`
flash connection to the QSPI module
- `enum Qspi_Ip_OpType`
flash operation type
- `enum Qspi_Ip_LutCommandsType`
Lut commands.
- `enum Qspi_Ip_LutPadsType`
Lut pad options.
- `enum Qspi_Ip_ReadModeType`
Read mode.
- `enum Qspi_Ip_DataRateType`
Clock phase used for sampling Rx data.
- `enum Qspi_Ip_SampleDelayType`
Delay used for sampling Rx data.
- `enum Qspi_Ip_SamplePhaseType`
Clock phase used for sampling Rx data.
- `enum Qspi_Ip_FlashDataAlignType`
Alignment of outgoing data with serial clock.
- `enum Qspi_Ip_ClockSourceType`
Source of QuadSPI AHB read interface, module and bus interface clock.
- `enum Qspi_Ip_ClockReferenceType`
Source of QuadSPI internal reference clock.
- `enum Qspi_Ip_LastCommandType`

Last command that was executed by the device flash.

- enum [Qspi_Ip_FlashMemoryType](#)

Parameter memory type.

Function Reference

- [Qspi_Ip_StatusType Qspi_Ip_Init](#) (uint32 instance, const [Qspi_Ip_MemoryConfigType](#) *pConfig, const [Qspi_Ip_MemoryConnectionType](#) *pConnect)

Initializes the serial flash memory driver.

- [Qspi_Ip_StatusType Qspi_Ip_Deinit](#) (uint32 instance)

De-initializes the serial flash memory driver.

- [Qspi_Ip_StatusType Qspi_Ip_EraseBlock](#) (uint32 instance, uint32 address, uint32 size)

Erase a sector in the serial flash.

- [Qspi_Ip_StatusType Qspi_Ip_EraseChip](#) (uint32 instance)

Erase the entire serial flash.

- [Qspi_Ip_StatusType Qspi_Ip_GetMemoryStatus](#) (uint32 instance)

Check the status of the flash device.

- [Qspi_Ip_StatusType Qspi_Ip_SetProtection](#) (uint32 instance, uint8 value)

Sets the protection bits to the requested value.

- [Qspi_Ip_StatusType Qspi_Ip_GetProtection](#) (uint32 instance, uint8 *value)

Returns the current value of the protection bits.

- [Qspi_Ip_StatusType Qspi_Ip_Reset](#) (uint32 instance)

Resets the flash device.

- [Qspi_Ip_StatusType Qspi_Ip_Enter0XX](#) (uint32 instance)

Enters 0-X-X (no command) mode. This mode assumes only reads are performed.

- [Qspi_Ip_StatusType Qspi_Ip_Exit0XX](#) (uint32 instance)

Exits 0-X-X (no command) mode. This allows operations other than reads to be performed.

- [Qspi_Ip_StatusType Qspi_Ip_ProgramSuspend](#) (uint32 instance)

Suspends a program operation.

- [Qspi_Ip_StatusType Qspi_Ip_ProgramResume](#) (uint32 instance)

Resumes a program operation.

- [Qspi_Ip_StatusType Qspi_Ip_EraseSuspend](#) (uint32 instance)

Suspends an erase operation.

- [Qspi_Ip_StatusType Qspi_Ip_EraseResume](#) (uint32 instance)

Resumes an erase operation.

- [Qspi_Ip_StatusType Qspi_Ip_Read](#) (uint32 instance, uint32 address, uint8 *data, uint32 size)

Read data from serial flash.

- [Qspi_Ip_StatusType Qspi_Ip_ReadId](#) (uint32 instance, uint8 *data)

Read manufacturer ID/device ID from serial flash.

- [Qspi_Ip_StatusType Qspi_Ip_ProgramVerify](#) (uint32 instance, uint32 address, const uint8 *data, uint32 size)

Verifies the correctness of the programmed data.

- [Qspi_Ip_StatusType Qspi_Ip_EraseVerify](#) (uint32 instance, uint32 address, uint32 size)

Checks whether or not an area in the serial flash is erased.

- [Qspi_Ip_StatusType Qspi_Ip_Program](#) (uint32 instance, uint32 address, const uint8 *data, uint32 size)

Writes data in serial flash.

- [Qspi_Ip_StatusType Qspi_Ip_RunCommand](#) (uint32 instance, uint16 lut, uint32 addr)

Launches a simple command for the serial flash.

- [Qspi_Ip_StatusType Qspi_Ip_RunReadCommand](#) (uint32 instance, uint16 lut, uint32 addr, uint8 *dataRead, const uint8 *dataCmp, uint32 size)

Launches a read command for the serial flash.

- [Qspi_Ip_StatusType Qspi_Ip_RunWriteCommand](#) (uint32 instance, uint16 lut, uint32 addr, const uint8 *data, uint32 size)

Launches a write command for the serial flash.

- [Qspi_Ip_StatusType Qspi_Ip_AhbReadEnable](#) (uint32 instance)

Sets up AHB reads to the serial flash.

- [Qspi_Ip_StatusType Qspi_Ip_ControllerGetStatus](#) (uint32 instance)

Check the status of the QSPI controller.

- [Qspi_Ip_StatusType Qspi_Ip_ControllerInit](#) (uint32 instance, const [Qspi_Ip_ControllerConfigType](#) *user←ConfigPtr)

Initializes the qspi driver.

- [Qspi_Ip_StatusType Qspi_Ip_ControllerDeinit](#) (uint32 instance)

De-initialize the qspi driver.

- [Qspi_Ip_StatusType Qspi_Ip_Abort](#) (uint32 instance)

Aborts any on-going transactions.

- [Qspi_Ip_StatusType Qspi_Ip_ReadSfdp](#) ([Qspi_Ip_MemoryConfigType](#) *pConfig, const [Qspi_Ip_MemoryConnectionType](#) *pConnect)

Initializes the serial flash memory configuration from SFDP table.

QuadSPI Driver

- void [Qspi_Ip_SetLut](#) (uint32 instance, uint8 LutRegister, [Qspi_Ip_InstrOpType](#) operation0, [Qspi_Ip_InstrOpType](#) operation1)

Configures LUT commands.

- void [Qspi_Ip_WriteLuts_Privileged](#) (uint32 Instance, uint8 StartLutRegister, const uint32 *Data, uint8 Size)

Configures LUT commands.

- void [Qspi_Ip_SetAhbSeqId_Privileged](#) (uint32 instance, uint8 seqID)

Sets sequence ID for AHB operations.

- uint32 [Qspi_Ip_GetBaseAdress](#) (uint32 instance, [Qspi_Ip_ConnectionType](#) connectionType)

Returns the physical base address of a flash device.

- [Qspi_Ip_StatusType Qspi_Ip_IpCommand](#) (uint32 instance, uint8 SeqId, uint32 addr)

Launches a simple IP command.

- [Qspi_Ip_StatusType Qspi_Ip_IpRead](#) (uint32 instance, uint8 SeqId, uint32 addr, uint8 *dataRead, const uint8 *dataCmp, uint32 size)

Launches an IP read command.

- [Qspi_Ip_StatusType Qspi_Ip_IpWrite](#) (uint32 instance, uint8 SeqId, uint32 addr, const uint8 *data, uint32 size)

Launches an IP write command.

- #define **FLS_START_SEC_CODE**
- #define **FLS_STOP_SEC_CODE**

6.2.2 Data Structure Documentation

6.2.2.1 struct Qspi_Ip_StateType

Driver configuration structure.

This structure is used to provide configuration parameters for the external flash driver at initialization time.

Definition at line 97 of file Qspi_Ip_Common.h.

Data Fields

const Qspi_Ip_MemoryConfigType *	configuration	Serial flash device configuration
const Qspi_Ip_MemoryConnectionType *	connection	Connection to a QSPI device
uint32	baseAddress	Base address of serial flash device
Qspi_Ip_LastCommandType	lastCommand	Last command sent to the flash device
uint16	activeReadLut	LUT number for currently active read mode
boolean	isInXIPMode	Status of XIP mode (TRUE = Enabled/ FALSE = Disabled)

6.2.2.2 struct Qspi_Ip_ControllerAhbConfigType

AHB configuration structure.

This structure is used to provide configuration parameters for AHB access to the external flash

Definition at line 296 of file Qspi_Ip_Types.h.

Data Fields

uint8	masters[4U]	List of AHB masters assigned to each buffer
uint16	sizes[4U]	List of buffer sizes
boolean	allMasters	Indicates that any master may access the last buffer

6.2.2.3 struct Qspi_Ip_ControllerConfigType

Driver configuration structure.

This structure is used to provide configuration parameters for the qspi driver at initialization time.

Definition at line 328 of file Qspi_Ip_Types.h.

Data Fields

Qspi_Ip_ClockSourceType	clockSrc	AHB read interface, module and bus interface clock
Qspi_Ip_ClockReferenceType	clockRef	Internal reference clock (async clock domain)
uint8	clockRefDiv	Divider value for internal reference clock
boolean	dqsInvertA	Inverted reference clock selection for DQS Flash A
boolean	dqsInvertB	Inverted reference clock selection for DQS Flash B
uint8	dqsDelayA	Fine delay chain configuration for Flash A
uint8	dqsDelayB	Fine delay chain configuration for Flash B
Qspi_Ip_DataRateType	dataRate	Single/double data rate
uint32	memSizeA1	Size of serial flash A1
uint32	memSizeA2	Size of serial flash A2
uint32	memSizeB1	Size of serial flash B1
uint32	memSizeB2	Size of serial flash B2
uint8	csHoldTime	CS hold time, expressed in serial clock cycles
uint8	csSetupTime	CS setup time, expressed in serial clock cycles
uint8	columnAddr	Width of the column address, 0 if not used
boolean	wordAddressable	True if serial flash is word addressable
Qspi_Ip_ReadModeType	readModeA	Read mode for incoming data from serial flash A
Qspi_Ip_ReadModeType	readModeB	Read mode for incoming data from serial flash B
Qspi_Ip_SampleDelayType	sampleDelay	Delay (in clock cycles) used for sampling Rx data
Qspi_Ip_SamplePhaseType	samplePhase	Clock phase used for sampling Rx data
boolean	dqsLatency	Enable DQS latency for reads (Hyperflash)
Qspi_Ip_FlashDataAlignType	dataAlign	Alignment of output data sent to serial flash
uint8	io2IdleValueA	(0 / 1) Logic level of IO[2] signal when not used on side A

Data Fields

uint8	io3IdleValueA	(0 / 1) Logic level of IO[3] signal when not used on side A
uint8	io2IdleValueB	(0 / 1) Logic level of IO[2] signal when not used on side B
uint8	io3IdleValueB	(0 / 1) Logic level of IO[3] signal when not used on side B
Qspi_Ip_ControllerAhbConfigType	ahbConfig	AHB buffers configuration

6.2.2.4 struct Qspi_Ip_StatusConfigType

Status register configuration structure.

This structure contains information about the status registers of the external flash

Definition at line 369 of file Qspi_Ip_Types.h.

Data Fields

uint16	statusRegInitReadLut	Command used to read the status register during initialization
uint16	statusRegReadLut	Command used to read the status register
uint16	statusRegWriteLut	Command used to write the status register
uint16	writeEnableSRLut	Write enable command used before writing to status register
uint16	writeEnableLut	Write enable command used before write or erase operations
uint8	regSize	Size in bytes of status register
uint8	busyOffset	Position of "busy" bit inside status register
uint8	busyValue	Value of "busy" bit which indicates that the device is busy; can be 0 or 1
uint8	writeEnableOffset	Position of "write enable" bit inside status register
uint8	blockProtectionOffset	Offset of block protection bits inside status register
uint8	blockProtectionWidth	Width of block protection bitfield
uint8	blockProtectionValue	Value of block protection bitfield, indicate the protected area

6.2.2.5 struct Qspi_Ip_EraseVarConfigType

Describes one type of erase.

This structure contains information about one type of erase supported by the external flash

Definition at line 391 of file Qspi_Ip_Types.h.

Data Fields

uint16	eraseLut	Lut index for erase command
uint8	size	Size of the erased area: 2^{size} ; e.g. 0x0C means 4 Kbytes

6.2.2.6 struct Qspi_Ip_EraseConfigType

Erase capabilities configuration structure.

This structure contains information about the erase capabilities of the external flash

Definition at line 403 of file Qspi_Ip_Types.h.

Data Fields

Qspi_Ip_EraseVarConfigType	eraseTypes[4U]	Erase types supported by the device
uint16	chipEraseLut	Lut index for chip erase command

6.2.2.7 struct Qspi_Ip_ReadIdConfigType

Read Id capabilities configuration structure.

This structure contains information about the read manufacturer/device ID command

Definition at line 415 of file Qspi_Ip_Types.h.

Data Fields

uint16	readIdLut	Read Id command
uint8	readIdSize	Size of data returned by Read Id command
uint8	readIdExpected[4U]	Device ID configured value (Memory density Memory type Manufacturer ID)

6.2.2.8 struct Qspi_Ip_SuspendConfigType

Suspend capabilities configuration structure.

This structure contains information about the Program / Erase Suspend capabilities of the external flash

Definition at line 428 of file Qspi_Ip_Types.h.

Data Fields

uint16	eraseSuspendLut	Lut index for the erase suspend operation
uint16	eraseResumeLut	Lut index for the erase resume operation
uint16	programSuspendLut	Lut index for the program suspend operation

Data Fields

uint16	programResumeLut	Lut index for the program resume operation
--------	------------------	--

6.2.2.9 struct Qspi_Ip_ResetConfigType

Soft Reset capabilities configuration structure.

This structure contains information about the Soft Reset capabilities of the external flash

Definition at line 442 of file Qspi_Ip_Types.h.

Data Fields

uint16	resetCmdLut	First command in reset sequence
uint8	resetCmdCount	Number of commands in reset sequence

6.2.2.10 struct Qspi_Ip_LutConfigType

List of LUT sequences.

List of LUT sequences. Each sequence describes a command to the external flash. Sequences are separated by a 0 operation

Definition at line 470 of file Qspi_Ip_Types.h.

Data Fields

uint16	opCount	Number of operations in the LUT table
Qspi_Ip_InstrOpType *	lutOps	List of operations

6.2.2.11 struct Qspi_Ip_InitOperationType

Initialization operation.

This structure describes one initialization operation.

Definition at line 482 of file Qspi_Ip_Types.h.

Data Fields

Qspi_Ip_OpType	opType	Operation type
uint16	command1Lut	Index of first command sequence in Lut; for RMW type this is the read command
uint16	command2Lut	Index of second command sequence in Lut, only used for RMW type, this is the write command

Data Fields

uint16	weLut	Index of write enable sequence in Lut, only used for Write and RMW type
uint32	addr	Address, if used in command.
uint8	size	Size in bytes of configuration register
uint8	shift	Position of configuration field inside the register
uint8	width	Width in bits of configuration field.
uint32	value	Value to set in the field
const Qspi_Ip_ControllerConfigType *	ctrlCfgPtr	New controller configuration, valid only for QSPI_IP_OP_TYPE_QSPI_CFG type

6.2.2.12 struct Qspi_Ip_InitConfigType

Initialization sequence.

Describe sequence that will be performed only once during initialization to put the flash in the desired state for operation. This may include, for example, setting the QE bit, activating 4-byte addressing, activating XPI mode

Definition at line 503 of file Qspi_Ip_Types.h.

Data Fields

uint8	opCount	Number of operations
Qspi_Ip_InitOperationType *	operations	List of operations

6.2.2.13 struct Qspi_Ip_MemoryConfigType

Driver configuration structure.

This structure is used to provide configuration parameters for the external flash driver at initialization time.

Definition at line 525 of file Qspi_Ip_Types.h.

Data Fields

Qspi_Ip_FlashMemoryType	memType	Mmemory device type
uint32	memSize	Memory size (in bytes)
uint32	pageSize	Page size (in bytes)
uint16	readLut	Command used to read data from flash

Data Fields

uint16	writeLut	Command used to write data to flash
uint16	read0xxLut	0-x-x mode read command
uint16	read0xxLutAHB	0-x-x mode AHB read command
Qspi_Ip_ReadIdConfigType	readIdSettings	Erase settings of the external flash
Qspi_Ip_EraseConfigType	eraseSettings	Erase settings of the external flash
Qspi_Ip_StatusConfigType	statusConfig	Status register information
Qspi_Ip_SuspendConfigType	suspendSettings	Program / Erase Suspend settings
Qspi_Ip_ResetConfigType	resetSettings	Soft Reset settings, used at runtime
Qspi_Ip_ResetConfigType	initResetSettings	Soft Reset settings, used for first time reset
Qspi_Ip_InitConfigType	initConfiguration	Operations for initial flash configuration
Qspi_Ip_LutConfigType	lutSequences	List of LUT sequences describing flash commands
Qspi_Ip_InitCalloutPtrType	initCallout	Pointer to init callout
Qspi_Ip_ResetCalloutPtrType	resetCallout	Pointer to reset callout
Qspi_Ip_ErrorCheckCalloutPtrType	errorCheckCallout	Pointer to error check callout
Qspi_Ip_EccCheckCalloutPtrType	eccCheckCallout	Pointer to ecc check callout
const Qspi_Ip_ControllerConfigType *	ctrlAutoCfgPtr	Initial controller configuration, if needed

6.2.2.14 struct Qspi_Ip_MemoryConnectionType

Flash-controller connections configuration structure.

This structure specifies the connections of each flash device to QSPI controllers at initialization time.

Definition at line 556 of file [Qspi_Ip_Types.h](#).

Data Fields

uint32	qspiInstance	QSPI Instance where this device is connected
Qspi_Ip_ConnectionType	connectionType	Device connection to QSPI module
uint8	memAlignment	Memory alignment required by the external flash

6.2.3 Macro Definition Documentation

6.2.3.1 QSPI_IP_MAX_READ_SIZE

```
#define QSPI_IP_MAX_READ_SIZE
```

Maximum number of bytes then can be read in one operation

Definition at line 99 of file Qspi_Ip.h.

6.2.3.2 QSPI_IP_MAX_WRITE_SIZE

```
#define QSPI_IP_MAX_WRITE_SIZE
```

Maximum number of bytes then can be written in one operation

Definition at line 101 of file Qspi_Ip.h.

6.2.3.3 QSPI_IP_ERASE_TYPES

```
#define QSPI_IP_ERASE_TYPES
```

Number of erase types that can be supported by a flash device

Definition at line 118 of file Qspi_Ip_Types.h.

6.2.3.4 QSPI_IP_AHB_BUFFERS

```
#define QSPI_IP_AHB_BUFFERS
```

Number of AHB buffers in the device.

Definition at line 121 of file Qspi_Ip_Types.h.

6.2.3.5 QSPI_IP_LUT_INVALID

```
#define QSPI_IP_LUT_INVALID
```

Invalid index in virtual LUT, used for unsupported features

Definition at line 124 of file Qspi_Ip_Types.h.

6.2.3.6 QSPI_IP_LUT_SEQ_END

```
#define QSPI_IP_LUT_SEQ_END
```

End operation for a LUT sequence

Definition at line 126 of file Qspi_Ip_Types.h.

6.2.3.7 QSPI_IP_PACK_LUT_REG

```
#define QSPI_IP_PACK_LUT_REG(  
    ops0,  
    ops1 )
```

Pack the two operations into a LUT register (each operation is a pair of instruction-operand)

Definition at line 128 of file Qspi_Ip_Types.h.

6.2.4 Types Reference

6.2.4.1 Qspi_Ip_InstrOpType

```
typedef uint16 Qspi_Ip_InstrOpType
```

Operation in a LUT sequence.

This type describes one basic operation inside a LUT sequence. Each operation contains:

- instruction (6 bits)
- number of PADs (2 bits)
- operand (8 bits) Qspi_Ip_LutCommandsType and Qspi_Ip_LutPadsType types should be used to form operations

Definition at line 215 of file Qspi_Ip_Types.h.

6.2.4.2 Qspi_Ip_InitCalloutPtrType

```
typedef Qspi_Ip_StatusType(* Qspi_Ip_InitCalloutPtrType) (uint32 instance)
```

Init callout pointer type.

Definition at line 270 of file Qspi_Ip_Types.h.

6.2.4.3 Qspi_Ip_ResetCalloutPtrType

```
typedef Qspi_Ip_StatusType(* Qspi_Ip_ResetCalloutPtrType) (uint32 instance)
```

Reset callout pointer type.

Definition at line 274 of file Qspi_Ip_Types.h.

6.2.4.4 Qspi_Ip_ErrorCheckCalloutPtrType

```
typedef Qspi_Ip_StatusType(* Qspi_Ip_ErrorCheckCalloutPtrType) (uint32 instance)
```

Error Check callout pointer type.

Definition at line 278 of file Qspi_Ip_Types.h.

6.2.4.5 Qspi_Ip_EccCheckCalloutPtrType

```
typedef Qspi_Ip_StatusType(* Qspi_Ip_EccCheckCalloutPtrType) (uint32 instance, uint32 startAddress, uint32 dataLength)
```

Ecc Check callout pointer type.

Definition at line 282 of file Qspi_Ip_Types.h.

6.2.5 Enum Reference

6.2.5.1 Qspi_Ip_StatusType

```
enum Qspi_Ip_StatusType
```

qspi return codes

Enumerator

STATUS_QSPI_IP_SUCCESS	Successful job
STATUS_QSPI_IP_ERROR	IP is performing an operation
STATUS_QSPI_IP_BUSY	Error - general code
STATUS_QSPI_IP_TIMEOUT	Error - exceeded timeout
STATUS_QSPI_IP_ERROR_PROGRAM_VERIFY	Error - selected memory area doesn't contain desired value

Definition at line 137 of file Qspi_Ip_Types.h.

6.2.5.2 Qspi_Ip_ConnectionType

```
enum Qspi_Ip_ConnectionType
```

flash connection to the QSPI module

Enumerator

QSPI_IP_SIDE_A1	Serial flash connected on side A1
QSPI_IP_SIDE_A2	Serial flash connected on side A2
QSPI_IP_SIDE_B1	Serial flash connected on side B1
QSPI_IP_SIDE_B2	Serial flash connected on side B2

Definition at line 149 of file Qspi_Ip_Types.h.

6.2.5.3 Qspi_Ip_OpType

```
enum Qspi_Ip_OpType
```

flash operation type

Enumerator

QSPI_IP_OP_TYPE_CMD	Simple command
QSPI_IP_OP_TYPE_WRITE_REG	Write value in external flash register
QSPI_IP_OP_TYPE_RMW_REG	RMW command on external flash register
QSPI_IP_OP_TYPE_READ_REG	Read external flash register until expected value is read
QSPI_IP_OP_TYPE_QSPI_CFG	Re-configure QSPI controller

Definition at line 160 of file Qspi_Ip_Types.h.

6.2.5.4 Qspi_Ip_LutCommandsType

enum [Qspi_Ip_LutCommandsType](#)

Lut commands.

Enumerator

QSPI_IP_LUT_INSTR_STOP	End of sequence
QSPI_IP_LUT_INSTR_CMD	Command
QSPI_IP_LUT_INSTR_ADDR	Address
QSPI_IP_LUT_INSTR_DUMMY	Dummy cycles
QSPI_IP_LUT_INSTR_MODE	8-bit mode
QSPI_IP_LUT_INSTR_MODE2	2-bit mode
QSPI_IP_LUT_INSTR_MODE4	4-bit mode
QSPI_IP_LUT_INSTR_READ	Read data
QSPI_IP_LUT_INSTR_WRITE	Write data
QSPI_IP_LUT_INSTR_JMP_ON_CS	Jump on chip select deassert and stop
QSPI_IP_LUT_INSTR_ADDR_DDR	Address - DDR mode
QSPI_IP_LUT_INSTR_MODE_DDR	8-bit mode - DDR mode
QSPI_IP_LUT_INSTR_MODE2_DDR	2-bit mode - DDR mode

Enumerator

QSPI_IP_LUT_INSTR_MODE4_DDR	4-bit mode - DDR mode
QSPI_IP_LUT_INSTR_READ_DDR	Read data - DDR mode
QSPI_IP_LUT_INSTR_WRITE_DDR	Write data - DDR mode
QSPI_IP_LUT_INSTR_DATA_LEARN	Data learning pattern
QSPI_IP_LUT_INSTR_CMD_DDR	Command - DDR mode
QSPI_IP_LUT_INSTR_CADDR	Column address
QSPI_IP_LUT_INSTR_CADDR_DDR	Column address - DDR mode
QSPI_IP_LUT_INSTR_JMP_TO_SEQ	Jump on chip select deassert and continue

Definition at line 171 of file Qspi_Ip_Types.h.

6.2.5.5 Qspi_Ip_LutPadsType

enum [Qspi_Ip_LutPadsType](#)

Lut pad options.

Enumerator

QSPI_IP_LUT_PADS↔ _1	1 Pad
QSPI_IP_LUT_PADS↔ _2	2 Pads
QSPI_IP_LUT_PADS↔ _4	4 Pads
QSPI_IP_LUT_PADS↔ _8	8 Pads

Definition at line 198 of file Qspi_Ip_Types.h.

6.2.5.6 Qspi_Ip_ReadModeType

enum [Qspi_Ip_ReadModeType](#)

Read mode.

Enumerator

QSPI_IP_READ_MODE_INTERNAL_DQS	Use internally generated strobe signal
QSPI_IP_READ_MODE_LOOPBACK	Use loopback clock from PAD as strobe signal

Enumerator

QSPI_IP_READ_MODE_LOOPBACK_DQS	Use loopback clock from PAD as strobe signal
QSPI_IP_READ_MODE_EXTERNAL_DQS	Use external strobe signal

Definition at line 219 of file Qspi_Ip_Types.h.

6.2.5.7 Qspi_Ip_DataRateType

enum [Qspi_Ip_DataRateType](#)

Clock phase used for sampling Rx data.

Enumerator

QSPI_IP_DATA_RATE_SDR	Single data rate
QSPI_IP_DATA_RATE_DDR	Double data rate

Definition at line 236 of file Qspi_Ip_Types.h.

6.2.5.8 Qspi_Ip_SampleDelayType

enum [Qspi_Ip_SampleDelayType](#)

Delay used for sampling Rx data.

Enumerator

QSPI_IP_SAMPLE_DELAY_SAME_DQS	Same DQS
QSPI_IP_SAMPLE_DELAY_HALFCYCLE_EARLY_DQS	Half-cycle early DQS

Definition at line 245 of file Qspi_Ip_Types.h.

6.2.5.9 Qspi_Ip_SamplePhaseType

enum [Qspi_Ip_SamplePhaseType](#)

Clock phase used for sampling Rx data.

Enumerator

QSPI_IP_SAMPLE_PHASE_NON_INVERTED	Sampling at non-inverted clock
QSPI_IP_SAMPLE_PHASE_INVERTED	Sampling at inverted clock

Definition at line 253 of file Qspi_Ip_Types.h.

6.2.5.10 Qspi_Ip_FlashDataAlignType

enum [Qspi_Ip_FlashDataAlignType](#)

Alignment of outgoing data with serial clock.

Enumerator

QSPI_IP_FLASH_DATA_ALIGN_REFCLK	Data aligned with the posedge of Internal reference clock of QSPI
QSPI_IP_FLASH_DATA_ALIGN_2X_REFCLK	Data aligned with 2x serial flash half clock

Definition at line 261 of file Qspi_Ip_Types.h.

6.2.5.11 Qspi_Ip_ClockSourceType

enum [Qspi_Ip_ClockSourceType](#)

Source of QuadSPI AHB read interface, module and bus interface clock.

Enumerator

QSPI_IP_CLK_SRC_SYS_CLK	FIRC_DIV1 is clock source of QuadSPI internal reference clock
QSPI_IP_CLK_SRC_BUS_CLK	PLL_DIV1 is clock source of QuadSPI internal reference clock

Definition at line 306 of file Qspi_Ip_Types.h.

6.2.5.12 Qspi_Ip_ClockReferenceType

enum [Qspi_Ip_ClockReferenceType](#)

Source of QuadSPI internal reference clock.

Enumerator

QSPI_IP_CLK_REF_PLL_DIV1	PLL_DIV1 is clock source of QuadSPI internal reference clock
QSPI_IP_CLK_REF_FIRC_DIV1	FIRC_DIV1 is clock source of QuadSPI internal reference clock

Definition at line 314 of file Qspi_Ip_Types.h.

6.2.5.13 Qspi_Ip_LastCommandType

enum Qspi_Ip_LastCommandType

Last command that was executed by the device flash.

Definition at line 452 of file Qspi_Ip_Types.h.

6.2.5.14 Qspi_Ip_FlashMemoryType

enum Qspi_Ip_FlashMemoryType

Parameter memory type.

Enumerator

QSPI_IP_SERIAL_FLASH	Traditional xSPI devices
----------------------	--------------------------

Definition at line 513 of file Qspi_Ip_Types.h.

6.2.6 Function Reference

6.2.6.1 Qspi_Ip_Init()

```
Qspi_Ip_StatusType Qspi_Ip_Init (
    uint32 instance,
    const Qspi_Ip_MemoryConfigType * pConfig,
    const Qspi_Ip_MemoryConnectionType * pConnect )
```

Initializes the serial flash memory driver.

This function initializes the external flash driver and prepares it for operation.

Parameters

<i>instance</i>	External flash instance number
<i>pConfig</i>	Pointer to the driver configuration structure.
<i>pConnect</i>	Pointer to the flash device connection structure.

Returns

Error or success status returned by API

6.2.6.2 Qspi_Ip_Deinit()

```
Qspi_Ip_StatusType Qspi_Ip_Deinit (
    uint32 instance )
```

De-initializes the serial flash memory driver.

This function de-initializes the qspi driver. The driver can't be used again until reinitialized. The state structure is no longer needed by the driver and may be freed after calling this function.

Parameters

<i>instance</i>	External flash instance number
-----------------	--------------------------------

Returns

Error or success status returned by API

6.2.6.3 Qspi_Ip_EraseBlock()

```
Qspi_Ip_StatusType Qspi_Ip_EraseBlock (  
    uint32 instance,  
    uint32 address,  
    uint32 size )
```

Erase a sector in the serial flash.

This function performs one erase sector (block) operation on the external flash. The erase size must match one of the device's erase types.

Parameters

<i>instance</i>	External flash instance number
<i>address</i>	Address of sector to be erased
<i>size</i>	Size of the sector to be erase. The sector size must match one of the supported erase sizes of the device.

Returns

Error or success status returned by API

6.2.6.4 Qspi_Ip_EraseChip()

```
Qspi_Ip_StatusType Qspi_Ip_EraseChip (  
    uint32 instance )
```

Erase the entire serial flash.

Parameters

<i>instance</i>	External flash instance number
-----------------	--------------------------------

Returns

Error or success status returned by API

6.2.6.5 Qspi_Ip_GetMemoryStatus()

```
Qspi_Ip_StatusType Qspi_Ip_GetMemoryStatus (  
    uint32 instance )
```

Check the status of the flash device.

Parameters

<i>instance</i>	External flash instance number
-----------------	--------------------------------

Returns

Error or success status returned by API

6.2.6.6 Qspi_Ip_SetProtection()

```
Qspi_Ip_StatusType Qspi_Ip_SetProtection (
    uint32 instance,
    uint8 value )
```

Sets the protection bits to the requested value.

Parameters

<i>instance</i>	External flash instance number
<i>value</i>	New value for the protection bits

Returns

Error or success status returned by API

6.2.6.7 Qspi_Ip_GetProtection()

```
Qspi_Ip_StatusType Qspi_Ip_GetProtection (
    uint32 instance,
    uint8 * value )
```

Returns the current value of the protection bits.

Parameters

<i>instance</i>	External flash instance number
<i>value</i>	Current value of the protection bits

Returns

Error or success status returned by API

6.2.6.8 Qspi_Ip_Reset()

```
Qspi_Ip_StatusType Qspi_Ip_Reset (
    uint32 instance )
```

Resets the flash device.

Parameters

<i>instance</i>	External flash instance number
-----------------	--------------------------------

Returns

Error or success status returned by API

6.2.6.9 Qspi_Ip_Enter0XX()

```
Qspi_Ip_StatusType Qspi_Ip_Enter0XX (
    uint32 instance )
```

Enters 0-X-X (no command) mode. This mode assumes only reads are performed.

Parameters

<i>instance</i>	External flash instance number
-----------------	--------------------------------

Returns

Error or success status returned by API

6.2.6.10 Qspi_Ip_Exit0XX()

```
Qspi_Ip_StatusType Qspi_Ip_Exit0XX (
    uint32 instance )
```

Exits 0-X-X (no command) mode. This allows operations other than reads to be performed.

Parameters

<i>instance</i>	External flash instance number
-----------------	--------------------------------

Returns

Error or success status returned by API

6.2.6.11 Qspi_Ip_ProgramSuspend()

```
Qspi_Ip_StatusType Qspi_Ip_ProgramSuspend (
    uint32 instance )
```

Suspends a program operation.

Parameters

<i>instance</i>	External flash instance number
-----------------	--------------------------------

Returns

Error or success status returned by API

6.2.6.12 Qspi_Ip_ProgramResume()

```
Qspi_Ip_StatusType Qspi_Ip_ProgramResume (
```

```
uint32 instance )
```

Resumes a program operation.

Parameters

<i>instance</i>	External flash instance number
-----------------	--------------------------------

Returns

Error or success status returned by API

6.2.6.13 Qspi_Ip_EraseSuspend()

```
Qspi_Ip_StatusType Qspi_Ip_EraseSuspend (
    uint32 instance )
```

Suspends an erase operation.

Parameters

<i>instance</i>	External flash instance number
-----------------	--------------------------------

Returns

Error or success status returned by API

6.2.6.14 Qspi_Ip_EraseResume()

```
Qspi_Ip_StatusType Qspi_Ip_EraseResume (
    uint32 instance )
```

Resumes an erase operation.

Parameters

<i>instance</i>	External flash instance number
-----------------	--------------------------------

Returns

Error or success status returned by API

6.2.6.15 Qspi_Ip_Read()

```
Qspi_Ip_StatusType Qspi_Ip_Read (
    uint32 instance,
    uint32 address,
    uint8 * data,
    uint32 size )
```

Read data from serial flash.

Parameters

<i>instance</i>	External flash instance number
<i>address</i>	Start address for read operation
<i>data</i>	Buffer where to store read data
<i>size</i>	Size of data buffer

Returns

Error or success status returned by API

6.2.6.16 Qspi_Ip_ReadId()

```
Qspi_Ip_StatusType Qspi_Ip_ReadId (
    uint32 instance,
    uint8 * data )
```

Read manufacturer ID/device ID from serial flash.

Parameters

<i>instance</i>	External flash instance number
<i>data</i>	Buffer where to store read data. Buffer size must match ReadId initialization settings.

Returns

Error or success status returned by API

6.2.6.17 Qspi_Ip_ProgramVerify()

```
Qspi_Ip_StatusType Qspi_Ip_ProgramVerify (
    uint32 instance,
    uint32 address,
    const uint8 * data,
    uint32 size )
```

Verifies the correctness of the programmed data.

Parameters

<i>instance</i>	External flash instance number
<i>address</i>	Start address of area to be verified
<i>data</i>	Data to be verified
<i>size</i>	Size of area to be verified

Returns

Error or success status returned by API

6.2.6.18 Qspi_Ip_EraseVerify()

```
Qspi_Ip_StatusType Qspi_Ip_EraseVerify (
    uint32 instance,
    uint32 address,
    uint32 size )
```

Checks whether or not an area in the serial flash is erased.

Parameters

<i>instance</i>	External flash instance number
<i>address</i>	Start address of area to be verified
<i>size</i>	Size of area to be verified

Returns

Error or success status returned by API

6.2.6.19 Qspi_Ip_Program()

```
Qspi_Ip_StatusType Qspi_Ip_Program (
    uint32 instance,
    uint32 address,
    const uint8 * data,
    uint32 size )
```

Writes data in serial flash.

Writes data in serial flash memory then exits (Async mode) The status of the flash memory must be verified by calling asynchronously the Qspi_Ip_GetMemoryStatus function until it is not busy, meaning that the write operation is complete. The maximum supported size is equal to the Qspi hardware TxBuffer size.

Parameters

<i>instance</i>	External flash instance number
<i>address</i>	Start address of area to be programmed
<i>data</i>	Data to be programmed in flash
<i>size</i>	Size of data buffer

Returns

Error or success status returned by API

6.2.6.20 Qspi_Ip_RunCommand()

```
Qspi_Ip_StatusType Qspi_Ip_RunCommand (
    uint32 instance,
```

```
uint16 lut,
uint32 addr )
```

Launches a simple command for the serial flash.

Parameters

<i>instance</i>	External flash instance number
<i>lut</i>	Index of command in virtual LUT
<i>addr</i>	Address used in the command, or base address of the target serial flash

Returns

Error or success status returned by API

6.2.6.21 Qspi_Ip_RunReadCommand()

```
Qspi_Ip_StatusType Qspi_Ip_RunReadCommand (
    uint32 instance,
    uint16 lut,
    uint32 addr,
    uint8 * dataRead,
    const uint8 * dataCmp,
    uint32 size )
```

Launches a read command for the serial flash.

This function can launch a read command in 3 modes:

- normal read (dataRead != NULL_PTR): Data is read from serial flash and placed in the buffer
- verify (dataRead == NULL_PTR, dataCmp != NULL_PTR): Data is read from serial flash and compared to the reference buffer
- blank check (dataRead == NULL_PTR, dataCmp == NULL_PTR): Data is read from serial flash and compared to 0xFF

Parameters

<i>instance</i>	External flash instance number
<i>lut</i>	Index of command in virtual LUT
<i>addr</i>	Start address for read operation in serial flash
<i>dataRead</i>	Buffer where to store read data
<i>dataCmp</i>	Buffer to be compared to read data
<i>size</i>	Size of data buffer

Returns

Error or success status returned by API

6.2.6.22 Qspi_Ip_RunWriteCommand()

```
Qspi_Ip_StatusType Qspi_Ip_RunWriteCommand (
    uint32 instance,
```

```
uint16 lut,
uint32 addr,
const uint8 * data,
uint32 size )
```

Launches a write command for the serial flash.

Parameters

<i>instance</i>	External flash instance number
<i>lut</i>	Index of command in virtual LUT
<i>addr</i>	Start address for write operation in serial flash
<i>data</i>	Data to be programmed in flash
<i>size</i>	Size of data buffer

Returns

Error or success status returned by API

6.2.6.23 Qspi_Ip_AhbReadEnable()

```
Qspi_Ip_StatusType Qspi_Ip_AhbReadEnable (
    uint32 instance )
```

Sets up AHB reads to the serial flash.

Parameters

<i>instance</i>	External flash instance number
-----------------	--------------------------------

Returns

Error or success status returned by API

6.2.6.24 Qspi_Ip_ControllerGetStatus()

```
Qspi_Ip_StatusType Qspi_Ip_ControllerGetStatus (
    uint32 instance )
```

Check the status of the QSPI controller.

Parameters

<i>instance</i>	QSPI peripheral instance number
-----------------	---------------------------------

Returns

Error or success status returned by API

6.2.6.25 Qspi_Ip_ControllerInit()

```
Qspi_Ip_StatusType Qspi_Ip_ControllerInit (
```

```
uint32 instance,
const Qspi_Ip_ControllerConfigType * userConfigPtr )
```

Initializes the qspi driver.

This function initializes the qspi driver and prepares it for operation.

Parameters

<i>instance</i>	QSPI peripheral instance number
<i>userConfigPtr</i>	Pointer to the qspi configuration structure.

Returns

Error or success status returned by API

6.2.6.26 Qspi_Ip_ControllerDeinit()

```
Qspi_Ip_StatusType Qspi_Ip_ControllerDeinit (
    uint32 instance )
```

De-initialize the qspi driver.

This function de-initializes the qspi driver. The driver can't be used again until reinitialized. The context structure is no longer needed by the driver and can be freed after calling this function.

Parameters

<i>instance</i>	QSPI peripheral instance number
-----------------	---------------------------------

Returns

Error or success status returned by API

6.2.6.27 Qspi_Ip_Abort()

```
Qspi_Ip_StatusType Qspi_Ip_Abort (
    uint32 instance )
```

Aborts any on-going transactions.

Force the Qspi controller to cancel the on-going IP transaction by performing the software reset sequence.

Parameters

<i>instance</i>	QSPI peripheral instance number
-----------------	---------------------------------

Returns

Error or success status returned by API

6.2.6.28 Qspi_Ip_ReadSfdp()

```
Qspi_Ip_StatusType Qspi_Ip_ReadSfdp (
    Qspi_Ip_MemoryConfigType * pConfig,
    const Qspi_Ip_MemoryConnectionType * pConnect )
```

Initializes the serial flash memory configuration from SFDP table.

This function uses the information in the SFDP table to auto-fill the memory configuration structure.

Parameters

<i>pConfig</i>	Pointer to the driver configuration structure.
<i>pConnect</i>	Pointer to the flash device connection structure.

Returns

Error or success status returned by API

6.2.6.29 Qspi_Ip_SetLut()

```
void Qspi_Ip_SetLut (
    uint32 instance,
    uint8 LutRegister,
    Qspi_Ip_InstrOpType operation0,
    Qspi_Ip_InstrOpType operation1 )
```

Configures LUT commands.

This function configures a pair of LUT commands in the specified LUT register. LUT sequences start at index multiple of 4 and can have up to 8 commands

Parameters

<i>instance</i>	QuadSPI peripheral instance number
<i>LutRegister</i>	Index in physical LUT array
<i>operation0</i>	First operation
<i>operation1</i>	Second operation Implements Qspi_Ip_SetLut_Activity

6.2.6.30 Qspi_Ip_WriteLuts_Privileged()

```
void Qspi_Ip_WriteLuts_Privileged (
    uint32 Instance,
    uint8 StartLutRegister,
    const uint32 * Data,
    uint8 Size )
```

Configures LUT commands.

This function configures pairs of LUT commands from the specified LUT register.

Parameters

<i>Instance</i>	QuadSPI peripheral instance number
<i>StartLutRegister</i>	Start index in physical LUT array
<i>Data</i>	Data to be written in LUT register
<i>Size</i>	Size of data buffer

6.2.6.31 Qspi_Ip_SetAhbSeqId_Privileged()

```
void Qspi_Ip_SetAhbSeqId_Privileged (
    uint32 instance,
    uint8 seqID )
```

Sets sequence ID for AHB operations.

Parameters

<i>instance</i>	QuadSPI peripheral instance number
<i>seqID</i>	Sequence ID in LUT for read operation Implements Qspi_Ip_SetAhbSeqId_Activity

6.2.6.32 Qspi_Ip_GetBaseAddress()

```
uint32 Qspi_Ip_GetBaseAddress (
    uint32 instance,
    Qspi_Ip_ConnectionType connectionType )
```

Returns the physical base address of a flash device.

This function returns the physical base address of a flash device, depending on the QSPI connection. The controller must be initialized prior to calling this function.

Parameters

<i>instance</i>	QuadSPI peripheral instance number
<i>connectionType</i>	Connection of the flash device to QSPI

6.2.6.33 Qspi_Ip_IpCommand()

```
Qspi_Ip_StatusType Qspi_Ip_IpCommand (
    uint32 instance,
    uint8 SeqId,
    uint32 addr )
```

Launches a simple IP command.

Parameters

<i>instance</i>	QuadSPI peripheral instance number
<i>SeqId</i>	Sequence ID where the command is to be found
<i>addr</i>	Address of the target serial flash

Returns

Error or success status returned by API

6.2.6.34 Qspi_Ip_IpRead()

```
Qspi_Ip_StatusType Qspi_Ip_IpRead (
    uint32 instance,
    uint8 SeqId,
    uint32 addr,
    uint8 * dataRead,
    const uint8 * dataCmp,
    uint32 size )
```

Launches an IP read command.

This function can launch a read command in 3 modes:

- normal read (dataRead != NULL_PTR): Data is read from serial flash and placed in the buffer
- verify (dataRead == NULL_PTR, dataCmp != NULL_PTR): Data is read from serial flash and compared to the reference buffer
- blank check (dataRead == NULL_PTR, dataCmp == NULL_PTR): Data is read from serial flash and compared to 0xFF Only normal read mode can use DMA.

Parameters

<i>instance</i>	QuadSPI peripheral instance number
<i>SeqId</i>	Sequence ID where the command is to be found
<i>addr</i>	Start address for read operation in serial flash
<i>dataRead</i>	Buffer where to store read data
<i>dataCmp</i>	Buffer to be compared to read data
<i>size</i>	Size of data buffer

Returns

Error or success status returned by API

6.2.6.35 Qspi_Ip_IpWrite()

```
Qspi_Ip_StatusType Qspi_Ip_IpWrite (
    uint32 instance,
    uint8 SeqId,
    uint32 addr,
    const uint8 * data,
    uint32 size )
```


Module Documentation

Launches an IP write command.

Parameters

<i>instance</i>	QuadSPI peripheral instance number
<i>SeqId</i>	Sequence ID where the command is to be found
<i>addr</i>	Start address for write operation in serial flash
<i>data</i>	Data to be programmed in flash
<i>size</i>	Size of data buffer

Returns

Error or success status returned by API

6.3 FTFC IP Driver

6.3.1 Detailed Description

Ftfc_Fls_Ip_Types.h_Artifact

Data Structures

- struct [Fls_ExceptionDetailsType](#)
Detailed information on the exception. [More...](#)
- struct [Ftfc_ConfigType](#)
Ftfc Configuration Structure. [More...](#)

Macros

- #define [FLASH_CMD_PROGRAM_PHRASE](#)
FCCOB commands IDs.
- #define [FTFC_WRITE_DOUBLE_WORD](#)
Program allignment.
- #define [FLS_SIZE_1BYTE](#)
the number of bytes uses to compare (1 byte).
- #define [FLS_SIZE_2BYTE](#)
the number of bytes uses to compare (2 bytes).
- #define [FLS_SIZE_4BYTE](#)
the number of bytes uses to compare (4 bytes).

Types Reference

- typedef uint8 [Fls_CompHandlerReturnType](#)
Return value of Fls handler function.
- typedef void(* [Ftfc_StartFlashAccessNotifPtrType](#)) (void)
Fls Start Flash Access Notification Pointer Type.
- typedef void(* [Ftfc_FinishedFlashAccessNotifPtrType](#)) (void)
Fls Finished Flash Access Notification Pointer Type.

Enum Reference

- enum [Ftfc_Fls_Ip_FlashBlocksNumberType](#)
Enumeration of Blocks of memory flash .
- enum [Ftfc_Fls_Ip_StatusType](#)
Enumeration of checking status errors or not.

Function Reference

- [Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_Init](#) (const [Ftfc_ConfigType](#) *Ftfc_Fls_Ip_pInitConfig)
Initializes the FTFC module.
- [Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_Abort](#) (void)

Abort a program or erase operation.

- [Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_Read](#) (Ftfc_Fls_Ip_AddressType u32SrcAddress, uint8 *pDest←AddressPtr, Ftfc_Fls_Ip_LengthType u32Length)

This function fills data to pDestAddressPtr.

- [Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_Compare](#) (Ftfc_Fls_Ip_AddressType u32SrcAddress, const uint8 *pCompareAddressPtr, Ftfc_Fls_Ip_LengthType u32Length)

Checks that there is the desired data at the specified address.

- [Ftfc_Fls_Ip_FlashBlocksNumberType Ftfc_Fls_Ip_GetBlockNumberFromAddress](#) (uint32 u32Target←Address)

Get block number from target address.

- [Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_SectorErase](#) (uint32 u32SectorStartAddress)

Accepts and erases a selected program flash or data flash sector if possible.

- [Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_SectorEraseStatus](#) (void)

Checks the status of the hardware erase started by the Ftfc_Fls_Ip_SectorErase function.

- [Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_Write](#) (uint32 u32DestAddress, const uint8 *pSourceAddressPtr, uint32 u32Length)

Writes data into the memory array using the main interface. Initiates the hardware write and then exits.

- [Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_WriteStatus](#) (void)

Checks the status of the hardware program started by the FTFC_Ip_Write function.

- [Fls_CompHandlerReturnType Ftfc_Fls_Ip_DsiHandler](#) (const [Fls_ExceptionDetailsType](#) *pException←DetailsPtr)

Low level data storage exception handler.

- void [Ftfc_Fls_Ip_InvalidPrefetchBuff_Ram](#) (void)

Invalidate prefetch buffer before reading to make sure that the driver always reads the new data from flash.

6.3.2 Data Structure Documentation

6.3.2.1 struct Fls_ExceptionDetailsType

Detailed information on the exception.

The following information will be checked by the driver:

- if there is a pending read, compare,
- data_pt matches address currently accessed by pending flash read or flash compare job,
- if the exception syndrome register indicates DSI or MCI reason,

Definition at line 132 of file Ftfc_Fls_Ip_Types.h.

Data Fields

Fls_InstructionAddressType	instruction_pt	pointer to the instruction that generated the ECC
Fls_DataAddressType	data_pt	data address that caused the ECC error
uint32	syndrome_u32	details on the type of exception

6.3.2.2 struct Ftfc_ConfigType

Ftfc Configuration Structure.

Implements : Ftfc_ConfigType_Class

Definition at line 170 of file Ftfc_Fls_Ip_Types.h.

Data Fields

Ftfc_StartFlashAccessNotifPtrType	startFlashAccessNotifPtr	Pointer to start flash access callout
Ftfc_FinishedFlashAccessNotifPtrType	finishedFlashAccessNotifPtr	Pointer to finish flash access callout

6.3.3 Macro Definition Documentation

6.3.3.1 FLASH_CMD_PROGRAM_PHRASE

```
#define FLASH_CMD_PROGRAM_PHRASE
```

FCCOB commands IDs.

Definition at line 79 of file Ftfc_Fls_Ip_Types.h.

6.3.3.2 FTFC_WRITE_DOUBLE_WORD

```
#define FTFC_WRITE_DOUBLE_WORD
```

Program alignment.

Definition at line 85 of file Ftfc_Fls_Ip_Types.h.

6.3.3.3 FLS_SIZE_1BYTE

```
#define FLS_SIZE_1BYTE
```

the number of bytes uses to compare (1 byte).

Definition at line 91 of file Ftfc_Fls_Ip_Types.h.

6.3.3.4 FLS_SIZE_2BYTE

```
#define FLS_SIZE_2BYTE
```

the number of bytes uses to compare (2 bytes).

Definition at line 97 of file Ftfc_Fls_Ip_Types.h.

6.3.3.5 FLS_SIZE_4BYTE

```
#define FLS_SIZE_4BYTE
```

the number of bytes uses to compare (4 bytes).

Definition at line 103 of file Ftfc_Fls_Ip_Types.h.

6.3.4 Types Reference

6.3.4.1 Fls_CompHandlerReturnType

```
typedef uint8 Fls_CompHandlerReturnType
```

Return value of Fls handler function.

Fls_DsiHandler and Fls_MciHandler can return the following value:

- FLS_UNHANDLED Fls driver is not responsible for this situation
- FLS_HANDLED_SKIP Fls driver is responsible for this exception and want to skip this job

Definition at line 116 of file Ftfc_Fls_Ip_Types.h.

6.3.4.2 Ftfc_StartFlashAccessNotifPtrType

```
typedef void(* Ftfc_StartFlashAccessNotifPtrType) (void)
```

Fls Start Flash Access Notification Pointer Type.

Pointer type of Ftfc_StartFlashAccessNotifPtrType function

Definition at line 156 of file Ftfc_Fls_Ip_Types.h.

6.3.4.3 Ftfc_FinishedFlashAccessNotifPtrType

```
typedef void(* Ftfc_FinishedFlashAccessNotifPtrType) (void)
```

Fls Finished Flash Access Notification Pointer Type.

Pointer type of Ftfc_FinishedFlashAccessNotifPtrType function

Definition at line 163 of file Ftfc_Fls_Ip_Types.h.

6.3.5 Enum Reference

6.3.5.1 Ftfc_Fls_Ip_FlashBlocksNumberType

```
enum Ftfc_Fls_Ip_FlashBlocksNumberType
```

Enumeration of Blocks of memory flash .

Enumerator

FLS_CODE_BLOCK_0	code block number 0
FLS_CODE_BLOCK_1	code block number 1
FLS_CODE_BLOCK_2	code block number 2
FLS_DATA_BLOCK	data block

Enumerator

FLS_BLOCK_INVALID	invalid block
-------------------	---------------

Definition at line 142 of file Ftfc_Fls_Ip_Types.h.

6.3.5.2 Ftfc_Fls_Ip_StatusType

```
enum Ftfc_Fls_Ip_StatusType
```

Enumeration of checking status errors or not.

Enumerator

STATUS_FTFC_FLS_IP_SUCCESS	Successful job
STATUS_FTFC_FLS_IP_BUSY	IP is performing an operation
STATUS_FTFC_FLS_IP_ERROR	Error - general code
STATUS_FTFC_FLS_IP_ERROR_TIMEOUT	Error - exceeded timeout
STATUS_FTFC_FLS_IP_ERROR_INPUT_PARAMETER	Error - wrong input parameter
STATUS_FTFC_FLS_IP_ERROR_BLANK_CHECK	Error - selected memory area is not erased
STATUS_FTFC_FLS_IP_ERROR_PROGRAM_VERIFY	Error - selected memory area doesn't contain desired value
STATUS_FTFC_FLS_IP_ERROR_USER_TEST_BREAK_SBC	Error - single bit correction
STATUS_FTFC_FLS_IP_ERROR_USER_TEST_BREAK_DBD	Error - double bit detection
STATUS_FTFC_FLS_IP_SECTOR_UNPROTECTED	Checked sector is unlocked
STATUS_FTFC_FLS_IP_SECTOR_PROTECTED	Checked sector is locked

Definition at line 179 of file Ftfc_Fls_Ip_Types.h.

6.3.6 Function Reference

6.3.6.1 Ftfc_Fls_Ip_Init()

```
Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_Init (
    const Ftfc_ConfigType * Ftfc_Fls_Ip_pInitConfig )
```

Initializes the FTFC module.

This function will initialize ftfc module and clear all error flags.

Parameters

in	<i>Ftfc_Fls_Ip_pInitConfig</i>	Pointer to the driver configuration structure.
----	--------------------------------	--

Returns

Ftfc_Fls_Ip_StatusType

Return values

<i>STATUS_FTFC_FLS_IP_SUCCESS</i>	Initialization is success
<i>STATUS_FTFC_FLS_IP_ERROR_TIMEOUT</i>	Errors Timeout because wait for the Done bit long time

6.3.6.2 Ftfc_Fls_Ip_Abort()

```
Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_Abort (
    void )
```

Abort a program or erase operation.

This function will abort a program or erase operation in user mode and clear all PGM, APM, ERS, AERS, EHV, AEHV bits in MCR,AMCRS registers

Returns

Ftfc_Fls_Ip_StatusType

Return values

<i>STATUS_FTFC_FLS_IP_SUCCESS</i>	: The operation is successful.
<i>STATUS_FTFC_FLS_IP_ERROR_TIMEOUT</i>	the operation error because wait for the Done bit long time

6.3.6.3 Ftfc_Fls_Ip_Read()

```
Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_Read (
    Ftfc_Fls_Ip_AddressType u32SrcAddress,
    uint8 * pDestAddressPtr,
    Ftfc_Fls_Ip_LengthType u32Length )
```

This function fills data to pDestAddressPtr.

This function fills data to pDestAddressPtr with data from the specified address

Parameters

in	<i>u32SrcAddress</i>	The start address of the area to be read.
in	<i>pDestAddressPtr</i>	Pointer to the destination of the read.
in	<i>u32Length</i>	Read size

Returns

Ftfc_Fls_Ip_StatusType

Return values

<i>STATUS_FTFC_FLS_IP_SUCCESS</i>	Read performed successfully.
-----------------------------------	------------------------------

Return values

<i>STATUS_FTFC_FLS_IP_ERROR_INPUT_PARAM</i>	Input parameters are invalid.
<i>STATUS_FTFC_FLS_IP_ERROR</i>	There was an error while reading.

Precondition

The module has to be initialized and not busy.

6.3.6.4 Ftfc_Fls_Ip_Compare()

```
Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_Compare (
    Ftfc_Fls_Ip_AddressType u32SrcAddress,
    const uint8 * pCompareAddressPtr,
    Ftfc_Fls_Ip_LengthType u32Length )
```

Checks that there is the desired data at the specified address.

Checks that there is the desired data at the specified address. If the compare is intended to be a blank check, the pSourceAddressPtr should be NULL.

Parameters

in	<i>u32SrcAddress</i>	The start address of the area to be checked.
in	<i>pCompareAddressPtr</i>	Pointer to the data expected to be read.
in	<i>u32Length</i>	Check size

Returns

Ftfc_Fls_Ip_StatusType

Return values

<i>STATUS_FTFC_FLS_IP_SUCCESS</i>	Read performed successfully.
<i>STATUS_FTFC_FLS_IP_ERROR_INPUT_PARAM</i>	Input parameters are invalid.
<i>STATUS_FTFC_FLS_IP_ERROR</i>	There was an error while reading.
<i>STATUS_FTFC_FLS_IP_ERROR_PROGRAM_VERIFY</i>	The expected data was not found completely at the specified address

Precondition

The module has to be initialized and not busy.

6.3.6.5 Ftfc_Fls_Ip_GetBlockNumberFromAddress()

```
Ftfc_Fls_Ip_FlashBlocksNumberType Ftfc_Fls_Ip_GetBlockNumberFromAddress (
    uint32 u32TargetAddress )
```

Get block number from target address.

Get block number from target address

Parameters

in	<i>u32TargetAddress</i>	target address
----	-------------------------	----------------

Returns

Ftfc_Fls_Ip_GetBlockNumberFromAddress

Return values

<i>The</i>	block number which contains the target address.
------------	---

6.3.6.6 Ftfc_Fls_Ip_SectorErase()

```
Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_SectorErase (
    uint32 u32SectorStartAddress )
```

Accepts and erases a selected program flash or data flash sector if possible.

Accepts an erase job over one of the sectors if possible. Starts the high voltage erase and then exits. The status of the hardware erase must be verified by calling asynchronously the *Ftfc_Fls_Ip_SectorEraseStatus* function. The *Ftfc_Fls_Ip_SectorErase* function shall cover all the available sectors.

Parameters

in	<i>u32SectorStartAddress</i>	The start address of the sector to be erased.
----	------------------------------	---

Returns

Ftfc_Fls_Ip_StatusType

Return values

<i>STATUS_FTFC_FLS_IP_SUCCESS</i>	Hardware erase started successfully
<i>STATUS_FTFC_FLS_IP_ERROR_INPUT_PARAMETER</i>	The selected sector is out of bound
<i>STATUS_FTFC_FLS_IP_ERROR</i>	There is another job configured or in progress or The sector is locked by another core or couldn't be unlocked.
<i>STATUS_FTFC_FLS_IP_ERROR_TIMEOUT</i>	The erase operation exceeded the timeout - Status value available only if the timeout feature is enabled

Precondition

The module has to be initialized.

6.3.6.7 Ftfc_Fls_Ip_SectorEraseStatus()

```
Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_SectorEraseStatus (
    void )
```

Checks the status of the hardware erase started by the `Ftfc_Fls_Ip_SectorErase` function.

Checks the status of the hardware erase started by the `Ftfc_Fls_Ip_SectorErase` function.

Returns

`Ftfc_Fls_Ip_StatusType`

Return values

<i>STATUS_FTFC_FLS_IP_SUCCESS</i>	Erase performed successfully
<i>STATUS_Ftfc_Fls_Ip_BUSY</i>	Hardware erase is still in progress
<i>STATUS_FTFC_FLS_IP_ERROR</i>	There was an error during the hardware erase.
<i>STATUS_FTFC_FLS_IP_ERROR_TIMEOUT</i>	The erase operation exceeded the timeout - Status value available only if the timeout feature is enabled.
<i>STATUS_FTFC_FLS_IP_ERROR_BLANK_CHECK_ECK</i>	The sector was not erased correctly - Status value available only if the blank check feature is enabled

Precondition

The module has to be initialized.

6.3.6.8 Ftfc_Fls_Ip_Write()

```
Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_Write (
    uint32 u32DestAddress,
    const uint8 * pSourceAddressPtr,
    uint32 u32Length )
```

Writes data into the memory array using the main interface. Initiates the hardware write and then exits.

Writes data into the memory array using the main interface. Initiates the hardware write and then exits. the status of the hardware erase must be verified by calling asynchronously the `Ftfc_Fls_Ip_WriteStatus` function.

Parameters

in	<i>u32DestAddress</i>	The start address of the write, must be aligned with 8 bytes.
in	<i>pSourceAddressPtr</i>	Source program buffer address.
in	<i>u32Length</i>	Size in bytes of the flash region to be programmed, must be aligned with 8 bytes and the maximum value is 128 bytes.

Returns

`Ftfc_Fls_Ip_StatusType`

Return values

<i>STATUS_FTFC_FLS_IP_SUCCESS</i>	Program performed successfully
<i>STATUS_FTFC_FLS_IP_ERROR_INPUT_PARAMETERS</i>	The input parameters are invalid.
<i>STATUS_FTFC_FLS_IP_ERROR</i>	There is another job configured or in progress or The sector is locked by another core or couldn't be unlocked.

Return values

<i>STATUS_FTFC_FLS_IP_ERROR_TIMEOUT</i>	The erase operation exceeded the timeout - Status value available only if the timeout feature is enabled
---	--

Precondition

The module has to be initialized.

6.3.6.9 Ftfc_Fls_Ip_WriteStatus()

```
Ftfc_Fls_Ip_StatusType Ftfc_Fls_Ip_WriteStatus (
    void )
```

Checks the status of the hardware program started by the FTFC_Ip_Write function.

Checks the status of the hardware program started by the FTFC_Ip_Write function.

Returns

Ftfc_Fls_Ip_StatusType

Return values

<i>STATUS_FTFC_FLS_IP_SUCCESS</i>	Program performed successfully
<i>STATUS_Ftfc_Fls_Ip_BUSY</i>	Hardware program is still in progress
<i>STATUS_FTFC_FLS_IP_ERROR</i>	There was an error during the hardware program.
<i>STATUS_FTFC_FLS_IP_ERROR_TIMEOUT</i>	The program operation exceeded the timeout - Status value available only if the timeout feature is enabled.
<i>STATUS_FTFC_FLS_IP_ERROR_PROGRAM_↔ VERIFY</i>	The data was not written corectly into the memory - Status available only of program verify feature is enabled

Precondition

The module has to be initialized.

6.3.6.10 Ftfc_Fls_Ip_DsiHandler()

```
Fls_CompHandlerReturnType Ftfc_Fls_Ip_DsiHandler (
    const Fls_ExceptionDetailsType * pExceptionDetailsPtr )
```

Low level data storage exception handler.

Returns

Fls_CompHandlerReturnType

Return values

<i>FLS_HANDLED_SKIP</i>	The data storage exception was caused by currently pending flash read or compare job
<i>FLS_UNHANDLED</i>	The data storage exception was NOT caused by currently pending flash read or compare job

6.3.6.11 Ftfc_Fls_Ip_InvalidPrefetchBuff_Ram()

```
void Ftfc_Fls_Ip_InvalidPrefetchBuff_Ram (  
    void )
```

Invalidate prefetch buffer before reading to make sure that the driver always reads the new data from flash.

Parameters

in	<i>void</i>	
----	-------------	--

Return values

<i>none</i>	
-------------	--

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.